

Motion Synergy API User Manual

Table of Contents

Motion Synergy API User Manual	1
1 Introduction	6
2 Standalone Operation.....	7
2.1 First Time Startup.....	8
2.2 Re-configuring the System	10
2.3 Standalone Operation Software Components.....	10
2.4 Linux Support	11
3 Configuring the System	12
3.1 Configuring the Linear Units	14
3.2 Miscellaneous Global Settings	14
3.3 Emulation	15
3.4 Configuring the Communications Settings	15
3.4.1 Serial Communications Options	15
3.4.2 Using Multiple Communication Cables.....	16
3.4.3 CAN Communications Options	17
3.5 Configuring an Axis.....	17
3.5.1 Named Positions	19
3.5.2 Named Distances.....	19
3.5.3 Move Profiles	19
3.5.4 Time-based User Defined Profiles (UDP)	20
3.5.5 Coordinated Motion using Master-driven User Defined Profiles (UDP)	21
3.5.6 Sine-wave based User Defined Profiles (UDP)	22
3.5.7 Flags.....	23
3.5.8 Biquad Filters	23
3.5.9 Configuring the Axis Driver.....	25
3.5.10 Axis Driver Low Level Configuration	25
3.5.11 Configuring the Axis Driver Flag Signals	27
3.5.12 Configuring the Axis Driver Commutation Settings	28
3.5.13 Emulated Axis Driver Configuration	29
3.5.14 Configuring Trace Capture Profiles	30
3.5.15 Configuring Digital Inputs	30

3.6	Configuring the MotionSynergyGUI	31
3.6.1	GUI Commands.....	33
3.7	Configuring the Logger	36
3.7.1	Controlling the amount of information logged.	36
3.7.2	Log Sinks	37
3.7.3	Log Destination.....	38
3.7.4	Log Format String.....	38
3.7.5	Log Filter	38
3.7.6	Rotating Log Files.....	38
3.7.7	Logger Write Behaviour.....	39
3.7.8	More information	39
3.7.9	Default Logging Configuration.....	39
4	.NET Software Integration.....	40
4.1	Building and Running the C# Console Application	41
4.1.1	Windows	41
4.1.2	Linux.....	42
4.2	Building the MotionSynergyAPI	43
4.2.1	Windows	43
4.2.2	Linux.....	43
4.3	LabVIEW Software Integration.....	44
4.3.1	Running the Example LabVIEW Project.....	44
4.3.2	LabVIEW and Async Methods.....	46
4.3.3	LabVIEW Troubleshooting	46
4.4	Writing Scripts	46
4.5	Error Reporting.....	48
4.6	MotionSynergyAPI State Machines.....	48
4.6.1	MotionSynergyAPI State Machine	49
4.6.2	Connection State Machine	49
4.7	Public Interface	50
4.7.1	Alerts.Alert Class	50
4.7.2	Alerts.AlertsClearedEventArgs Class	52
4.7.3	Alerts.AlertDetails Class.....	52
4.7.4	Alerts.AlertInfo Class.....	53
4.7.5	Alerts.AlertRaisedEventArgs Class.....	54

4.7.6	<i>BiquadFilter Class</i>	55
4.7.7	<i>CommunicationSettings Class</i>	55
4.7.8	<i>ConnectionStateChangedEventArgs Class</i>	57
4.7.9	<i>DigitalInputEventArgs Class</i>	57
4.7.10	<i>FirstProduct Class</i>	58
4.7.11	<i>InstrumentResult Class</i>	59
4.7.12	<i>InstrumentSettings Class</i>	59
4.7.13	<i>InstrumentStateChangedEventArgs Class</i>	59
4.7.14	<i>InvalidEnumValueAttribute Class</i>	60
4.7.15	<i>InvalidEnumValueHelper Class</i>	60
4.7.16	<i>IsDeselectableEnumValueHelper Class</i>	61
4.7.17	<i>LogEntry Class</i>	61
4.7.18	<i>ProgressUpdate Class</i>	62
4.7.19	<i>UnDeselectableEnumValueAttribute Class</i>	62
4.7.20	<i>Alerts.IAlerts Interface</i>	62
4.7.21	<i>IAxis Interface</i>	63
4.7.22	<i>IConfiguration Interface</i>	78
4.7.23	<i>IConnectionState Interface</i>	81
4.7.24	<i>IDevice Interface</i>	82
4.7.25	<i>IDeviceEmulator Interface</i>	83
4.7.26	<i>IDiagnostics Interface</i>	84
4.7.27	<i>IDigitalInput Interface</i>	85
4.7.28	<i>IMotionControllerConfiguration Interface</i>	86
4.7.29	<i>IMotionSynergyAPI Interface</i>	91
4.7.30	<i>IScripting Interface</i>	95
4.7.31	<i>IUserDefinedProfile Interface</i>	95
4.7.32	<i>BiquadFilterDefinition Struct</i>	97
4.7.33	<i>BiquadFilterSettings Struct</i>	99
4.7.34	<i>CurrentLoopSettings Struct</i>	99
4.7.35	<i>ElectricalProperties Struct</i>	100
4.7.36	<i>FirmwareVersionInfo Struct</i>	101
4.7.37	<i>MotionErrorSettings Struct</i>	102
4.7.38	<i>MotionTrackingSettings Struct</i>	103
4.7.39	<i>PhaseAngleInfo Struct</i>	103

4.7.40	<i>PositionLoopSettings Struct</i>	104
4.7.41	<i>ScriptParameter Struct</i>	107
4.7.42	<i>MotionSynergyAPI Namespace</i>	107
4.7.43	<i>Alerts Namespace</i>	116
4.7.44	<i>Alert Codes</i>	118
5	Python Software Integration	120
5.1	Running the example Python Console Application	120
5.2	Python API	121
5.2.1	<i>Python and Async Methods</i>	121
6	C++ Software Integration	122
6.1	Building and Running the C++ Console Application	122
6.1.1	<i>Windows</i>	122
6.1.2	<i>Linux</i>	130
6.2	Public Interface	133
6.2.1	<i>imp::CommunicationSettings Class</i>	133
6.2.2	<i>imp::IAxisDevice Class</i>	133
6.2.3	<i>imp::ICCPDevice Class</i>	147
6.2.4	<i>imp::IDigitalInputDevice Class</i>	152
6.2.5	<i>imp::IDigitalIOPublisher Class</i>	152
6.2.6	<i>imp::IDigitalIOReader Class</i>	154
6.2.7	<i>imp::IMotionControllerConfiguration Class</i>	154
6.2.8	<i>imp::MotionErrorSettings Class</i>	158
6.2.9	<i>imp::MotionTrackingSettings Class</i>	158
6.2.10	<i>MotionSynergyAPI Class</i>	159
6.2.11	<i>imp::IAxisDevice::Movement Struct</i>	161
6.2.12	<i>imp Namespace</i>	162
7	Linux Support	170
7.1	Install Developer Tools	170
7.2	Linux Drivers	171
7.2.1	<i>Installing the IXXAT CAN driver for Raspberry Pi</i>	171
7.2.2	<i>Installing the MOXA Serial driver for Raspberry Pi</i>	172
8	Appendix	173
8.1	NVRAM Support	173
8.2	3rd-Party Software	176

8.3	PC Specification	176
8.4	Referenced Documents.....	176
8.5	Definitions	177
8.6	Revision History.....	177
8.7	Part Numbers	177
8.8	Support.....	177

1 Introduction

This document describes the Motion Synergy software libraries and applications used to control the *SmartStage™* product range (e.g. *SmartStage™ XY*, *SmartStage™ Linear*, *DOF-5*, etc) and *Dover Motion Control Module (DMCM)* product. The software can control the hardware as a stand-alone system or be integrated into a larger system. Various integration options are provided depending on the target software environment and required level of customization.

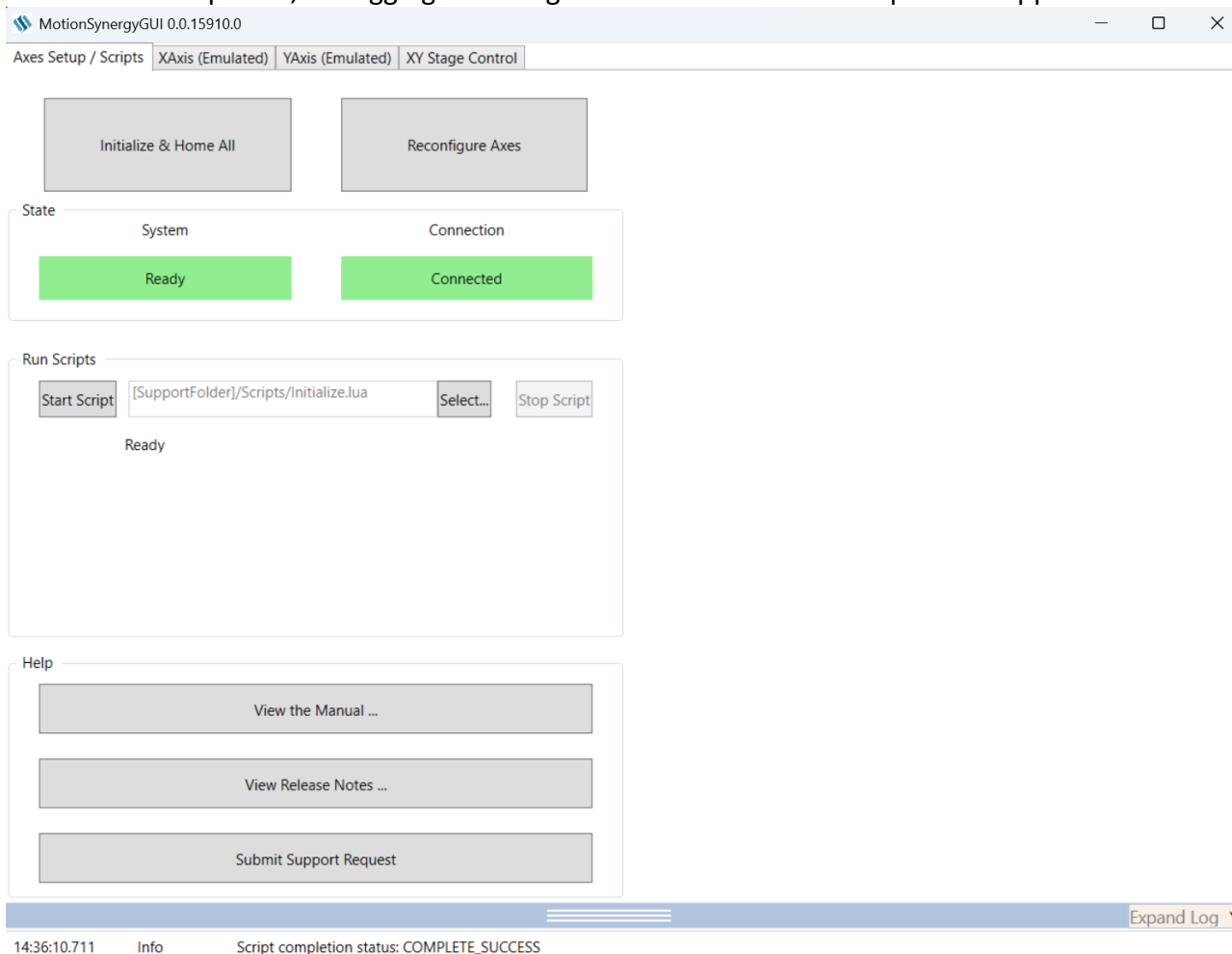
If you need help using the *MotionSynergyAPI* or *MotionSynergyGUI*, have discovered a bug or would like to request a new feature, you can submit a request using the [Request Software Support](#) online form.

2 Standalone Operation

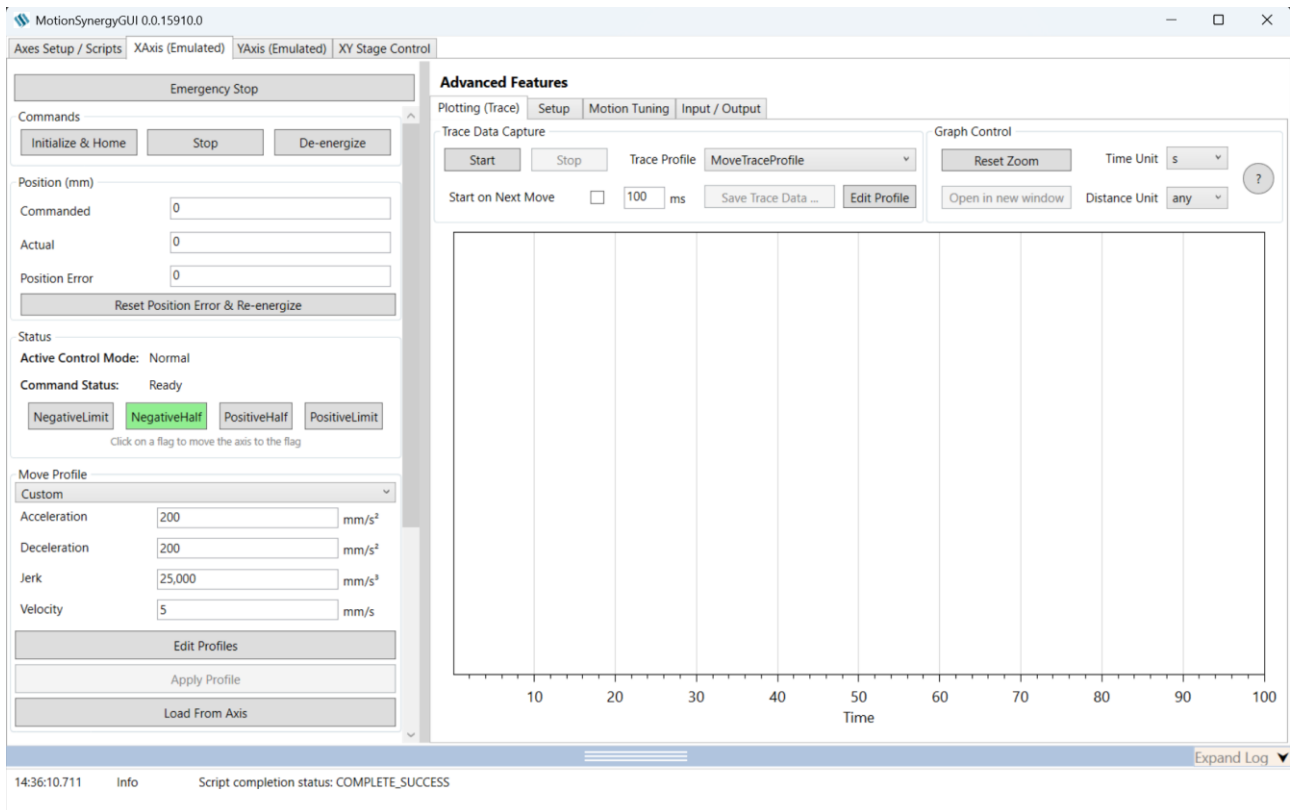
Prior to integrating the Dover Motion product(s) into a larger system, it's often useful to operate the hardware as a standalone subsystem. Common scenarios include:

- Product evaluation.
- Subsystem development prior to integration.
- Subsystem tuning and performance testing.

Dover Motion provides a *MotionSynergyGUI* application which supports both manual and scripted control of the hardware. Its extensive use of configuration files and scripting provides a great deal of flexibility without the need to setup a full software development environment. It's useful during software development, debugging and integration. The main axes setup screen appears as follows:

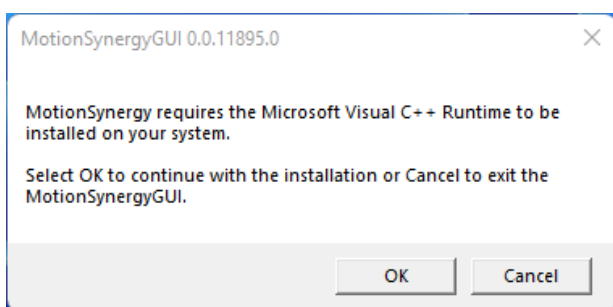


The main screen looks as follows when an axis tab is selected:



2.1 First Time Startup

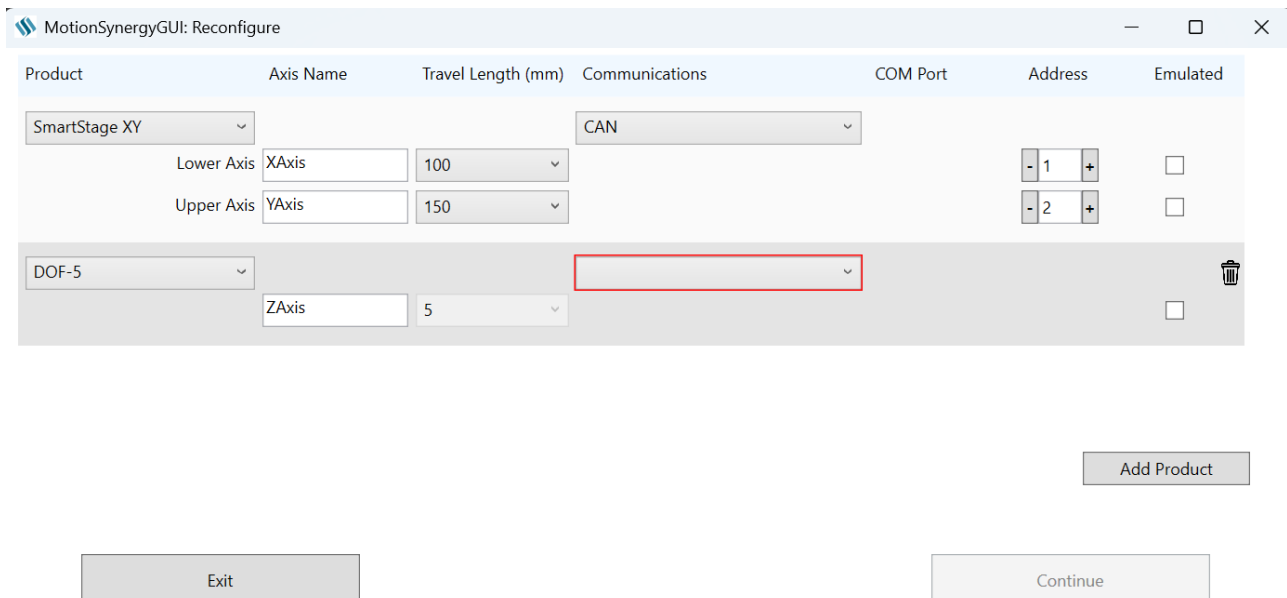
The first time you run the *MotionSynergyGUI* application, it will check to see if you have the [Microsoft Visual C++ Redistributable](#) installed. If it's not installed, you will be guided through the installation process. If it's already installed, you will proceed directly to the First Time Startup Window.



Select OK to continue with the installation:



The first time you run the *MotionSynergyGUI* application, you will be prompted to select your product(s) and configure various settings.



If you select *CAN* communications, it will check to see if the appropriate *IXXAT* driver has been installed. If it can't be detected, you will be given the option to install it.

If you select *RS232/RS485* communications, it will check to see if the appropriate *MOXA* driver has been installed. If it can't be detected, you will be given the option to install it.

Once you've selected the product and communications settings applicable to your hardware setup, it will use these settings each time you run the *MotionSynergyGUI* application and not prompt you again.

First time startup performs the following operations behind the scenes:

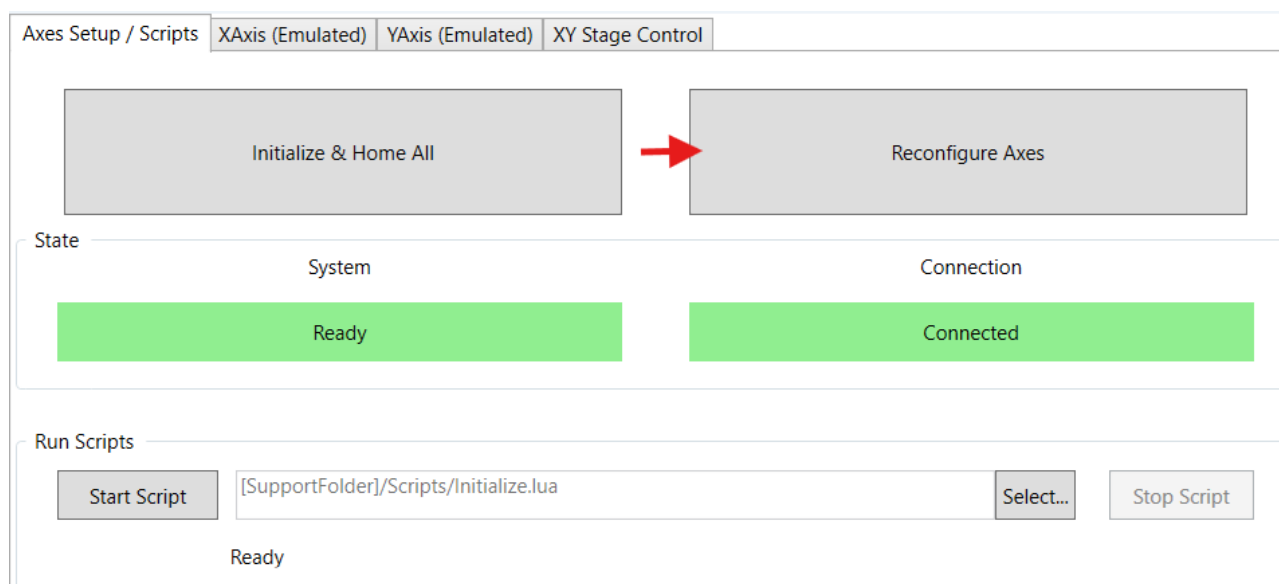
- Generates two configuration files (*MotionSynergyGUI.cfg* & *Instrument.cfg*) based on the *Templates/Configuration/*.cfg* files and the product(s) you have selected. These are copied into the *SupportFolder* directory.

- Generates a set of script files (*Initialize.lua* and one file for each axis (e.g.) *XAxis.lua*) based on the *Templates/Scripts/*.lua* files and the product(s) you have selected. These are copied into the *SupportFolder/Scripts* directory.
- These configuration and script files are the ones used by the *MotionSynergyGUI* and the example applications.

Note: There are many other configuration settings which can be customized. These are described in the section on 'Configuring the System'.

2.2 Re-configuring the System

You can re-configure the system at any time by clicking on the *Reconfigure Axes* button on the main axes setup screen.

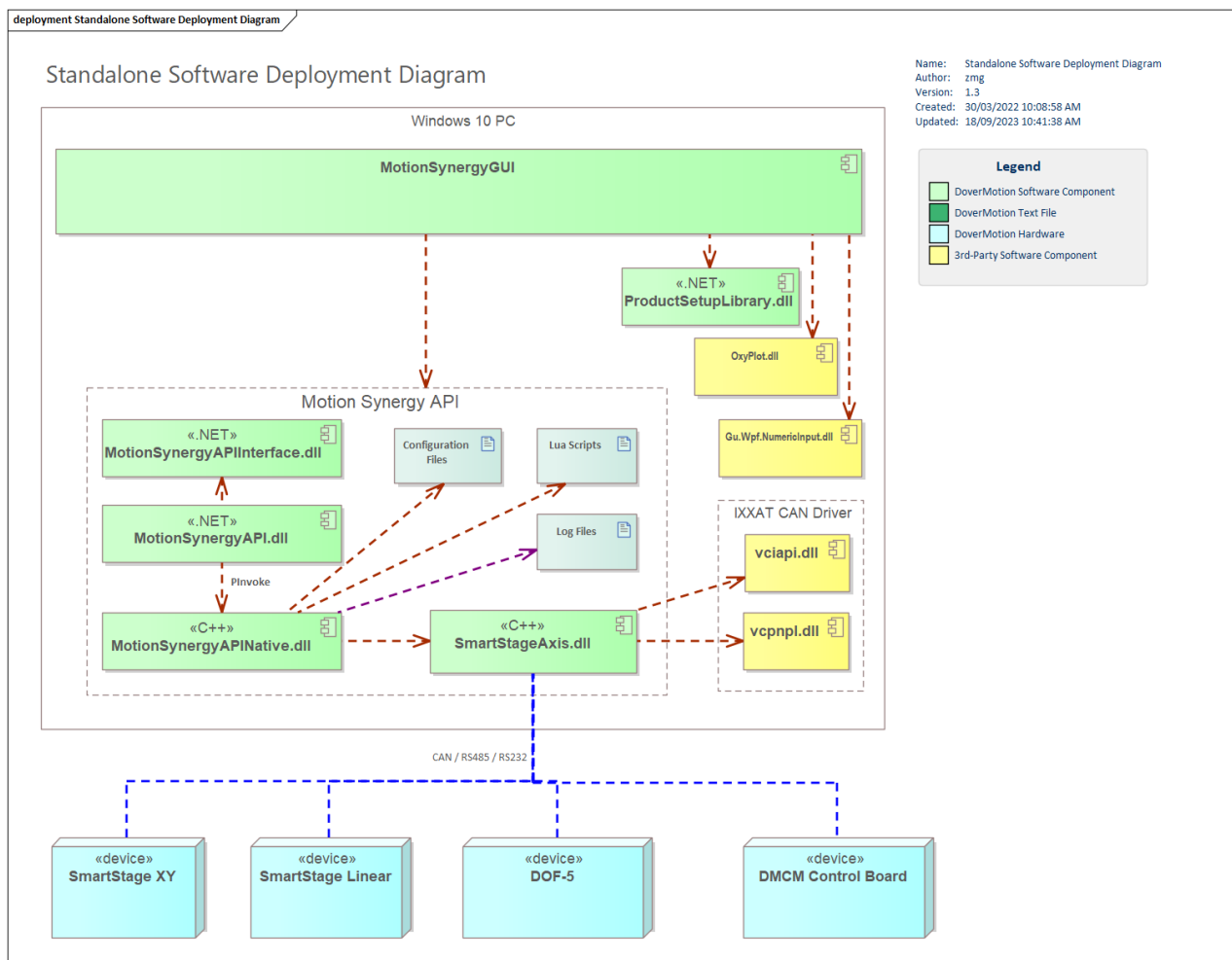


The next time you start the *MotionSynergyGUI* application, the configuration window will be displayed. This allows you to:

- Enable / disable emulation for individual axes.
- Rename axes.
- Change the communications settings for individual axes.
- Change the SmartStage XY axis travel lengths.
- Add / remove products.

2.3 Standalone Operation Software Components

The software components used during Standalone Operation are illustrated below:



2.4 Linux Support

Motion Synergy can be run on Linux-based operating systems. It has been tested with Ubuntu 22.04 LTS for x64 and arm64 platforms.

Motion Synergy for Linux is a separate software distribution and can be requested by contacting Dover Motion.

Motion Synergy for Linux uses the same codebase as the Windows release and supports all features except:

- The Motion Synergy GUI.
- LabView integration.

3 Configuring the System

Each Dover Motion product comes with an *Instrument.cfg* file that defines many settings used by the software to connect and control the hardware. Some of the things defined in configuration include:

- A definition for each axis that makes up the product (e.g. the SmarStage XY has an X and a Y axis, the DOF-5 has a single Z axis)
- The communication type and settings used to connect to the hardware (e.g. RS232, RS485 or CAN)
- Named values such as Positions, Distances, Flags and Movement Profiles which improve the readability and maintainability of the software as well as providing the ability to calibrate for manufacturing differences in end user products.
- The ability to turn on/off hardware emulation for one or more axis.
- The units to use (i.e. mm, um or nm).

While Dover Motion provides an *Instrument.cfg* file that works out of the box with a specific product, it is intended this default configuration file will be modified by the end user.

The *Instrument.cfg* file is a plain text file that uses the *Ini* file format. This format contains a series of sections (e.g. *[Instrument]*), each of which has one or more key/value pairs (e.g. *LinearUnits=Millimetres*). Blank lines and comment lines (starting with a ';') are ignored. The following illustrates the layout of a typical *Ini* file.

```
; Example Ini file
[Section1]
Setting1=Value1
Setting2=Value2
; Note this setting refers to another section. This is a common way of nesting settings within an Ini file.
Setting3=Section2

[Section2]
Setting1=Value1
Setting2=Value2
```

The *Instrument.cfg* file is organized as follows:

```
; -----
; Global settings used by MotionSynergyAPI to configure the system
[Instrument]
LinearUnits=Millimetres
; Other global settings.

; -----
; Communication Links

[SerialComms]
; RS232/RS485 Serial Communication settings

[CANComms]
; CAN Communication settings

; -----
; Products

[XAxis_Device]
```

```

; Axis settings and references to other sections

[XAxisDriver]
; Axis driver references to other driver configuration sections

[XAxisNamedPositions]
; A list of named positions the axis can be moved when performing an absolute move.

[XAxisNamedDistances]
; A list of named distance the axis can move when performing a relative move.

[XAxisHomingProfile]
; The move profile used when homing

[XAxisStandardMoveProfile]
; The move profile used once homed

[XAxisAdditionalMoveProfile1]
; Additional move profiles can be defined as needed and given descriptive names.

[XAxisNamedFlags]
; A list of named flags to both axis on the SmartStage XY.

[XAxis_Filter1]
[XAxis_Filter2]
; The filter parameters used to generate a set of biquad filter coefficients.
; These settings will only be present if biquad filter(s) have been applied.

; -----

; Low Level Axis Configuration Settings

[XAxisDriverConfig]
; Low Level Magellan Controller settings

[XAxisFlagSignals]
; Low Level Magellan Controller signals used to detect flags and a list of flag definition sections.

[XAxisFlag1Configuration]
; Configuration of the first flag

[XAxisFlag2Configuration]
; Configuration of the second flag

; ...

; -----
; Emulated XAxis Driver Settings

[EmulatedXAxisDriver]
; Emulated driver settings used when running with hardware emulation

[EmulatedXAxisDriverFlagPositions]
; Defines the flag locations when running in emulation

[EmulatedXAxisDriverFlagExtents]
; Defines the length (extent) of each flag when running in emulation

; -----
; Digital Inputs

; Definition of digital input device(s)

[XAxisDigitalInput_DisabledDevice]
; Definition of a digital input device

[EmulatedXAxisDigitalInputDriver]
; Definition of an emulated digital input driver

; ...

```

```

; -----
[YAxis_Device]
; The same set of settings and sections for the Y axis

; -----
; Further axis device definitions
; ...

; -----
; Trace Capture Profiles

[MoveTraceProfile]
; Definition of a trace capture specifying the trace period and set of variables to capture.

; ...

```

3.1 Configuring the Linear Units

The system can be configured to use Millimetres, Micrometres or Nanometres. The default system configuration has been set to Millimetres. The selected units will be used when specifying positions and distances for axis move commands. They are used when entering positions, distances and move profiles into the EngineeringGUI. When changing the units, it's **important** to update (i.e scale) all positions, distances and move profiles specified in configuration or in scripts. The linear units is a global system setting which can be updated as follows:

```

[Instrument]
; Set the linear units to use for all axis: Millimetres, Micrometres or Nanometres
; WARNING: IF THE UNITS ARE CHANGED ALL DEFAULT VALUES IN THIS FILE WILL NEED TO BE SCALED
APPROPRIATELY
LinearUnits=Micrometres

```

If the units are changed from Millimetres to Micrometres, all positions and distances must be multiplied by 1000. For example the following:

```

[XAxisNamedPositions]
; Multiply all named positions by 1000 when changing from mm to um.
...

[XAxisNamedDistances]
; Multiply all named distances by 1000 when changing from mm to um.
...

[XAxisHomingProfile]
; Multiply all velocity, acceleration, deceleration and jerk settings in Move Profiles by 1000 when
changing from mm to um.
...

```

3.2 Miscellaneous Global Settings

There are a few miscellaneous global settings specified in the *[Instrument]* section.

```

; Script run during initialization
InitializeScript=[SupportFolder]/Scripts/SmartStageXY/Initialize.lua

```

This setting specifies the script run by the Initialize command. This script typically calls initialize on each axis and homes each axis ready for use. Note the *[SupportFolder]* in the above path refers to the location of the SupportFolder which can be specified at runtime.

3.3 Emulation

MotionSynergyAPI provides the ability to emulate the behavior of each axis, allowing the software to be run and tested without needing actual hardware. Emulation is implemented in the driver layer. Each axis device has an associated driver. When running with real hardware, the device uses a real driver. When running under emulation the device uses an emulated driver.

Each emulated driver includes a test interface which allows various state changes to be introduced. This is useful during automated testing, especially when testing error flows.

Emulation can be enabled or disabled for all devices using the following configuration setting:

```
; When using real hardware, set the value to blank: EmulatedDevices=
; To emulate a single axis, specify the axis name: EmulatedDevices=YAxis
; To emulate all axis, set the value to ALL:      EmulatedDevices=ALL
EmulatedDevices=
```

3.4 Configuring the Communications Settings

The following communications mechanisms are supported:

- Serial (RS232 / RS485)
- CAN

Once the communications mechanisms have been configured, each axis driver will need to be configured to specify:

- The type of communications to use (e.g. *SerialComms* or *CANComms*)
- The unique *nodeID* corresponding to each axis (this must match the node ID configured in the axis' NVRAM).

3.4.1 Serial Communications Options

The following two settings will need to be configured depending on your setup:

1. *hwid* - Specifies the COM port to use.
2. *protocol* - For RS232, set this to *Point2Point*, for RS485 set this to *MultiDropUsingIdleLineDetection*

The remaining settings should not need to be changed.

```
[SerialComms]
TransportType=SERIAL
TransportOptions=SerialCommsOptions

[SerialCommsOptions]
; Select the COM port to use (e.g COM1, COM2, etc for Windows and /dev/ttyUSB0, /dev/ttyUSB1, ... on Linux)
hwid=COM2

; Selects the communications protocol: Point2Point or MultiDropUsingIdleLineDetection.
protocol=MultiDropUsingIdleLineDetection

; Baud Rate to use (1200,2400,9600,19200,57600,115200)
baudrate=115200

; Stop bits (ONESTOPBIT, FIVESTOPBITS, TWOSTOPBITS)
stop=ONESTOPBIT
```

```
;Parity of serial (NOPARITY, ODDPARITY, EVENPARITY, MARKPARITY, SPACEPARITY)
parity=NOPARITY

; Timeout in milliseconds before a communications error is reported.
timeout=250
```

3.4.2 Using Multiple Communication Cables

It's possible to use more than one cable to connect devices to the PC. For example, you could connect two DOF-5 devices, each with its own RS232 cable. Another option would be to connect a SmartStageXY or SmartStageLinear using CAN and a DOF-5 using an RS232 cable.

The *First Time Startup / Re-configure* window allows you to configure multiple communications links and is the preferred method of configuration.

However you can also edit the configuration file directly if you need greater control (e.g. using multiple CAN adapters).

To set up a second RS232 connection, you can simply copy the *[SerialComms]* section and *[SerialCommsOptions]* section and rename them. You will also need to set the *hwid* to specify a different COM port. e.g. To define a 2nd RS232 link on COM3:

```
[SerialComms2]
TransportType=SERIAL
TransportOptions=SerialCommsOptions2

[SerialCommsOptions2]
; Select the COM port to use (e.g COM1, COM2, etc for Windows and /dev/ttyUSB0, /dev/ttyUSB1, ... on Linux)
hwid=COM3

; Selects the communications protocol: Point2Point or MultiDropUsingIdleLineDetection.
protocol=Point2Point

; Baud Rate to use (1200,2400,9600,19200,57600,115200)
baudrate=115200

; Stop bits (ONESTOPBIT, ONE5STOPBITS, TWOSTOPBITS)
stop=ONESTOPBIT

;Parity of serial (NOPARITY, ODDPARITY, EVENPARITY, MARKPARITY, SPACEPARITY)
parity=NOPARITY

; Timeout in milliseconds before a communications error is reported.
timeout=250
```

You then need to update the axis driver to use this cable by setting the *Comms* field to use the *SerialComms2* settings. E.g.

```
[Z2AxisDriver]
; Selects the communications mechanism: SerialComms or CANComms.
Comms=SerialComms2
; Unique Multidrop address for this axis (e.g. 1=XAxis, 2=YAxis, 3=ZAxis,...). Do not use a nodeID of zero as this will address all devices on the CAN bus.
nodeID=3
Type=SmartStageAxisDriver
Config=ZAxisDriverConfig
FlagSignals=ZAxisFlagSignals
```


3.4.3 CAN Communications Options

3.4.3.1 Windows

The following set of options are applicable to the CAN communications mechanism. These don't usually need to be changed.

Note: If there is more than one IXXAT CAN adapter connected, then the *hwid* (hardware ID) setting can be used to specify a specific adapter.

```
[CANComms]
TransportType=CAN
TransportOptions=CANCommsOptions

[CANCommsOptions]
; Baud Rate to use, where 0 = 1 Mbit/s
baudrate=0
; Timeout in milliseconds before a communications error is reported.
timeout=100

; hwid can be used to select a specific IXXAT adapter when there is more than one adapter connected.
; If 'hwid' is not specified (e.g. commented out as below), the first IXXAT adapter found will be used.
; hwid=HWnnnnnn
```

3.4.3.2 Linux

On Linux systems you need to set the *hwid* to the desired CAN device.

```
[CANComms]
TransportType=CAN
TransportOptions=CANCommsOptions

[CANCommsOptions]
; Baud Rate to use, where 0 = 1 Mbit/s
baudrate=0
; Timeout in milliseconds before a communications error is reported.
timeout=100

; If 'hwid' is not specified (e.g. commented out as below), the first SocketCAN device (can0) will be used.
; hwid=can1
```

The baud-rate is controlled by using an external linux network utility. Example of setting the baud-rate of can0 using the network configuration 'ip' command.

```
sudo ip link set can0 up type can bitrate 1000000 restart-ms 100
```

If you want to automate the setting of the can baud-rate, you can add an entry to */etc/systemd/network*. Example of configuring can0 by creating file *"*/etc/systemd/network/can0.network*":*

```
[Match]
Name=can0

[CAN]
BitRate=1000K
RestartSec=100ms
```

3.5 Configuring an Axis

Each Dover Motion product is made up of one or more axis. An axis represents a single, linear axis of motion. The *Instrument.cfg* file contains one or more axis which make up the system. The

following configuration settings use the XAxis as an example. To configure a different axis, simply rename the settings (e.g. use YAxis, ZAxis, Z2Axis, etc...)

The following settings define an axis. Many settings at this level link to further *[Section]*s which specify different aspects of the axis behavior.

```

; -----
; X Axis

[XAxis_Device]
Type=AxisDevice
DefaultTimeout_ms=45000
MaxTimeout_ms=120000
NamedPositions=XAxisNamedPositions
NamedDistances=XAxisNamedDistances
NamedMoveProfiles=XAxisHomingProfile,XAxisStandardMoveProfile
NamedFlags=XAxisNamedFlags
Driver=XAxisDriver
EmulatedDriver=EmulatedXAxisDriver
; Specifies the home script run by the MotionSynergyGUI when the user selects the 'Home' command.
HomeScript=[SupportFolder]/Scripts/SmartStageXY/HomeXAxis.lua

[XAxis_Device]

```

Defines the axis device section. The format of the section name is *[AxisName_Device]*. The *_Device* suffix must be present so the software can recognise this section as an axis device section.

Type=AxisDevice

Specifies the type of device. This system only supports *AxisDevice*.

DefaultTimeout_ms=45000

Specifies the default timeout (in milliseconds) to use when calling the *Wait()* command without a timeout parameter.

MaxTimeout_ms=120000

Specifies the maximum timeout (in milliseconds) for the *Wait()* command. The timeout used by *Wait()* must be less or equal to this value.

NamedPositions=XAxisNamedPositions

Specifies the section containing a list of named positions for this axis. Named positions are used by the *NMoveAbsolute* and *NMoveOffset* commands.

NamedDistances=XAxisNamedDistances

Specifies the section containing a list of named distances for this axis. Named distances are used by the *NMoveRelative*, *NMoveRelativeToFlag* and *NMoveOffset* commands.

NamedMoveProfiles=XAxisHomingProfile,XAxisStandardMoveProfile

Specifies a list of move profiles for this axis. Each move profile is defined in its own section. Move profiles are used by the *SelectMoveProfile* command.

NamedFlags=XAxisNamedFlags

Specifies the section containing a list of named flags for this axis. Named flags are used by the *(N)MoveRelativeToFlag*, *GetFlagStatus* and *GetLatchedFlagPosition* commands.

Driver=XAxisDriver

Specifies the section containing settings for the axis driver used when controlling real hardware. The axis driver settings contain communications settings and other low-level settings.

EmulatedDriver=EmulatedXAxisDriver

Specifies the section containing settings for the emulated axis driver used when running without real hardware. The emulated axis driver settings contain settings used to emulate the behavior of a real axis device.

3.5.1 Named Positions

Named (absolute) positions provide a mechanism to refer to absolute positions by name. There can be as many or few positions defined as necessary. Named positions should identify application specific locations. Moving to named positions make code and scripts a lot more readable. They also provide a means of compensating for manufacturing differences across system instances. The configuration infrastructure provides a means for 'local' overrides which allows each system instance to have slightly different (i.e. calibrated) values for each position.

```
; -----
; XAxis Named Absolute Positions

[XAxisNamedPositions]
; These named positions are for a 100mm stage with home in the center.
Home=0

; Default limits are for a 100mm stage.
; These limits may be updated during the first time startup to suit a different stage.
NegativeLimit=-50
PositiveLimit=50
```

3.5.2 Named Distances

Named (relative) distances provide a mechanism to refer to distances used during relative moves by name. There can be as many or few positions defined as necessary. Named distances are application specific. Moving by a named distance make code and scripts a lot more readable.

```
; -----
; XAxis Named Relative Distances

[XAxisNamedDistances]
; These named positions are for a stage with home in the center.

; Used during homing to ensure center flag is always approached from the same direction.
MoveToCenterOffset=-10

MaxPositiveHomingDistance=102
MaxNegativeHomingDistance=-102
```

3.5.3 Move Profiles

Move profiles provide a way of defining the set of parameters that define a move profile. There can be as many or few profiles defined as necessary. The *SelectMoveProfile* command provides a convenient mechanism to load these parameters from configuration and set the *Velocity*, *Acceleration*, *Deceleration* and *Jerk* parameters for an axis in one operation.

```
; -----
; Move Profiles

[XAxisHomingProfile]
Velocity=10
Acceleration=200
Deceleration=200
Jerk=250
```

When adding a move profile, you must also add the move profile name to the list of move profiles for the axis. e.g.

```
[XAxis_Device]
...
NamedMoveProfiles=XAxisHomingProfile,XAxisStandardMoveProfile,XAxisMyNewMoveProfile
...
```

3.5.4 Time-based User Defined Profiles (UDP)

A Time-based User Defined Profile is a special type of move profile where you can specify a series of (time, position) pairs to define the move profile you wish to perform. It allows complex motion to be performed in a single move command. When a UDP move profile is selected, the series of (time, position) pairs are downloaded to the motion controller. When the MoveRelativeUDP() command is executed, the motion controller will use the (time, position) pairs to perform the move.

A complete description of the UDP feature can be found in [Magellan User-Defined Profile Mode Documentation](#). Note that the MotionSynergyAPI uses Time as the Master Source.

There are a couple of example Time-based UDP profiles defined in the [Templates/Configuration](#) folder in UDPProfiles.cfg.

A Time-based UDP profile is defined in configuration as follows:

```
[XAxisUDPSinProfile]
UDPMODE=Time
UDPInputStreamFile=[SupportFolder]/UDP_SIN_10s_0_22_-10_10mm.csv
RateScalar=1.0
UDPMODE=Time
```

Specifies the UDP profile mode, either Time for time-based profiles or Encoder for master-driven profiles (see below).

```
UDPInputStreamFile=[SupportFolder]/UDP_SIN_10s_0_22_-10_10mm.csv
```

Specifies the path to a CSV file containing a series of time, position pairs. The first column specifies the time in milliseconds, the second column specifies the position in the current units (e.g. mm).

```
0,0
5,0.06783175
10,0.135662879
15,0.203492769
20,0.271320798
...
```

Note that if the last position in the profile is not zero and the profile is repeated (i.e. the repeats parameter passed to MoveRelativeUDP() is greater than one), each repeated move will be offset by the last position. For example if the last position is 10mm, the first repeated move will start from 10mm and finish at 20mm and the next will start from 20mm and finish at 30mm and so on.

```
RateScalar=1.0
```

The RateScalar can be used to scale the time values. For unscaled values this should be set to 1.0. To double the speed of the move it should be set to 2.0. Note that MotionSynergyAPI only supports Rate Scalar value **greater than 0**.

When adding a UDP move profile, you must also add the profile name to the list of move profiles for the axis. e.g.

```
[XAxis_Device]
...
NamedMoveProfiles=XAxisHomingProfile,XAxisStandardMoveProfile,XAxisUDPSinProfile
...
```

3.5.5 Coordinated Motion using Master-driven User Defined Profiles (UDP)

Coordinated Motion between two axes can be achieved using a Master-driven User Defined Profile. This profile type allows you to specify a series of (master axis position, driven axis position) pairs to move a 'Driven' axis in relation to a 'Master' axis. When a Master-Driven UDP move profile is selected for the driven axis, the series of (master axis position, driven axis position) pairs are downloaded to the motion controller. When the master axis is moved via a MoveRelative or MoveAbsolute command, the motion controller will use the (master axis position, driven axis position) pairs to update the driven axis position based on the current master actual position.

A complete description of the UDP feature can be found in [Magellan User-Defined Profile Mode Documentation](#). Note that the MotionSynergyAPI uses Master axis 'Actual Position' as the Master Source.

There is an example Master-driven UDP profile defined in the [Templates/Configuration](#) folder in UDPProfiles.cfg.

In addition to defining a Master-driven UDP profile, the GPIO configuration must be set correctly for both axes. For a SmartStageXY, the following GPIO settings must be used.

Coordinated Motion	XAxis GPIO Configuration	YAxis GPIO Configuration
Lower (XAxis) Master, Upper (YAxis) Driven	PMDGpioConfiguration::S5	PMDGpioConfiguration::S1 or PMDGpioConfiguration::S3
Lower (XAxis) Driven, Upper (YAxis) Master	PMDGpioConfiguration::S6	PMDGpioConfiguration::S7

See the section below on setting GPIO configuration for an axis for more information.

A Master-driven UDP profile is defined in configuration as follows:

```
[YAxisDrivenProfile]
UDPMode=Encoder
UDPInputStreamFile=[SupportFolder]/UDP_MasterDriven0_5mm.csv
RateScalar=1.0
UDPMode=Encoder
```

Specifies the UDP profile mode, either Time for time-based profiles (see above) or Encoder for master-driven profiles.

```
UDPInputStreamFile=[SupportFolder]/UDP_MasterDriven0_5mm.csv
```

Specifies the path to a CSV file containing a series of master position, driven position pairs. Both columns specify position in the current units (e.g. mm).

```
0,0
0.00958,0.00958
0.01916,0.01916
0.02874,0.02874
0.03832,0.03832
0.0479,0.0479
0.05748,0.05748
0.06706,0.06706
0.07664,0.07664
0.08622,0.08622
0.0958,0.0958
...
RateScalar=1.0
```

The RateScalar should be set to one for master-driven UDP profiles.

When adding a UDP move profile, you must also add the profile name to the list of move profiles for the axis. e.g.

```
[YAxis_Device]
...
NamedMoveProfiles=YAxisHomingProfile,YAxisStandardMoveProfile,YAxisDrivenProfile
...
```

3.5.5.1 Stop and EStop behavior while using Coordinated Motion

To stop motion while using Coordinated Motion, you must send a *Stop* or an *EStop* command to the master axis. This will bring the master axis to a halt, which in turn will cause the driven axis to come to a halt.

If you send a *Stop* command to the driven axis while using Coordinated Motion, it will have no effect and Motion Synergy will report an invalid state error. The driven axis will continue to follow the master axis.

If you send an *EStop* command to the driven axis while using Coordinated Motion, the driven axis will halt but the master axis motion will never complete. In this case the master axis must also be sent an *EStop*.

3.5.5.2 On-the-fly updates while using Coordinated Motion

It's possible to update/adjust the values in a User Defined Profile while a UDP move is in progress. There are a number of limitations to be aware of:

1. An initial table of data pairs must be loaded by using SelectMoveProfile to load a user defined profile.
2. The size of this table cannot be changed on-the-fly (i.e. extended or reduced in size).
3. An on-the-fly update to this table will only affect the axis motion if the write index of the updated value is greater than the current datastream index (i.e. the target position being updated has not yet been used by the motion controller to calculate the axis' target position).

To update a UDP position on-the-fly, the following command is available (in C#):

```
IAxis.UserDefineProfile.UpdateProfileData(uint index, double position)
```

It's also possible to read the datastream index within the User Defined Profile that's currently being used by the motion controller. This allows you to determine the next position to update. e.g.

```
IAxis.UserDefineProfile.GetCurrentDataStreamIndex()
```

These commands are available for all Motion Synergy supported languages (i.e. C#, C++, Lua, Python, LabView).

3.5.6 Sine-wave based User Defined Profiles (UDP)

The user can command an axis to move in a sine-wave pattern using this profile. This mode cannot be set via the configuration file. To set the Sine-wave UDP profile, run the following command (in C# or Lua).

Example in C#:

```
// Frequency to 1Hz, Amplitude to 0.5 unit (mm, um, or nm), Angular Frequency to 1000 Hz/sec
IAxis.UserDefineProfile.SelectSineWaveMoveProfile(1, 0.5, 1000)
```

When MoveRelativeUDP() command is executed, the axis will move in the sine wave pattern based on the profile. The number of sine waves can be passed to MoveRelativeUDP(). The number of sine waves is limited to the maximum of unsigned 32-bit integer i.e., 4,294,967,296.

3.5.7 Flags

A flag is a hardware switch that is triggered whenever an axis reaches a particular location on its travel. Common examples of flags include a limit switch at each end of travel, a half vane travel switch, home, and encoder index switches. Flags are often used during homing with the (N)MoveRelativeToFlag commands.

```
; A List of named flags and their associated FlagIndex
[XAxisNamedFlags]
NegativeLimit=0
NegativeHalf=1
PositiveHalf=2
PositiveLimit=3
```

Each flag has a name and an associated index. The above example is the set of flags used with the SmartStageXY or SmartStageLinear. Each flag is defined in the axis driver settings (both real and emulated drivers).

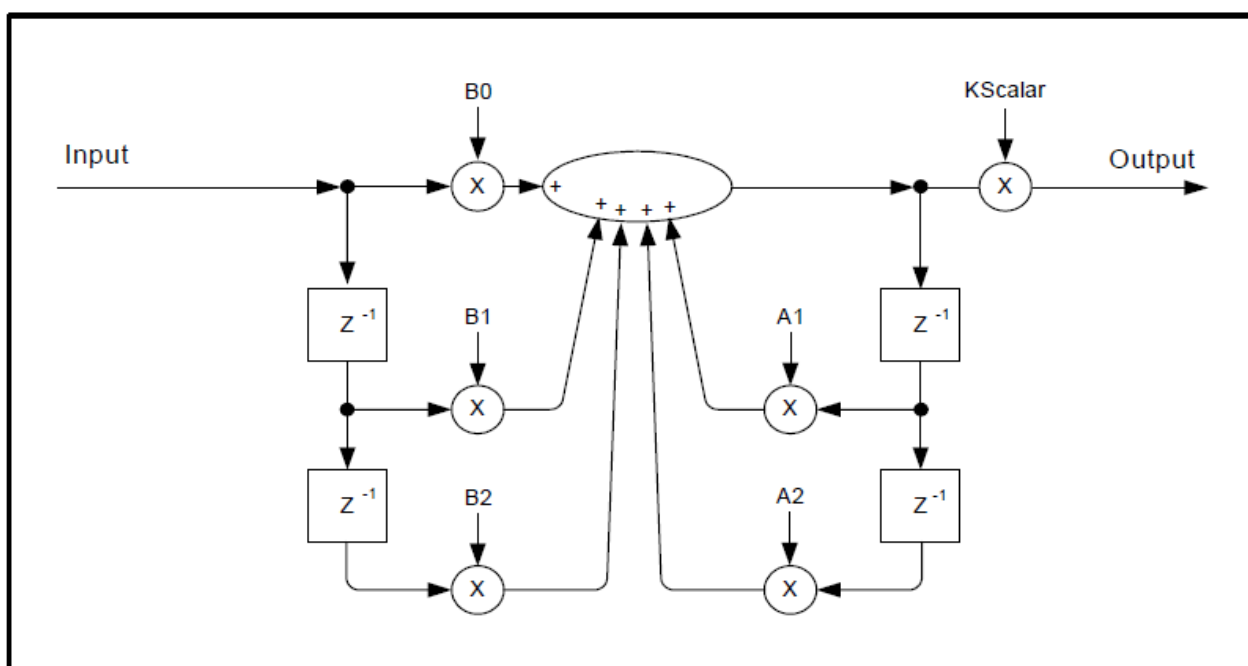
3.5.8 Biquad Filters

In some cases, a complex mechanical plant will produce resonant frequency peaks. If these resonances are at sufficiently high frequency, additional servo loop filters can provide some relief. The simplest of these is a low-pass filter, where the response rolls off smoothly above the filter frequency. When presented with a single narrow resonance, another option is to place a notch filter directly on the resonance of interest. In the case of a notch filter, excitation is strongly attenuated for just the narrow range of frequencies of the resonance. While notch filters can help tame an unruly plant, it isn't realistic to push the servo bandwidth close to the notch frequency; otherwise, phase shifts from the filter itself will endanger stability. A good rule of thumb is to shoot for a servo bandwidth no higher than half of the frequency of the notch filter. In practical terms this means not setting a low-pass or notch filter below ~500 Hz.

A biquad is a generic digital filter structure. With the proper coefficients, it can be programmed to be a low-pass filter, high-pass filter, band-pass filter, notch filter, or custom filter. Programs such as [Octave](#) may be used to find the coefficients.

The controller supports two programmable biquad output filters for each axis. These filters are chained. When both are enabled, the output of Filter1 feeds the input of Filter2. If Filter1 is disabled (the default state), the entire filter chain is bypassed, and the motion control IC output passes unfiltered to the motor.

To create a Low Pass filter, enter the target cutoff frequency and quality factor. The low-pass filter will attenuate the response above the cutoff frequency. To create a Notch filter, enter the center frequency and quality factor.



The output of the filter at time n is determined with the following equation:

$$Y_n = K \times (B_0 \times X_n + B_1 \times X_{n-1} + B_2 \times X_{n-2} + A_1 \times Y_{n-1} + A_2 \times Y_{n-2})$$

The following list summarizes the representation and range for the settable biquad parameters.

- Y_n = output of the filter at time n
- X_n = input to the filter at time n
- K = positive scaling value used to avoid rounding errors
- B_0 = programmable biquad coefficient
- B_1 = programmable biquad coefficient
- B_2 = programmable biquad coefficient
- A_1 = programmable biquad coefficient
- A_2 = programmable biquad coefficient

See the full Magellan Motion Control IC User Guide for additional details on calculating the biquad filter coefficients using Matlab or Octave.

The MotionSynergyGUI allows the user to specify a filter by selecting its type (Notch or LowPass), frequency and quality factor. These 3 parameters are then used to calculate a set of biquad coefficients (B_0 , B_1 , B_2 , A_1 , A_2 and K). Only the coefficients are downloaded to the motion controller. The MotionSynergyGUI stores a record of the original parameters used to generate the coefficients in configuration. On startup, the coefficients are retrieved from the motion controller. If they match the coefficients calculated using the filter type, frequency and quality factor stored in configuration, the filter will be correctly identified and displayed in the GUI. If they don't match, the filter type is displayed as *Unknown* in the GUI.

It's important to note, the filter parameters stored in configuration are only used to recognise the coefficients retrieved from the motion controller. They are **not** used to configure the motion controller on startup, unlike other configuration parameters.

```
[XAxis_Filter1]
FilterType=Notch
Frequency=700
QualityFactor=1

[XAxis_Filter2]
FilterType=None
Frequency=0
QualityFactor=0
```

3.5.9 Configuring the Axis Driver

The axis driver configuration specifies the communications mechanism to the hardware, the status bits associated with each flag and a number of other low level settings. Apart from the communication settings, most other settings will vary rarely need to be changed.

```
; -----
; X Axis Driver Settings

[XAxisDriver]
Comms=SerialComms
nodeID=1
Type=SmartStageAxisDriver
Config=XAxisDriverConfig
FlagSignals=XAxisFlagSignals
CommutationConfig=XAxisCommutation
```

The above settings are described as follows:

`Comms=SerialComms`

Specifies the section containing the serial or CAN communication settings for this axis driver.

`nodeID=1`

Unique address for this axis when using CAN or RS485 (e.g. 1=XAxis, 2=YAxis, 3=ZAxis,...). Do not use a nodeID of zero as this will address all devices on the CAN bus.

`Type=SmartStageAxisDriver`

Specifies the driver type. The axis driver for controlling real hardware should always be set to *SmartStageAxisDriver*.

`Config=XAxisDriverConfig`

Specifies the section containing low level settings for this axis driver.

`FlagSignals=XAxisFlagSignals`

Specifies the section containing a definition of each flag used by this axis driver.

`CommutationConfig=XAxisCommutation`

Specifies the section containing the commutation settings used by this axis driver.

3.5.10 Axis Driver Low Level Configuration

The following configuration settings should not need to be changed for typical scenarios. They do provide the ability to further tune the software.

```
[XAxisDriverConfig]
NanoMetersPerStep=5
```

```

LegCurrentMilliampsPerCount=0.611
BusCurrentMilliampsPerCount=0.0775
BusVoltageMillivoltsPerCount=0.9672
GpioConfiguration=PMDGpioConfiguration::S2
EStopPeriodicIntervalInMs=50
EStopTimeoutInMs=250
MovePeriodicIntervalInMs=25
StopTimeoutInMs=5000
DigitalInputPollingPeriodMs=250

NanoMetersPerStep=5

```

Used to convert from real world units into step counts. This value is specific to a particular product and should not be change.

```
LegCurrentMilliampsPerCount=0.611
```

Used to convert from counts into milliamps for leg currents. Used when capturing trace data for current variables. This value is specific to a particular product and should not be change.

```
BusCurrentMilliampsPerCount=0.0775
```

Used to convert from counts into milliamps for the bus current. Used when capturing trace data for current variables. This value is specific to a particular product and should not be change.

```
BusVoltageMillivoltsPerCount=0.9672
```

Used to convert from counts into millivolts for the bus voltage. Used when capturing trace data for voltage variables. This value is specific to a particular product and should not be change.

```
GpioConfiguration=PMDGpioConfiguration::S2
```

Only applicable to the SmartStageXY and SmartStageLinear. Used to configure the GPIO signals, where *GpioConfiguration* can be set to one of the following:

Signal Information / Pinout

Configuration Setting	I/O State	I/O 1	I/O 2	I/O 3	MotionSynergyAPI Function
PMDGpioConfiguration::S1	S1	Step Input	Direction Input	GPO, TOP LUT	External Control, Driven upper axis in Coordinated motion mode
PMDGpioConfiguration::S2	S2	TOP Incremental	TOP Gate	GPO	Trigger on Position (TOP), Digital Output Trigger
PMDGpioConfiguration::S3	S3	GPI	GPI	GPO	Two general purpose inputs which can be read using the digital input device, Driven upper axis in Coordinated motion mode
PMDGpioConfiguration::S4	S4	GPI	Encoder A Out	Encoder B Out	A single general purpose input which can be read using the

					digital input device
PMDGpioConfiguration::S5	S5	Coordinated motion mode,	lower axis Master	GPO	Master lower axis in Coordinated motion mode
PMDGpioConfiguration::S6	S6	Coordinated motion mode,	upper axis Master	GPO	Driven lower axis in Coordinated motion mode
PMDGpioConfiguration::S7	S7	GPO	Encoder A Out	Encoder B Out	Master upper axis in Coordinated motion mode

`EStopPeriodicIntervalInMs=50`

The polling frequency in milliseconds used during an EStop operation. Reducing this number will increase the responsiveness with which the software reports the completion of the EStop command at the expensive of additional CPU cycles. Conversely increasing this number will reduce the responsiveness but use less CPU cycles.

`EStopTimeoutInMs=250`

The time to wait in milliseconds for an EStop command to complete before reporting a timeout. When an EStop command is issued to an axis, it will perform an abrupt stop, which must complete within this timeout period.

`MovePeriodicIntervalInMs=25`

The polling frequency in milliseconds used while an axis is moving. Reducing this number will increase the responsiveness with which the software reports the completion of a move at the expensive of additional CPU cycles. Conversely increasing this number will reduce the responsiveness but use less CPU cycles.

`StopTimeoutInMs=5000`

The time to wait in milliseconds for a Stop command to complete before reporting a timeout. When a Stop command is issued to an axis, it will perform a smooth stop, which must complete within this timeout period.

`DigitalInputPollingPeriodMs=250`

The polling period to use for digital inputs when asynchronous updates are enabled. In this case, the digital inputs will be polled 4 times per second.

3.5.11 Configuring the Axis Driver Flag Signals

The axis flag signals provide a mapping between the Axis flags and the associated signals generated by the Magellan Motion Controller Chip used by the Dover Motion products. These flag signals have been preconfigured for each Dover Motion product and shouldn't need to be changed. However there may be advanced scenarios where this is useful. The signals specified for each flag are documented in the [Magellan Motion Control IC User Guide.pdf](#).

```
[XAxisFlagSignals]
; Mask containing complete set of signals applicable to flags
SignalsMask=PMDSignalStatus::PositiveLimit | PMDSignalStatus::NegativeLimit | PMDSignalStatus::AxisIn
; Set of signals which need the polarity inverted
```

```
InvertedSignals=PMDSignalStatus::PositiveLimit | PMDSignalStatus::NegativeLimit
; A list of flags to configure
Flags=XAxisNegativeLimitFlag,XAxisNegativeHalfFlag,XAxisPositiveHalfFlag,XAxisPositiveLimitFlag
SignalsMask=PMDSignalStatus::PositiveLimit | PMDSignalStatus::NegativeLimit | PMDSignalStatus::AxisIn
```

Specifies the complete set of signals applicable to flags. This allows other status bits not pertaining to the flag states to be masked out. Note that Signal bits can be or'ed together using the '|' character.

```
InvertedSignals=PMDSignalStatus::PositiveLimit | PMDSignalStatus::NegativeLimit
```

Specifies the set of signals (if any) which need their polarity reversed.

```
Flags=XAxisNegativeLimitFlag,XAxisNegativeHalfFlag,XAxisPositiveHalfFlag,XAxisPositiveLimitFlag
```

A list of flags to configure, where each entry references a section of settings. There must be one entry per named axis flag (see the axis NamedFlags defined above).

```
; Configuration section for the NegativeLimit flag
[XAxisNegativeLimitFlag]
; Flag index for this flag.
FlagIndex=0
; The Magellan Controller SignalStatus bit indicating the axis is in this flag.
FlagSignal=PMDSignalStatus::NegativeLimit
; Indicates this flag corresponds to the negative limit switch
IsNegativeLimit=true

[XAxisNegativeHalfFlag]
FlagIndex=1
; Note the ! indicates this flag will be true whenever the AxisIn signal is NOT high.
FlagSignal=!PMDSignalStatus::AxisIn

[XAxisPositiveHalfFlag]
FlagIndex=2
FlagSignal=PMDSignalStatus::AxisIn

[XAxisPositiveLimitFlag]
FlagIndex=3
FlagSignal=PMDSignalStatus::PositiveLimit
; Indicates this flag corresponds to the positive limit switch
IsPositiveLimit=true
```

The above settings configure the signal status for the SmartStageXY or SmartStageLinear flags.

```
FlagIndex=0
```

The FlagIndex for this flag. It must correspond to the equivalent NamedFlag. FlagIndexes start at zero.

```
FlagSignal=PMDSignalStatus::NegativeLimit
```

The Magellan Controller SignalStatus bit indicating the axis is in this flag. It's possible to specify the flag is triggered on the inverted signal by preceding the flag with the '!' character.

```
IsNegativeLimit=true
```

Optional setting indicating this flag corresponds to the negative limit switch.

```
IsPositiveLimit=true
```

Optional setting indicating this flag corresponds to the positive limit switch.

3.5.12 Configuring the Axis Driver Commutation Settings

The following settings are used during the commutation phase of initialization. The MaxMotorCurrentPercent may need to be increased for axis with a heavy payload. If the axis is

intermittently failing to initialize due to a commutation error, the `MaxMotorCurrentPercent` can be increased, up to a maximum of 50%.

```
[XAxisCommutation]
; When DirectSetCommutationEnabled is true, DirectSetCommutation is performed instead of algorithmic
commutation.
StepDirectSetPhaseInitializationEnabled=false
; PhaseAngle to use when at half travel vane during DirectSetCommutation. The PhaseAngle is determined
in the factory and unique to a stage.
PhaseAngle=676
; MaxMotorCurrentPercent is used during Commutation to set the max motor output. The
MaxMotorCurrentPercent
; is limited to 50% in software.
MaxMotorCurrentPercent=25
```

3.5.13 Emulated Axis Driver Configuration

The following settings are applicable to the emulated axis driver.

```
[EmulatedXAxisDriver]
Type=EmulatedAxisDriver
NamedFlagPositions=EmulatedAxisDriverFlagPositions
NamedFlagExtents=EmulatedAxisDriverFlagExtents
ScalingFactor=100
Type=EmulatedAxisDriver
```

Specifies the driver type. The emulated axis driver should always be set to *EmulatedAxisDriver*.

```
NamedFlagPositions=EmulatedAxisDriverFlagPositions
```

Specifies the section used to define actual positions for each flag.

```
NamedFlagExtents=EmulatedAxisDriverFlagExtents
```

Specifies the section used to define the extent (length) of each flag.

```
ScalingFactor=100
```

The `ScalingFactor` is used to speed up the duration of axis moves during emulation. A `ScalingFactor` of 1 will result in the duration of emulated axis moves being close to the duration of real moves. A `ScalingFactor` of 100 will speed up the moves by a factor of 100x. This helps reduce the running time of things such as automated tests.

```
[EmulatedXAxisDriverFlagPositions]
0=-50
1=0
2=0
3=50
```

The emulated axis driver flag positions are used to define the location of the flags. These should correspond to the location of the physical flags of the real hardware. For each flag, identified by its flag index, the flag location needs to be specified in the selected linear units (i.e. Millimetres, Micrometres or Nanometres).

```
[EmulatedXAxisDriverFlagExtents]
0=-50.01
1=-50
2=50
3=50.01
```

Physical flags are not defined by a single point. Instead they occupy some length of the axis, albeit a very small distance in many cases. The emulated axis driver extents help define the length of each flag. For each flag, identified by its flag index, the emulated axis driver flag position defines one end of the flag and the emulated axis driver flag extent defines the other end of the flag in the selected linear units (i.e. Millimetres, Micrometres or Nanometres).

3.5.14 Configuring Trace Capture Profiles

A trace capture profile is used when capturing trace data during an axis move (see `EnableTraceCapture()`, `NEnableTraceCapture()` and `SaveTraceCapture()`). A trace capture profile defines:

- The trace period. A period of 1 will capture each variable every clock cycle of the motion control chip. A period of 10 will capture each variable every 10 clock cycles of the motion control chip.
- Up to 4 variables to capture during the axis move (see `TraceVariable` in the C++ API section for a list of possible values).

A typical trace capture profile looks as follows:

```
[MoveTraceProfile]
Period=10
Variable1=CommandedVelocity
Variable2=PositionError
Variable3=CurrentLoopActualCurrentA
```

3.5.15 Configuring Digital Inputs

Each axis controller can be configured to support two general purpose inputs. The two general purpose inputs on each controller are read using a single digital input device. The default configuration has entries for a single digital input device (SmartStageLinear, DOF-5 and DMCM products) or two digital input devices (SmartStage XY). They are disabled by default and need to be enabled for use. This is done by removing 'Disabled' from the device section name.

```
; -----
; Digital Inputs

; The digital input device is disabled by default. To enable this device so the
; two general purpose inputs can be read, remove 'Disabled' from the device section name

[XAxisDigitalInput_DisabledDevice]
Type=DigitalInputDevice
Driver=XAxisDriver
EmulatedDriver=EmulatedXAxisDigitalInputDriver
DefaultTimeout_ms=2000

[EmulatedXAxisDigitalInputDriver]
Type=EmulatedDigitalInputDriver
```

Note: For the SmartStageXY or SmartStageLinear product you must also configure the [GpioConfiguration setting](#) to `PMDGpioConfiguration::S3` or `PMDGpioConfiguration::S4` to support general purpose inputs.

3.5.15.1 Digital Input Asynchronous Updates

The digital input device provides a command to enable asynchronous updates. If asynchronous updates are enabled (by calling `EnableChangeNotification()` in C++, `EnableAsynchChangeUpdates()` in C# or Lua), the controller will be polled regularly and whenever a state change is detected, a callback (C++) or a .NET event (C#) will be raised. The polling frequency can be configured by adjusting the following setting:

```
[XAxisDriverConfig]
; Polling period to use for digital inputs when async updates are enabled.
DigitalInputPollingPeriodMs=250
```

3.5.15.2 Digital Input Pin Mapping

Each digital input device returns a single byte which contains the state of both general purpose inputs as follows:

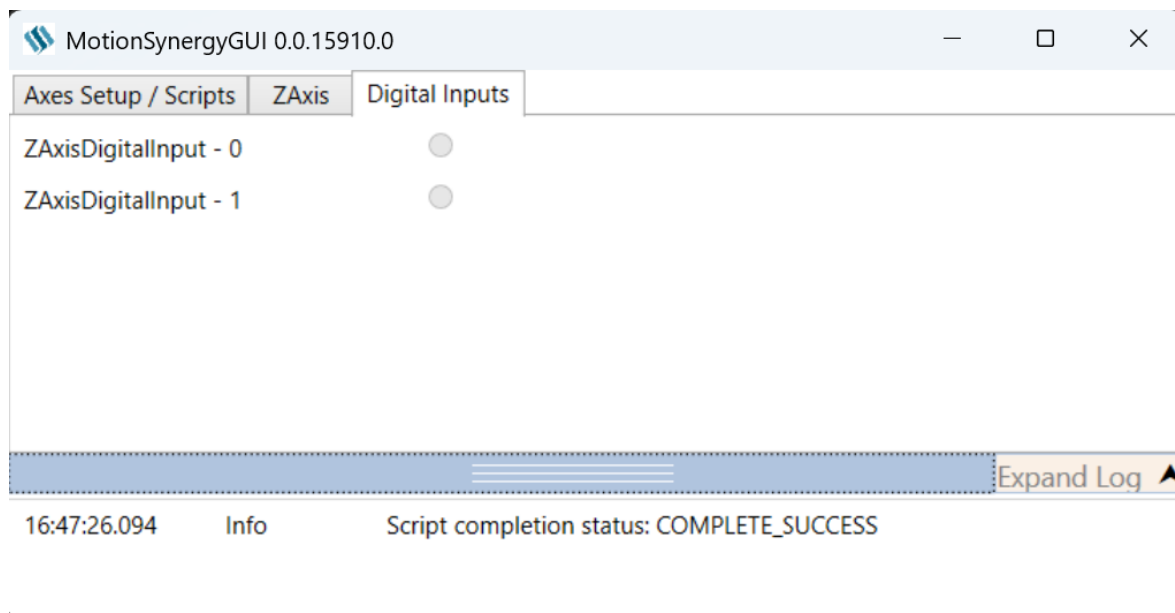
I/O 1 I/O 2

Bit 0 Bit 1

The mapping of I/O 1 and I/O 2 to specific I/O lines is defined in the product (SmartStageLinear, SmartStage XY, DOF-5, DMCM) user manuals.

3.5.15.3 Monitoring Digital Inputs in the MotionSynergyGUI

Once a digital input has been enabled, it will appear in a separate *Digital Inputs* tab in the MotionSynergyGUI.



3.6 Configuring the MotionSynergyGUI

The MotionSynergyGUI has a separate *MotionSynergyGUI.cfg* file which contains settings specific to the MotionSynergyGUI as well as an 'include' statement to include the settings from the *Instrument.cfg* file. The *MotionSynergyGUI.cfg* file is only needed when running the MotionSynergyGUI.

A typical *MotionSynergyGUI.cfg* file is as follows:

```

; -----
; Include the Instrument configuration file.
IncludeFile1=[SupportFolder]/Instrument.cfg
; -----
; Settings applicable to the Application
[MotionSynergyGUI]
InitializeAndHomeOnStartup=false

```

```

; -----
; Settings applicable to the Script Tab

[ScriptControl]
DefaultScript=[SupportFolder]/Scripts/Initialize.lua

; -----
; Settings applicable to the Axis Tab

[AxisControl]
DMCMMoveRelativeLargeStepPosition=10.0
DMCMMoveRelativeMediumStepPosition=5.0
DMCMMoveRelativeSmallStepPosition=1.0
SmartStageMoveRelativeLargeStepPosition=10.0
SmartStageMoveRelativeMediumStepPosition=5.0
SmartStageMoveRelativeSmallStepPosition=1.0
DOF5MoveRelativeLargeStepPosition=1.0
DOF5MoveRelativeMediumStepPosition=0.5
DOF5MoveRelativeSmallStepPosition=0.1
TraceProfiles=MoveTraceProfile,VelocityTraceProfile,PositionErrorTraceProfile,CommandedVelocityTraceProfile,VelAndPositionErrorTraceProfile,FullTraceProfile

; -----
; Settings applicable to the Stage Control XY(Z) Tab

[StageControl]
XYSlowAcceleration=100
XYFastAcceleration=500
XYMinVelocity=20
XYMaxVelocity=100
XYSlowAccelerationWindow=2
XYEdgeMargin=2
ZAcceleration=500
ZVelocity=100
ZSmallStep=0.005
ZMediumStep=0.001

; -----
; GUI Commands
;
; These commands will appear on the scripts control tab of the MotionSynergyGUI

[GUICommands]
; Defines the order of commands to display on the UI. If a command is not listed here, it will not be
; displayed on the UI.
;Order=Command1,Command2

;[Command1]
;...

```

The above settings are used as follows:

```

[MotionSynergyGUI]
InitializeAndHomeOnStartup=false

```

Controls whether the *Initialize and Home All* command is automatically run on startup by the MotionSynergyGUI. It's set to *false* by default, so the user must manually click the *Initialize and Home All* button. If it's set to *true*, the MotionSynergyGUI will automatically run the *Initialize and Home All* command. Note this command will cause each axis to move.

```

[ScriptControl]
DefaultScript=[SupportFolder]/Scripts/Initialize.lua

```

The default script to select for the 'Execute Script' command on the UI. If there is one script the user wishes to run regularly, this can be specified here, avoiding the need to select the same script each time the MotionSynergyGUI is started.


```
[AxisControl]
DMCMMoveRelativeLargeStepPosition=10.0
DMCMMoveRelativeMediumStepPosition=5.0
DMCMMoveRelativeSmallStepPosition=1.0
SmartStageMoveRelativeLargeStepPosition=10.0
SmartStageMoveRelativeMediumStepPosition=5.0
SmartStageMoveRelativeSmallStepPosition=1.0
DOF5MoveRelativeLargeStepPosition=1.0
DOF5MoveRelativeMediumStepPosition=0.5
DOF5MoveRelativeSmallStepPosition=0.1
```

Specifies the default amount to move in a positive or negative direction for each product type (DMCM, SmartStageXY/Linear, DOF-5) when the jog commands (i.e. '<<', '>>', etc) on the axis tab are selected.

```
TraceProfiles=MoveTraceProfile,VelocityTraceProfile,PositionErrorTraceProfile,CommandedVelocityTraceProfile,VelAndPositionErrorTraceProfile,FullTraceProfile
```

A list of the trace capture profiles defined in the *Instrument.cfg*. These profile names are used to populate the trace capture drop down list on the axis tab.

The following settings are applicable to the Stage Control XY(Z) Tab

```
[StageControl]
; XY Stage acceleration for short moves.
XYSlowAcceleration=100
; XY Stage acceleration for larger moves.
XYFastAcceleration=500
; XY Stage minimum velocity (the velocity is scaled based on the move length).
XYMinVelocity=20
; XY Stage maximum velocity (the velocity is scaled based on the move length).
XYMaxVelocity=100
; XY Stage (moves less than this value will use the slow acceleration value)
XYSlowAccelerationWindow=2
; XY Stage (moves closer to the edge of the stage than this value will use the fast acceleration value)
XYEdgeMargin=2
; Z axis acceleration to use for all stage control moves.
ZAcceleration=500
; Z axis velocity to use for all stage control moves.
ZVelocity=100
; Z axis small step size (can be reduced for finer control)
ZSmallStep=0.005
; Z axis medium step size (can be reduced for finer control)
ZMediumStep=0.001
```

3.6.1 GUI Commands

It's possible to add GUI Commands to the MotionSynergyGUI. The MotionSynergyGUI will read a series of GUI commands defined in the *MotionSynergyGUI.cfg* configuration and display these commands on the Scripts tab. Each GUI command has a name, an optional set of parameters and an associated script to execute. When the user clicks on a GUI command button, the associated script will be executed.

The format used to specify GUI Commands is as follows:

```
[GUICommands]
; Defines the order of commands to display on the UI. If a command is not listed here, it will not be displayed on the UI.
Order=Command1,Command2,...
```

Each command is then defined in its own section:

```
[Command1]
; Display name appears on the command button.
DisplayName=Command 1
; true = display a separator (horizontal line) after this command, false = do not display a separator (this allows commands to be grouped together)
```

```
ShowSeparator=true
; Full name and path of the script to execute for this command.
ScriptName=[SupportFolder]\Scripts\Command1.lua
; Optional series of parameters
Parameter1=Parameter 1 Fields...
Parameter2=Parameter 2 Fields...
...
ParameterN=Parameter N Fields...
```

A command may have an optional list of parameters. Parameter values can be input by the user or hardcoded in the configuration file itself.

Each parameter has a number of fields. Fields are comma-separated. The first two fields are the same for all parameters. Subsequent fields are dependent on the parameter source and type.

```
ParameterN=ParameterName,Source,...
```

- **ParameterName** - Name of the parameter. This must match a parameter of the script being invoked.
- **Source** - The input source of the parameter value.
 - **Fixed** - The value is specified directly in configuration as part of this parameter definition.
 - **User** - The user is able to enter a value via the UI.

3.6.1.1 Fixed Parameter Types

A Fixed parameter has the following set of fields.

```
ParameterN=ParameterName,Source,Value
```

The *Value* will be assigned to the parameter value when the script command is executed. It's treated as a string but can be used to pass numeric values or boolean values (true/false) to the script.

3.6.1.2 User Parameter Types

A User parameter is followed by a *Type* and then one or more fields dependent on the type.

```
ParameterN=ParameterName,Source,Type,...
```

The following Types are supported:

- **ConfigKeyList** - Populates a drop down list with a set of values retrieved from a configuration setting (i.e. Section.Key). The configuration setting should contain a comma-separated list of values to use.
- **ConfigKeyValueList** - Populates a drop down list with a set of keys retrieved from a configuration section containing a set of key/value pairs (e.g. a set of Axis named positions).
- **DoubleRange** - Displays a numeric field which allows the user to enter a floating point value within a specified range.
- **FixedList** - Populates a drop down list with a set of values specified in a subsequent field of this parameter definition.
- **IntegerRange** - Displays a numeric field which allows the user to enter an integer value within a specified range.

- String - Displays a text field which allows the user to enter a string value.

3.6.1.2.1 ConfigKeyList

The ConfigKeyList parameter type has the following fields:

`ParameterN=ParameterName,Source,Type,ConfigurationSetting,Default`

- ConfigurationSetting - Specifies a configuration setting containing a comma-separated list of values the user can choose from.
- Default - An optional default value to select in the list. If no default is specified or the default doesn't match any values from the list, the first value in the list will be selected.

Example:

```
Parameter2=MoveProfile,User,ConfigKeyList,Instrument.CommonAxisMoveProfiles
[Instrument]
CommonAxisMoveProfiles=HomingProfile,StandardMoveProfile
```

3.6.1.2.2 ConfigKeyValueList

The ConfigKeyValueList parameter type has the following fields:

`ParameterN=ParameterName,Source,Type,ConfigurationSection,Default`

- ConfigurationSection - Specifies a configuration section containing a list of key/value pairs. The keys will be displayed in a list the user can choose from.
- Default - An optional default key to select in the list. If no default is specified or the default doesn't match any keys from the list, the first key in the list will be selected.

Example:

```
Parameter1=NamedPosition,User,ConfigKeyValueList,XAxisNamedPositions,NegativeLimit
[XAxisNamedPositions]
; These named positions are for a 100mm stage with home in the center.
Home=0
NegativeLimit=-50
PositiveLimit=50
```

3.6.1.2.3 DoubleRange

The DoubleRange parameter type has the following fields:

`ParameterN=ParameterName,Source,Type,Min,Max,Step,DecimalDigits,Default`

- Min - Specifies the minimum value that can be entered.
- Max - Specifies the maximum value that can be entered.
- Step - Specifies the step (increment) that is added or subtracted from the current value when the spinner up or down arrow is clicked.
- DecimalDigits - Specifies the number of decimal digits to allow when entering a floating point number.
- Default - The default value to set the control to on startup.

Example:

```
Parameter2=Setpoint,User,DoubleRange,20,60,0.5,2,37
```

3.6.1.2.4 FixedList

The FixedList parameter type has the following fields:

`ParameterN=ParameterName,Source,Type,ValuesList,Default`

- ValuesList - Specifies a colon-separated list of values the user can choose from.
- Default - An optional default value to select in the list. If no default is specified or the default doesn't match any values from the list, the first value in the list will be selected.

Example:

`Parameter1=ImageCaptureMode,User,FixedList,Raw:Preprocessed:Best,Best`

3.6.1.2.5 IntegerRange

The IntegerRange parameter type has the following fields:

`ParameterN=ParameterName,Source,Type,Min,Max,Step,Default`

- Min - Specifies the minimum value that can be entered.
- Max - Specifies the maximum value that can be entered.
- Step - Specifies the step (increment) that is added or subtracted from the current value when the spinner up or down arrow is clicked.
- Default - The default value to set the control to on startup.

Example:

`Parameter1=Green,User,IntegerRange,0,100,5,50`

3.6.1.2.6 String

The String parameter type has the following fields:

`ParameterN=ParameterName,Source,Type,Default`

- Default - The default value to set the control to on startup.

Example:

`Parameter4=Path,User,String,C:\ProgramData\MyCompany\Data`

3.7 Configuring the Logger

The diagnostic logging infrastructure has a separate *Logger.cfg* file which contains settings for controlling how much diagnostic logging information is generated, the format of the output and the destination sinks (i.e. log files, console, etc).

3.7.1 Controlling the amount of information logged.

Each log message the software outputs has an associated *LogLevel*. The default log level is set to *Info* (Severity 2). This level ensures information messages, warnings and errors are logged.

`[Core]
LogLevel=Info`

Log	Severity	Purpose
-----	----------	---------

Level		
Trace	0	Logs software function calls at a very fine grained (verbose) level of detail. Can be useful when diagnosing software defects.
Debug	1	Logs software function calls at a coarse level of detail. Can be useful when diagnosing software defects.
Info	2	Outputs information useful for observing the current operation being performed by the software.
Warning	3	Outputs a warning indicating the software has encountered an unexpected result but can continue without intervention.
Error	4	Outputs a error indicating the software has encountered an unexpected result which it cannot recovery from without a re-initialization.
Fatal	5	Outputs a fatal error indicating the software has entered an unknown state and must be restarted.

To change the amount of information logged, set the *LogLevel* in the configuration file. e.g. To include debug information in the logs, change the *LogLevel* from *Info* to *Debug*. You can also use the MotionSynergyGUI to change this setting. It takes effect immediately and updates the *LogLevel* in the configuration file.

```
[Core]
LogLevel=Debug
```

To disable all logging, set the *DisableLogging* setting to true. e.g.

```
[Core]
DisableLogging=true
```

Note this will not disable log messages displayed in the MotionSynergyGUI. The MotionSynergyGUI will always display log messages with a log level of *Info* or above.

3.7.2 Log Sinks

The logging configuration allows you to define multiple log sinks. A log sink specifies a destination to write messages, a filter to determine which messages are logged and a format string specifying which fields are logged for each message.

Log sinks can be added or removed from the configuration file as required. The format of a log sink configuration is as follows:

```
[Sinks.Diagnostics]
Destination=TextFile
FileName="%Y-%m-%d_%H%M%S_Debug.log"
Format="%TimeStamp% [%ThreadID%] [%Severity%] %Message% [%ComponentName%] [%SourceLocation%]"
Filter="%Channel% matches "diagnostics""
RotationSize=52428800
RotationTime=00:00
Asynchronous=true
AutoFlush=true
```

Each log sink configuration has its own section in the form of:

```
[Sinks.Name]
```

Each sink has a number of settings explained in the following sections.

3.7.3 Log Destination

```
Destination=TextFile/Console
```

where destination can be a *TextFile* or the *Console*

```
FileName="log filename"
```

Only applicable for *TextFile* sinks. Specifies the name of the log file. Note that a timestamp can be included in the filename using a format string such as *%Y-%m-%d_%H%M%S*, where the timestamp fields are as follows:

- %Y - Year (e.g. 2022)
- %m - Month (e.g. 06)
- %d - Day (e.g. 27)
- %H - Hour (e.g. 13)
- %M - Minute (e.g. 55)
- %S - Second (e.g. 37)

3.7.4 Log Format String

```
Format="log format string"
```

Specifies a format string for each log entry which can include one or more of the following fields:

- %TimeStamp% - Timestamp of when the log entry was created.
- %ThreadID% - The Id of the thread which created the log entry.
- %Severity% - The Log Level (Severity) of the log entry.
- %Message% - The message writtem to the log.
- %ComponentName% - The name of the software component which created the log entry.
- %SourceLocation% - The source code location which created the log entry.

3.7.5 Log Filter

The log filter allows you specify different log levels for different log sinks. The *Filter* setting can include a comparison against the log *Severity* (each *LogLevel* has a corresponding *Severity*). Log messages are first filtered by the global *LogLevel* in the *[Core]* section and then filtered by the *Severity* for an individual log sink.

e.g. To filter at Debug level and above, add a *Severity* comparison of *>= 1*:

```
Filter="(%Severity% >= 1) and (%Channel% matches "diagnostics")"
```

or to only log error or fatal messages, set the *Severity* comparison to a 4.

```
Filter="(%Severity% >= 4) and (%Channel% matches "diagnostics")"
```

3.7.6 Rotating Log Files

A log file can be 'rotated' whenever it reaches a specified size limit and/or at a specific time each day. Rotating ensures log files do not grow to an unmanagable size in applications that run for

days or weeks without being restarted. The default configuration will rotate the diagnostic log file whenever it reaches 50Mb and at midnight every day. e.g.

```
RotationSize=52428800
RotationTime=00:00
```

To disable rotating of log files based on size or time of day, simply remove the *RotationSize* or *RotationTime* setting as required.

3.7.7 Logger Write Behaviour

The *Asynchronous* and *AutoFlush* settings specify how each log entry is written to the log file. These settings should not be changed. Doing so may affect the performance of the logging infrastructure.

3.7.8 More information

The logging infrastructure uses the *Boost.Log v2* library. More information can be found here: [Boost.Log v2 Library](#)

3.7.9 Default Logging Configuration

The default *Logger.cfg* file is as follows:

```
# Motion Synergy API Logging Configuration
# Severity: 0=trace, 1=debug, 2=info, 3=warning, 4=error, 5=fatal

[Core]
DisableLogging=false

[Sinks.InfoConsole]
Destination=Console
Format="[%Severity%] %Message%"
Filter="(%Severity% >= 2) and (%Channel% matches "diagnostics")"
Asynchronous=false
AutoFlush=true

[Sinks.Diagnostics]
Destination=TextFile
FileName="%Y-%m-%d_%H%M%S_Debug.log"
Format="%TimeStamp% [%ThreadID%] [%Severity%] %Message% [%ComponentName%] [%SourceLocation%]"
Filter="(%Severity% >= 2) and (%Channel% matches "diagnostics")"
RotationSize=52428800
RotationTime=00:00
Asynchronous=true
AutoFlush=true
```

4 .NET Software Integration

Dover Motion provides a .NET assembly (dll) called *Motion Synergy API* for rapid integration with .NET applications (e.g. C#) or other 3rd-party applications which support .NET integration (e.g. LabVIEW). The *Motion Synergy API* library provides a high level *IAxis* interface for each axis of motion as well as supporting the following basic infrastructure:

- Scripting
- Configuration
- Logging
- Error Reporting
- Device Emulation

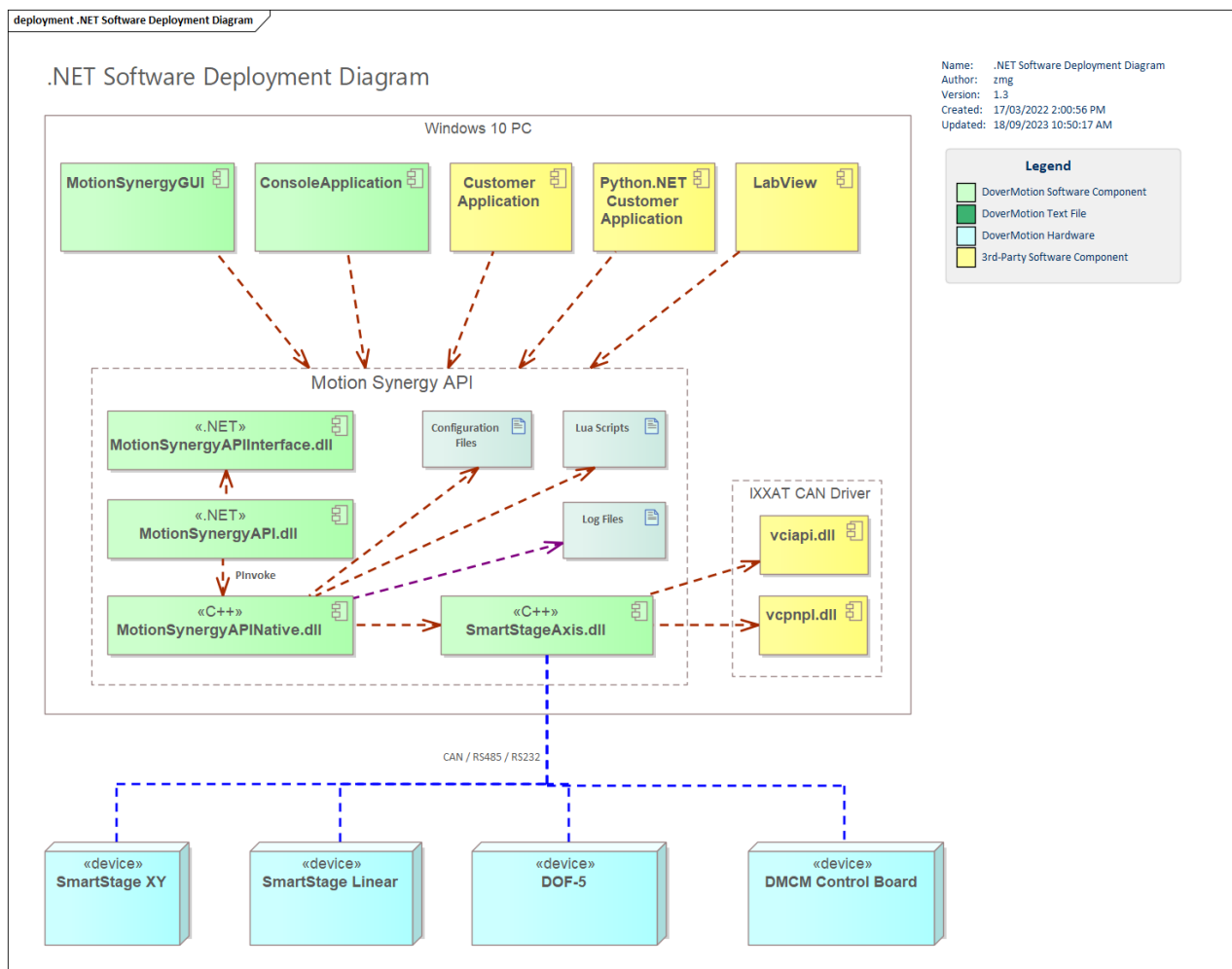
The *Motion Synergy API* library depends on the *MotionSynergyAPIInterface.dll* which defines the interface to the library. It also depends on the *SmartStageAxis.dll* which communicates with the *Magellan* motion control chip(s) used in the Dover Motion products via one of the supported communication links (RS232, RS485 or CAN).

The *Motion Synergy API* assemblies target the following .NET frameworks:

- Windows - net462 and net8.0
- Linux - net8.0

The Windows *net462* assemblies can be found in the top-level folder alongside *MotionSynergyGUI.exe*. These should be used for .NET 4.6.2 Applications and LabView. The Windows *net8.0* assemblies can be found in the *SourceCode/Examples/CSharp/ConsoleApplication/Libraries/*. These should be used for .NET 8.0 and above Applications.

The following diagram illustrates where *Motion Synergy API* sits in the software system:



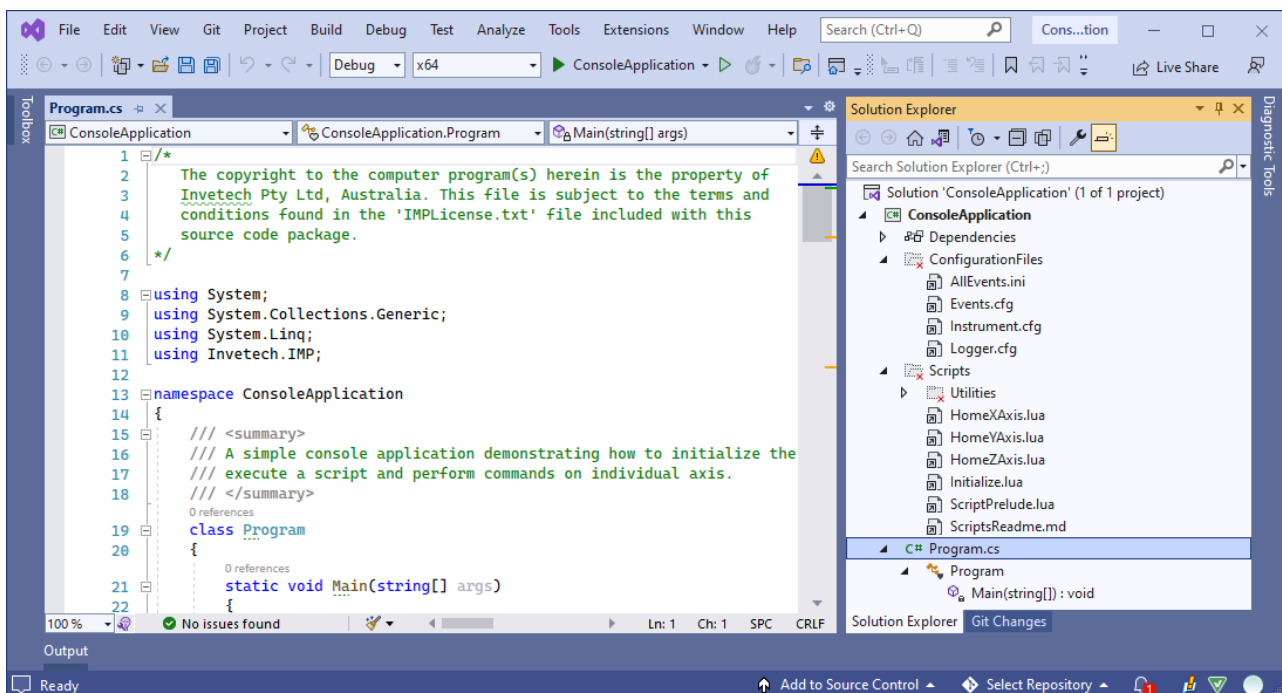
4.1 Building and Running the C# Console Application

There is an example which demonstrates how to integrate the *Motion Synergy API* library into a C# console application. It's located in the following folder:

\SourceCode\Examples\CSharp\ConsoleApplication

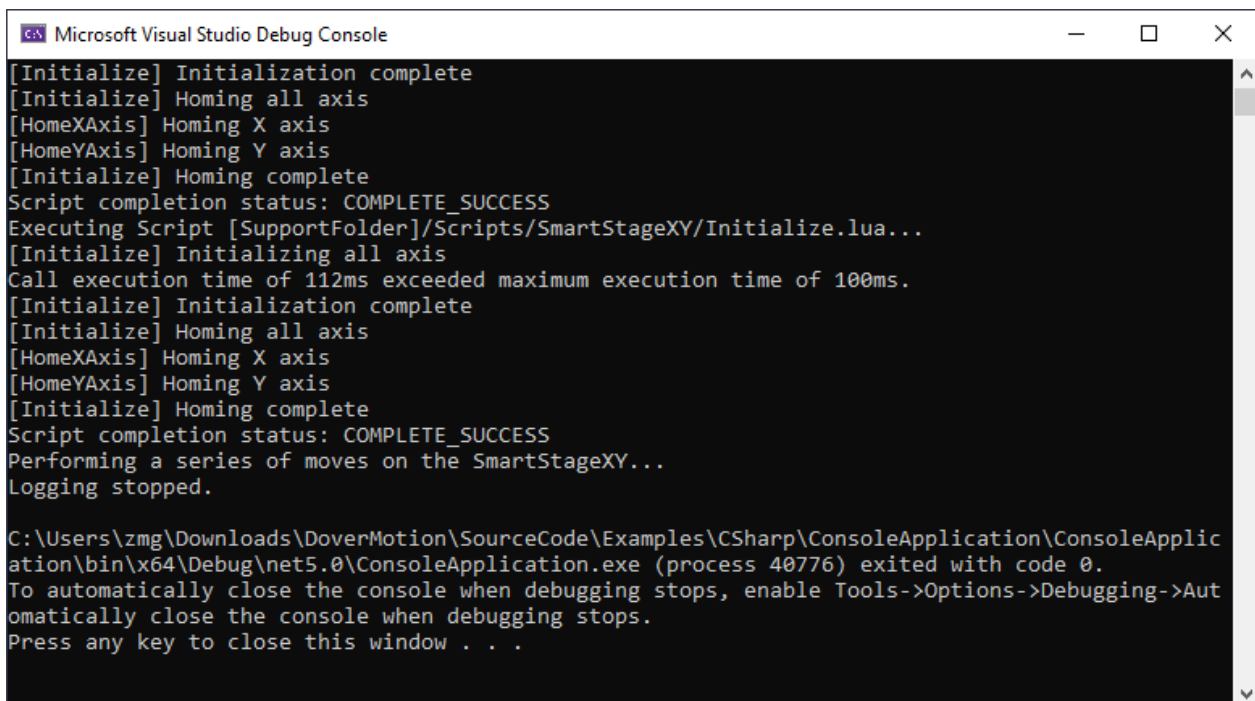
4.1.1 Windows

Within this folder is a *ConsoleApplication.sln*. You can build this solution with Microsoft Visual Studio 2022.



Ensure the *MotionSynergyGUI* application has been run and you have selected your product type and communications settings. You should have tested your setup to ensure you can control your Dover Motion product.

You can run this application and it will use the *SupportFolder/Instrument.cfg* to initialize and perform a series of moves applicable to your Dover Motion product.



4.1.2 Linux

Before you build the example applications on Linux, you will need to configure your system on Windows using the *MotionSynergyGUI* application. You will then need to copy the configuration

files (*.cfg) and entire Scripts folder from the SupportFolder on Windows into the SupportFolder on Linux. When you build the example application, the build procedure will copy these files into the output bin folder.

The example application has been tested on Ubuntu 22.04 LTS. Before building the C# application you will need to install the .NET 8.0 SDK.

```
sudo apt-get update
sudo apt-get install -y dotnet-sdk-8.0
```

Within the \SourceCode\Examples\CSharp\ConsoleApplication\ConsoleApplication folder is a *ConsoleApplication.csproj* project file. You can build this using dotnet on Linux as follows:

```
cd SourceCode/Examples/CSharp/ConsoleApplication/ConsoleApplication/
dotnet build ConsoleApplication.csproj
```

To run the ConsoleApplication, you need to change to the bin directory.

```
cd build_linux_x64_debug/bin
./ConsoleApplication
```

e.g.

```
root@INVMELLT0353:~/test# cd SourceCode/Examples/CSharp/ConsoleApplication/ConsoleApplication/
root@INVMELLT0353:~/test/SourceCode/Examples/CSharp/ConsoleApplication/ConsoleApplication# dotnet build ConsoleApplication.csproj
MSBuild version 17.4.8+6918b863a for .NET
Determining projects to restore ...
All projects are up-to-date for restore.
ConsoleApplication -> /root/test/SourceCode/Examples/CSharp/ConsoleApplication/ConsoleApplication/build_linux_x64_debug/bin/ConsoleApp
lication.dll

Build succeeded.
0 Warning(s)
0 Error(s)

Time Elapsed 00:00:01.03
root@INVMELLT0353:~/test/SourceCode/Examples/CSharp/ConsoleApplication/ConsoleApplication# cd build_linux_x64_debug/bin/
root@INVMELLT0353:~/test/SourceCode/Examples/CSharp/ConsoleApplication/ConsoleApplication/build_linux_x64_debug/bin# ./ConsoleApplicatio
n
MotionSynergyAPI C# Example!
Logging started, using configuration at: "/root/test/SourceCode/Examples/CSharp/ConsoleApplication/ConsoleApplication/build_linux_x64_de
bug/bin/Logger.cfg"
```

4.2 Building the MotionSynergyAPI

The *Motion Synergy API* library can be built from source (available on request). This allows the API to be customized for specific applications. It's located in the following folder:

```
\SourceCode\MotionSynergyAPI
```

The *Motion Synergy API* library includes both C# and C++ source code. You must set up your development environment to build C++. This is covered in the **C++ Software Integration** section.

4.2.1 Windows

Within this folder is a *createVisualStudioSolution.ps1* which will create a MotionSynergyAPI.sln solution for Visual Studio 2022 in the SourceCode\build_windows_x64 folder. You can then load and build this solution in Visual Studio. This solution includes a library and a number of unit tests. The solution doesn't include an executable.

4.2.2 Linux

Within this folder are two shell scripts which can be used to build the *Motion Synergy API* library on Linux. You must set up your Linux development environment to build C++. This is covered in the **C++ Software Integration** section.

```
./createLinux.sh  
./buildLinux.sh
```

4.3 LabVIEW Software Integration

The *Motion Synergy API* library can also be integrated into a LabVIEW application using LabVIEW's .NET Connectivity Nodes. In order to run the *Motion Synergy API* from LabVIEW, the following folders and files must be placed alongside the LabVIEW project (i.e. MyProjectName.vi).

- ProgramDataFolder
- SupportFolder
- MotionSynergyAPI.dll
- MotionSynergyAPIInterface.dll
- MotionSynergyAPINative.dll
- SmartStageAxis.dll
- vciapi.dll
- vcinpl.dll

The *MotionSynergyAPI.dll* and *MotionSynergyAPIInterface.dll* must both be referenced from a LabVIEW project. The *MotionSynergyAPI.dll* is needed to get a reference to the *IMotionSynergyAPI* interface using the *MotionSynergyAPINative()* constructor and the *MotionSynergyAPIInterface.dll* contains all the data types and interface definitions provided by the library.

4.3.1 Running the Example LabVIEW Project

There is an example LabVIEW Project called *MoveAxisExample.vi* which demonstrates how to access the *Motion Synergy API* from LabVIEW and perform a single axis move. It's located in the following folder:

```
\\SourceCode\\Examples\\LabVIEW
```

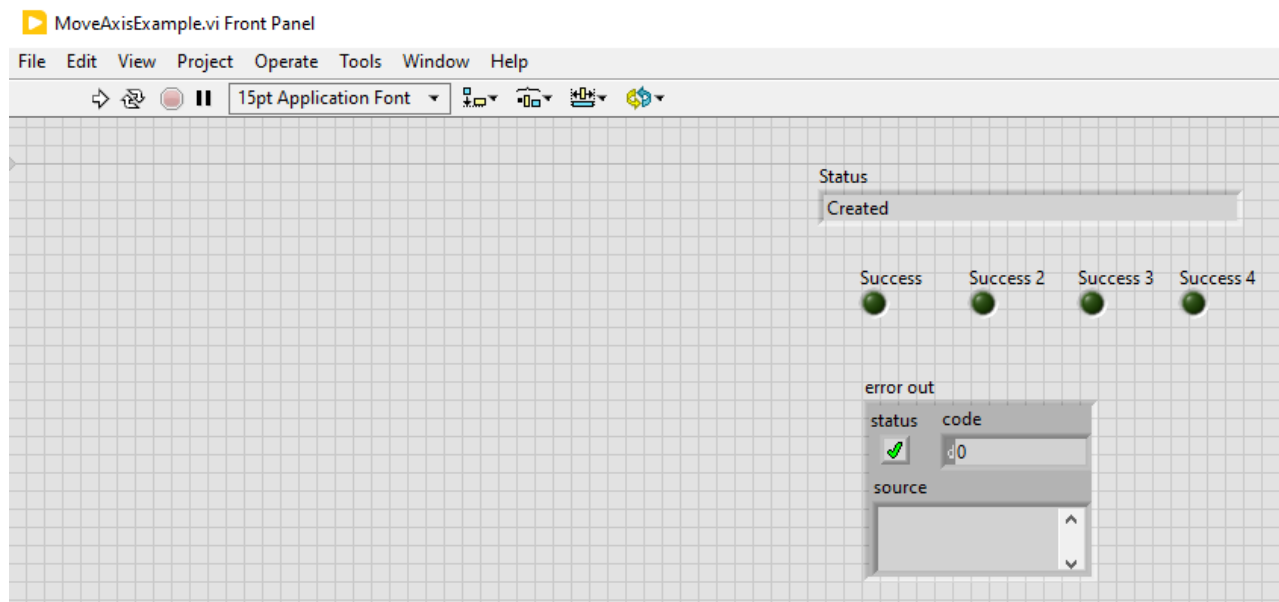
There are a number of steps you will need to follow to ensure the LabVIEW integration is successful.

1. Ensure you have run the *MotionSynergyGUI* and selected your product and communication settings. You should have tested your setup to ensure you can control your Dover Motion product.
2. Copy the *MoveAxisExample.vi* from the `\\SourceCode\\Examples\\LabVIEW` folder into the top-level folder alongside the *MotionSynergyAPI.dll*. This is the easiest way to ensure the project can find all the DLLs and support files it needs.
3. You must use the *net462* version of *MotionSynergyAPI.dll* and *MotionSynergyAPIInterface.dll* which are compatible with LabVIEW and located in the top-level folder alongside the *MotionSynergyGUI.exe*.
4. You must use a 64-bit version of LabVIEW. This example has been tested with LabVIEW 2021 64-bit (21.0.1f1). It should work with older or newer 64-bit versions of LabVIEW which support .NET connectors.

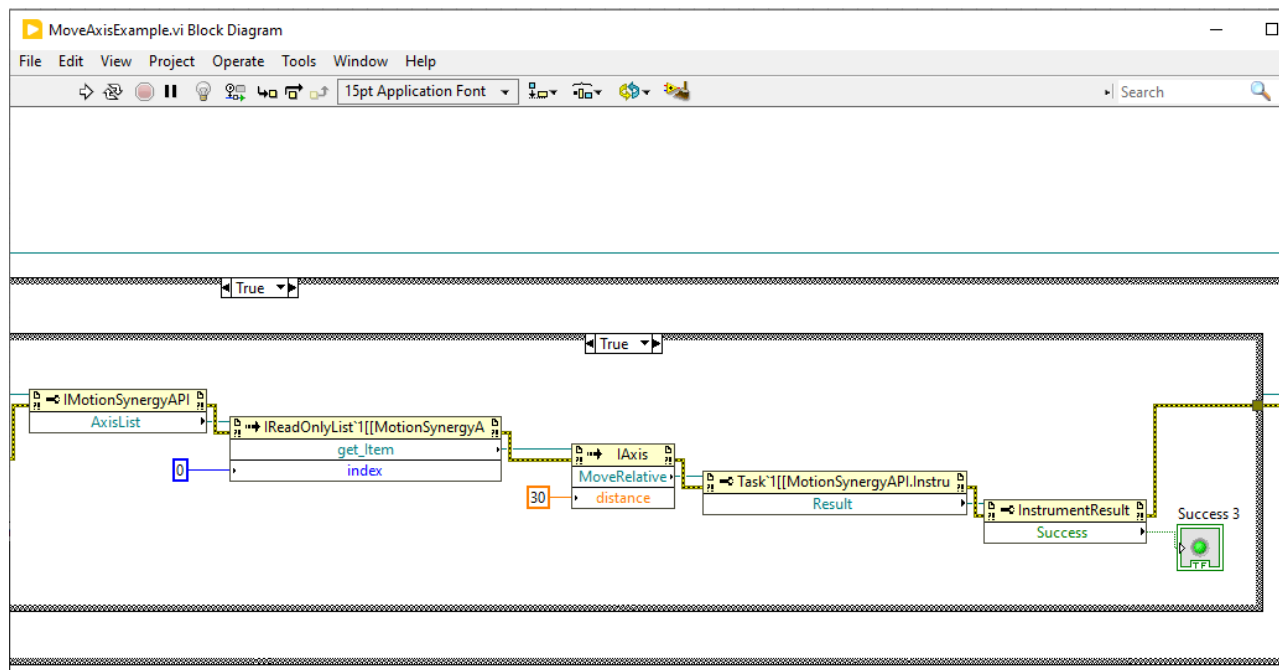
- You must disable shadow-copying of assemblies by adding the following line to *C:\Program Files\National Instruments\LabVIEW 2021\LabVIEW.ini*. This ensures the assemblies are loaded from their original location and the dependent DLLs and support files can be found.

`AppDomainSetup_ShadowCopyFiles=False`

- Load the project in LabVIEW. The front panel should look similar to the following:



- If you are using a DOF-5 you will need to adjust the distance the axis is moved before running the project. You should adjust the 'distance' constant from 30mm to a value between -5 and 5.



4.3.2 LabVIEW and Async Methods

Most of the methods on the axis interface are asynchronous. When you call one of these methods from LabVIEW, you will receive a reference to an async task. To wait for the task to complete, you need to access the *Result* property of the task. In the above example you can see there is an invoke node which calls *MoveRelative*. This invoke node will start the move and return a task. The next node is a property node which accesses the *Result* property on the task. This will block until the move is complete and the result of the move can be reported. To move a second axis in parallel, you could invoke a *Move** function on the second axis before you access the *Result* property for the first move.

4.3.3 LabVIEW Troubleshooting

If there are errors reported when loading the example project, which indicate the .NET assembly could not be loaded or there are invalid property or invoke nodes, it's most likely caused by LabVIEW being unable to load the *Motion Synergy API* .NET assemblies due to security restrictions. The following article from National Instruments explains how to resolve this:

[Error Loading .NET Assemblies in LabVIEW](#)

4.4 Writing Scripts

The *MotionSynergyGUI* and the *Motion Synergy API* provide the ability to execute [Lua](#) scripts. Each script can control one or more axis. Scripts also have access to the configuration and logging infrastructure. Scripting provides a way of rapidly prototyping workflows without the need to recompile an application.

The Lua Scripting API documentation can be viewed here:

`\LuaScriptingAPI\index.html`

The *Motion Synergy API* comes with scripts for initializing and homing each product. The scripts are located in the following folders:

```
\SupportFolder\Scripts
\SupportFolder\Scripts\DMCM
\SupportFolder\Scripts\DOF-5
\SupportFolder\Scripts\SmartStageXY
\SupportFolder\Scripts\SmartStageLinear
\SupportFolder\Scripts\Utilities
```

Here is the initialize script (`\SupportFolder\Scripts\SmartStageXY\Initialize.lua`) for the SmartStageXY:

```
require("ScriptPrelude").Run()

require("SmartStageXY.HomeXAxis")
require("SmartStageXY.HomeYAxis")

local config = require("lua_config")
local instrumentConfig = config.get_config()

local Log = require("Utilities.LogWrapper").New("Initialize")

function InitializeAllAxis()
    Log:info("Initializing all axis")

    XAxis.Initialize()
    YAxis.Initialize()
```

```

        Log:info("Initialization complete")
    end

    function HomeAllAxis()
        Log:info("Homing all axis")

        HomeXAxis()
        HomeYAxis()

        Log:info("Homing complete")
    end

    InitializeAllAxis()
    HomeAllAxis()

```

The following is an example script which will perform a series of short moves along the X axis. It can be used with the SmartStageXY, SmartStageLinear or DMCM.

```

local clock = require 'lua_clock'
local log = require 'lua_logger'

require("ScriptPrelude").Run()

XAxis.SelectMoveProfile("XAxisStandardMoveProfile")
XAxis.Wait()

-- Perform 5 short moves in the positive direction
for i = 1,5 do
    log.info("Positive move " .. tostring(i))

    XAxis.MoveRelative(5)
    XAxis.Wait()

    log.info("Pause for one second (1000ms).")
    clock.sleep_ms(1000)
end

-- Perform 5 short moves in the negative direction
for i = 1,5 do
    log.info("Negative move " .. tostring(i))

    XAxis.MoveRelative(-5)
    XAxis.Wait()

    log.info("Pause for one second (1000ms).")
    clock.sleep_ms(1000)
end

```

Here is a similar script for the DOF-5 which performs a series of short moves along the Z axis.

```

local clock = require 'lua_clock'
local log = require 'lua_logger'

require("ScriptPrelude").Run()

ZAxis.SelectMoveProfile("ZAxisStandardMoveProfile")
ZAxis.Wait()

-- Perform 5 short moves in the positive direction
for i = 1,5 do
    log.info("Positive move " .. tostring(i))

    ZAxis.MoveRelative(0.75)
    ZAxis.Wait()

    log.info("Pause for one second (1000ms).")
    clock.sleep_ms(1000)
end

-- Perform 5 short moves in the negative direction

```

```

for i = 1,5 do
    log.info("Negative move " .. tostring(i))

    ZAxis.MoveRelative(-0.75)
    ZAxis.Wait()

    log.info("Pause for one second (1000ms).")
    clock.sleep_ms(1000)
end

```

Save the above script to the *SupportFolder/Scripts* folder as *ShortMoves.lua*.

Test the script by running it from the *MotionSynergyGUI* application.

You can also execute this script from a C# application. See the Example C# Console application which demonstrates the use of the *IMotionSynergyAPI.Scripting.ExecuteScript()* function.

4.5 Error Reporting

All commands will return an *InstrumentResult*. The *InstrumentResult* has a *Success* property which indicates if a command succeeded or failed. If the command failed, the *AlertCode* property can be examined to determine what specifically caused the failure and the *Description* property used to determine under what context the failure occurred.

When a command executes, it may raise one or more Alerts. Each *Alert* has a severity which ranges from *Low* to *Safety Critical*.

Severity	Description
Low	Alert is raised for information purposes only. The system will not take any action based on alerts with this severity.
Medium	Alert indicates the current command has failed but the system is still in a valid state to execute further commands.
High	Alert indicates the current command has failed. The system will be placed in an 'Error' state and a re-initialization must be performed before further commands attempted.
SafetyCritical	Alert indicates the current command has failed due to an unexpected error. The state of the MotionSynergyAPI software is unknown and a restart is required before attempting further commands.

The complete list of Alerts is documented in section 4.5.26. The *AlertCode* reported as part of the *InstrumentResult* is the first alert raised during command execution with a severity of *High* or *Safety Critical*.

MotionSynergyAPI will never throw an exception. All errors are reported via the *InstrumentResult*.

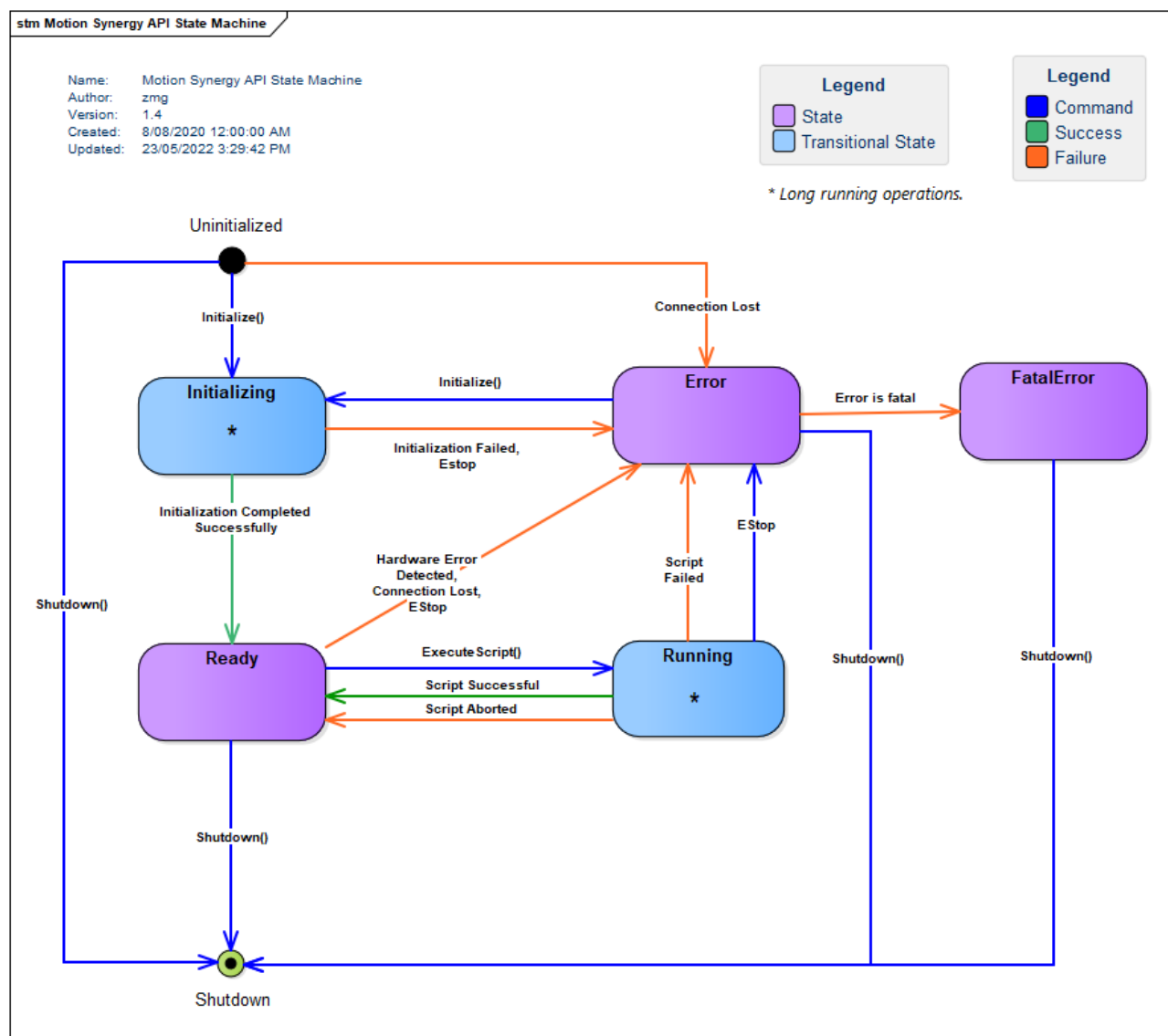
4.6 MotionSynergyAPI State Machines

This section describes the state machines maintained by MotionSynergyAPI.

4.6.1 MotionSynergyAPI State Machine

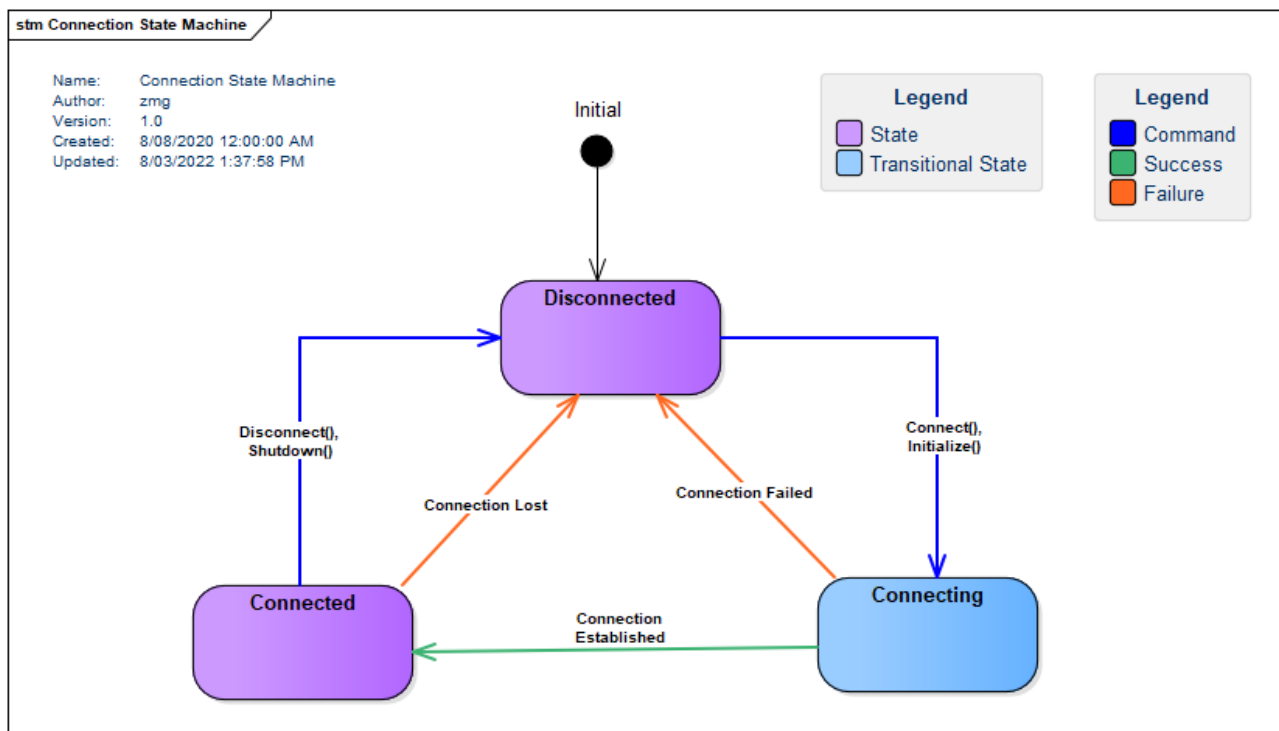
The following high-level state machine describes the overall state of the Dover Motion hardware.

See the *IMotionSynergyAPI* interface for a detailed description of which interface functions can be called in each state. If a function is listed as able to be called in a particular state, but no transition is shown on this diagram, the state machine will remain in the same state. If a function is not listed as able to be called, the function call will return an error result. While any *IMotionSynergyAPI* action is running (i.e. before a Result has been returned), attempting to start another action will result in that action immediately returning a failed Result.



4.6.2 Connection State Machine

The following state machine describes the state of the connection between MotionSynergyAPI and the hardware.



4.7 Public Interface

4.7.1 Alerts.Alert Class

```
public class Alert
```

An alert allows almost any component within MotionSynergyAPI to raise a notification to report the occurrence of some sort of action, warning or error. Once an alert has been raised, it remains 'active' until cleared. It may be cleared by the component which raised it or by an MotionSynergyAPI client software component by calling `IAleris.ClearAlert(uint)` or `IAleris.ClearAllAlerts`. Examples of alerts include a device warning such as 'Axis PositionError' or an error such as 'Unable to establish connection'.

4.7.1.1 Properties

4.7.1.1.1 Code

```
public AlertCode Code { get; }
```

A unique code to identify the type of Alert.

4.7.1.1.2 Description

```
public string Description { get; }
```

Provides a description and additional context regarding the specific instance of the Alert. This information is useful for diagnosing hardware and software faults but should not be displayed to the end user of the instrument.

4.7.1.1.3 ReportId

```
public uint ReportId { get; }
```

A unique Id to identify this instance of an Alert. Can be used to clear this specific instance of the alert using `IAerts.ClearAlert(uint)`.

Details

The Id is not unique across restarts of MotionSynergyAPI.

4.7.1.1.4 Severity

```
public AlertSeverity Severity { get; }
```

The severity of the Alert dictates the type of action the software must take. Low and Medium severity Alerts are non-fatal and serve as information or warnings. High and SafetyCritical indicate a fatal error has occurred and require the software to be re-initialized, restarted or a service call to rectify.

4.7.1.1.5 Source

```
public string Source { get; }
```

Identifies the component which raised the Alert.

4.7.1.1.6 Timestamp

```
public DateTime Timestamp { get; }
```

Indicates the date and time the Alert was first raised by MotionSynergyAPI.

4.7.1.1.7 UserDescription

```
public string UserDescription { get; }
```

Provides a description which can be displayed to the end user of the instrument. This may not be the actual string but a key used to look up a localized version of the string.

4.7.1.1.8 UserTitle

```
public string UserTitle { get; }
```

Provides a short title for the alert which will be to the end user of the instrument. This may not be the actual string but a key used to look up a localized version of the string.

4.7.1.2 Public Methods

4.7.1.2.1 Constructor: Alert()

```
public Alert(AlertCode code,
    uint reportId,
    AlertSeverity severity,
    string source,
    DateTime timestamp,
    string description,
    string userDescription,
    string userTitle)
```

Create an alert with the following inputs. This constructor is used from managed interop layer.

Parameters:

code	Alert code which should be an enum. Note the specific enum type will vary for each project, depending on the set of alerts used by the project.
reportId	Raw report Id
severity	Alert severity, mapped to appropriate enum value
source	Alert source

timestamp Seconds since Epoc (Jan 1, 1970, 00:00:00) UTC

description Alert description text field

userDescription A description which can be displayed to the end user of the instrument

userTitle A short title for the alert which will be to the end user of the instrument

4.7.1.2.2 ToString()

```
public override string ToString()
```

Convert the Alert to a string.

Returns:

The alert in string form.

4.7.2 Alerts.ClearedEventArgs Class

```
public class ClearedEventArgs : EventArgs
```

Event indicating one or more Alerts have been cleared.

4.7.2.1 Properties

4.7.2.1.1 AlertReportIdList

```
public IReadOnlyList<uint> AlertReportIdList { get; }
```

Indicates the set of Alert(s) which have been cleared, each identified by their reported Id.

4.7.2.2 Public Methods

4.7.2.2.1 Constructor: Alerts.ClearedEventArgs()

```
public ClearedEventArgs(IReadOnlyList< uint > alertList)
```

Constructor.

Parameters:

alertList A list of alerts which have been cleared, each identified by their reported Id.

4.7.3 Alerts.AlertDetails Class

```
public class AlertDetails
```

A singleton class which provides information about each type of alert which can be generated by the system.

4.7.3.1 Public Methods

4.7.3.1.1 GetAlertInfo()

```
public AlertInfo GetAlertInfo(AlertCode alertCode)
```

Look up information for a specific type of Alert.

Parameters:

`alertCode` Alert code identifying the type of alert.

Returns:

The details for the specific type of Alert.

Exceptions:

`ArgumentOutOfRangeException` Indicates details for the requested type of alert could not be found. Should never happen if valid `AlertCodes` are used.

4.7.3.1.2 `GetInstance()`

```
public static AlertDetails GetInstance()
```

Provides access to the singleton instance.

Returns:

The `AlertDetails` instance.

4.7.3.2 Public Variables**4.7.3.2.1 All**

```
ReadOnlyDictionary<AlertCode, AlertInfo> All => m_alertDetails
```

Return the complete alert details dictionary.

4.7.4 Alerts.AlertInfo Class

```
public class AlertInfo
```

Provides information about a specific type of alert.

4.7.4.1 Properties**4.7.4.1.1 Code**

```
public AlertCode Code { get; }
```

A unique code to identify the type of Alert.

4.7.4.1.2 Description

```
public string Description { get; }
```

Provides a description and additional context regarding the specific instance of the Alert. This information is useful for diagnosing hardware and software faults but should not be displayed to the end user of the instrument.

4.7.4.1.3 Severity

```
public AlertSeverity Severity { get; }
```

The severity of the Alert dictates the type of action the software must take. Low and Medium severity Alerts are non-fatal and serve as information or warnings. High and SafetyCritical indicate a fatal error has occurred and require the software to be re-initialized, restarted or a service call to rectify.

4.7.4.1.4 UserDescription

```
public string UserDescription { get; }
```

Provides a description which can be displayed to the end user of the instrument. This may not be the actual string but a key used to look up a localized version of the string.

4.7.4.1.5 UserTitle

```
public string UserTitle { get; }
```

Provides a short title for the alert which will be to the end user of the instrument. This may not be the actual string but a key used to look up a localized version of the string.

4.7.4.2 Public Methods

4.7.4.2.1 Constructor: AlertInfo()

```
public AlertInfo(AlertCode alertCode,
    AlertSeverity alertSeverity,
    string description,
    string userDescription,
    string userTitle)
```

Constructor

Parameters:

alertCode	A unique code to identify the type of Alert.
alertSeverity	The severity of the Alert.
description	Provides a description of the alert.
userDescription	A description which can be displayed to the end user of the instrument
userTitle	A short title for the alert which will be to the end user of the instrument

4.7.5 Alerts.AlertRaisedEventArgs Class

```
public class AlertRaisedEventArgs : EventArgs
```

Event indicating a new Alert has been raised.

4.7.5.1 Properties

4.7.5.1.1 Alert

```
public Alert Alert { get; }
```

The Alert which has been raised.

4.7.5.2 Public Methods

4.7.5.2.1 Constructor: AlertRaisedEventArgs()

```
public AlertRaisedEventArgs(Alert alert)
```

Constructor.

Parameters:

alert	The alert being raised
-------	------------------------

4.7.6 BiquadFilter Class

```
public class BiquadFilter
```

Class used to calculate a set of coefficients for a Biquad filter.

4.7.6.1 Public Methods

4.7.6.1.1 Calculate()

```
public static BiquadFilterDefinition Calculate(BiquadFilterType filterType,
double frequency,
double qualityFactor,
double alphaOrBeta)
```

Calculate a set of coefficients for a Biquad filter.

Parameters:

filterType	The type of filter to generate. Valid values are None, LowPass, Notch, Lead, and Lag.
frequency	The frequency parameter used to generate the filter coefficients. Must be greater than zero.
qualityFactor	The quality factor parameter used to generate the filter coefficients. Must be greater than zero.
alphaOrBeta	The alpha or beta parameter used to generate the filter coefficients. Only used in Lead or Lag filters.

Returns:

A filter definition containing the calculated coefficients

Exceptions:

ArgumentOutOfRangeException The filterType parameter specified Unknown.

4.7.6.1.2 GetEmpty()

```
public static BiquadFilterDefinition GetEmpty(BiquadFilterType filterType=BiquadFilterType.None)
```

Return an empty filter definition (i.e. where the filter type is None and all other values are zero).

Parameters:

filterType	Specify a filter type other than the default of None.
------------	---

Returns:

The empty filter definition

4.7.7 CommunicationSettings Class

```
public class CommunicationSettings
```

Defines the Serial and CAN communication settings for an axis.

4.7.7.1 Properties

4.7.7.1.1 CanAddress

```
public ushort CanAddress { get; set; }
```

CAN Node ID address (0-127).

4.7.7.1.2 CanBaudRate

```
public CanBaudRate CanBaudRate { get; set; }
```

Baud rate to use for CAN mode (0-7).

4.7.7.1.3 SerialAddress

```
public ushort SerialAddress { get; set; }
```

Multi-drop address for use with RS485 (0-31).

4.7.7.1.4 SerialBaudRate

```
public SerialBaudRate SerialBaudRate { get; set; }
```

Baud rate to use for serial mode (1,200-460,800).

4.7.7.1.5 SerialProtocol

```
public SerialProtocol SerialProtocol { get; set; }
```

Protocol to use for serial mode.

4.7.7.2 Public Methods

4.7.7.2.1 Constructor: CommunicationSettings()

```
public CommunicationSettings()
```

Default constructor. Can be used when reading settings from file.

4.7.7.2.2 Constructor: CommunicationSettings()

```
public CommunicationSettings(SerialBaudRate serialBaudRate,
    SerialProtocol serialProtocol,
    ushort serialAddress,
    CanBaudRate canBaudRate,
    ushort canAddress)
```

Constructor

Parameters:

serialBaudRate Baud rate to use for serial mode (1,200-460,800).

serialProtocol Protocol to use for serial mode.

serialAddress Multi-drop address for use with RS485 (0-31).

canBaudRate Baud rate to use for CAN mode (0-7).

canAddress CAN Node ID address (0-127).

4.7.7.2.3 Constructor: CommunicationSettings()

```
public CommunicationSettings(CommunicationSettings other)
```

Copy constructor

Parameters:

other Settings to copy

4.7.8 ConnectionStateChangedEventArgs Class

```
public class ConnectionStateChangedEventArgs : EventArgs
```

Event indicating the state of the connection between MotionSynergyAPI and the Subsystem Controller has changed.

4.7.8.1 Properties

4.7.8.1.1 CurrentState

```
public ConnectionState CurrentState { get; }
```

The current (new) state of the connection.

4.7.8.1.2 PreviousState

```
public ConnectionState PreviousState { get; }
```

The previous state of the connection (i.e. prior to this event).

4.7.8.2 Public Methods

4.7.8.2.1 Constructor: ConnectionStateChangedEventArgs()

```
public ConnectionStateChangedEventArgs(ConnectionState currentState,
                                       ConnectionState previousState)
```

Constructor which sets the event properties.

Parameters:

currentState The current (new) state of the connection.

previousState The previous state of the connection (i.e. prior to this event).

4.7.9 DigitalInputEventArgs Class

```
public class DigitalInputEventArgs : EventArgs
```

Event indicating the state of a digital input device register has changed.

4.7.9.1 Properties

4.7.9.1.1 CurrentRegisterValue

```
public byte CurrentRegisterValue { get; }
```

The current (new) state of the digital input register.

4.7.9.2 Public Methods

4.7.9.2.1 Constructor: DigitalInputEventArgs()

```
public DigitalInputEventArgs(byte registerValue)
```

Constructor which sets the event properties.

Parameters:

registerValue The current value of the digital input register.

4.7.10 FirstProduct Class

```
public class FirstProduct
```

Contains the type of the first product in the configuration and its associated axis names.

4.7.10.1 Properties**4.7.10.1.1 AxisNames**

```
public List<string> AxisNames { get; set; }
```

The name(s) of the axes associated with this product.

4.7.10.1.2 ProductType

```
public string ProductType { get; set; }
```

Product type - SmartStage, DOF-5, DMCM, etc.

4.7.10.1.3 ProductVariant

```
public string ProductVariant { get; set; }
```

Product variant of the ProductType - SmartStageOF, SmartStageShuttle, SmartStageMonolithic, etc.

4.7.10.2 Public Methods**4.7.10.2.1 Constructor: FirstProduct()**

```
public FirstProduct(string productType,
    string productVariant,
    List< string > axisNames)
```

Constructor**Parameters:**

productType Product type - SmartStage, DOF-5, DMCM, etc.

productVariant Product variant of the Product type - SmartStageOF, SmartStageShuttle, SmartStageMonolithic, etc.

axisNames The name(s) of the axes associated with this product.

4.7.10.2.2 GetName()

```
public string GetName()
```

Get the name of this product.

Returns:

The name of this product.

4.7.11 InstrumentResult Class

```
public class InstrumentResult
```

Represents the success / failure of a command executed by MotionSynergyAPI. All synchronous and asynchronous commands will return an InstrumentResult. Includes an AlertCode to indicate the specific reason the command failed and a description of the context in which the failure occurred.

4.7.12 InstrumentSettings Class

```
public class InstrumentSettings
```

The set of instrument configuration settings to be applied during initialization. These settings are common to all applications.

4.7.12.1 Properties

4.7.12.1.1 ApplicationVersionString

```
public string ApplicationVersionString { get; set; }
```

Version of the Application using Motion Synergy API in string form.

4.7.12.1.2 ConfigurationFilename

```
public string ConfigurationFilename { get; set; }
```

Specifies the configuration file used during initialization. If this string is null or empty, it will use the default configuration file.

4.7.12.1.3 ConvertRelativePathsToAbsolute

```
public bool ConvertRelativePathsToAbsolute { get; set; }
```

If the ProgramDataFolder path or SupportFolder path is relative, convert it to an absolute path, using the locating of this assembly as the root.

Details

Supported on Windows only, not Linux.

4.7.12.1.4 ProgramDataFolder

```
public string ProgramDataFolder { get; set; }
```

Path to a writable folder which can be used by MotionSynergyAPI to store program data such as logs, configuration, etc (e.g. C:).

4.7.12.1.5 SupportFolder

```
public string SupportFolder { get; set; }
```

Path to a readable folder containing configuration files and scripts required for MotionSynergyAPI

4.7.13 InstrumentStateChangedEventArgs Class

```
public class InstrumentStateChangedEventArgs : EventArgs
```

Event indicating the high level state of the instrument has changed.

4.7.13.1 Properties

4.7.13.1.1 CurrentState

```
public InstrumentState CurrentState { get; }
```

The current (new) state of the instrument.

4.7.13.1.2 PreviousState

```
public InstrumentState PreviousState { get; }
```

The previous state of the instrument (i.e. prior to this event).

4.7.13.2 Public Methods

4.7.13.2.1 Constructor: InstrumentStateChangedEventArgs()

```
public InstrumentStateChangedEventArgs(InstrumentState currentState,
                                       InstrumentState previousState)
```

Constructor which sets the event properties.

Parameters:

`currentState` The current (new) state of the instrument.

`previousState` The previous state of the instrument (i.e. prior to this event).

4.7.14 InvalidEnumValueAttribute Class

```
public class InvalidEnumValueAttribute : Attribute
```

Attribute to indicate a particular enum value is not a valid value the user can select.

4.7.15 InvalidEnumValueHelper Class

```
public class InvalidEnumValueHelper
```

Provides the IsValueValid extension method.

4.7.15.1 Public Methods

4.7.15.1.1 IsValueValid< T >()

```
public static bool IsValueValid< T >(this T enumValue)
```

Extension method to determine if a specific enum value is invalid and should not be selected by the user.

Parameters:

`enumValue` The enum value to check

Returns:

true if the value is valid, false if not

4.7.16 IsDeselectableEnumValueHelper Class

```
public class IsDeselectableEnumValueHelper
```

Provides the IsValueDeselectable extension method.

4.7.16.1 Public Methods

4.7.16.1.1 IsValueDeselectable< T >()

```
public static bool IsValueDeselectable< T >(this T enumValue)
```

Extension method to determine if a specific enum value can be deselected by the user.

Parameters:

enumValue The enum value to check

Returns:

true if the value can be deselected, false if not

4.7.17 LogEntry Class

```
public class LogEntry
```

Holds a single log entry published by MotionSynergyAPIService.

4.7.17.1 Properties

4.7.17.1.1 Level

```
public LogLevel Level { get; }
```

The log level (Debug, Info, Warning, Error, Fatal).

4.7.17.1.2 Message

```
public string Message { get; }
```

The message logged.

4.7.17.1.3 ThreadId

```
public string ThreadId { get; }
```

The Id of the thread which created the log entry.

4.7.17.1.4 Timestamp

```
public string Timestamp { get; }
```

Indicates when the log entry was created.

4.7.17.2 Public Methods

4.7.17.2.1 Constructor: LogEntry()

```
public LogEntry(string timestamp,  
               string threadId,
```

```
LogLevel level,
string message)
```

Constructor

Parameters:

```
timestamp Indicates when the log entry was created.
threadId The Id of the thread which created the log entry.
level The log level (Debug, Info, Warning, Error, Fatal).
message The message logged.
```

4.7.18 ProgressUpdate Class

```
public class ProgressUpdate
```

A data structure used to provide periodic progress updates for long running operations (e.g. Initialize, RunTest). Note that the first progress update for an operation should set all properties (i.e. TimeRemainingInSeconds, TotalTimeInSeconds, CurrentStep) but subsequent progress updates may only change one or two properties (e.g. CurrentStep or TimeRemainingInSeconds).

4.7.19 UnDeselectableEnumValueAttribute Class

```
public class UnDeselectableEnumValueAttribute : Attribute
```

Attribute to indicate a particular enum value cannot be deselected by the user. The associated user control should disable selection in this case.

4.7.20 Alerts.IAlerts Interface

```
public interface IAlerts
```

Interface which allows a client to receive events whenever an Alert is raised or cleared and provides commands for clearing Alerts either individually or all at once.

4.7.20.1 Events

4.7.20.1.1 AlertRaised

```
public event AlertRaisedEventHandler AlertRaised
```

Event indicating a new Alert has been raised.

4.7.20.1.2 AlertsCleared

```
public event AlertsClearedEventHandler AlertsCleared
```

Event indicating one or more existing Alerts have been cleared.

4.7.20.2 Public Methods

4.7.20.2.1 ClearAlert()

```
public InstrumentResult ClearAlert(uint alertReportId)
```

Allows a single Alert to be cleared.

Details

This will generate an AlertsCleared event.

Parameters:

`alertReportId` The reported Id of the alert used to identify the instance to clear.

Returns:

An InstrumentResult which can be used to determine the success or failure of the command.

4.7.20.2.2 ClearAllAlerts()

```
public InstrumentResult ClearAllAlerts()
```

Allows all existing alerts to be cleared in a single operation.

Details

This will generate an AlertsCleared event.

Returns:

An InstrumentResult which can be used to determine the success or failure of the command.

4.7.20.2.3 RaiseAlert()

```
public InstrumentResult<uint> RaiseAlert(AlertCode alertCode)
```

Raise an alert with the specified alert code.

Parameters:

`alertCode` The alert code specifies the type of alert to raise.

Returns:

An InstrumentResult containing the reported Id of the alert which can be used to clear the alert.

4.7.21 IAxis Interface

```
public interface IAxis : IDevice
```

Interface to control an Axis device, a device capable of providing linear motion (e.g. x, y, z stage).

Details

All commands which run asynchronously return a .NET task that can be awaited on. If Task.Wait() is called directly on the task (instead of using await), there is no need to specify a timeout parameter. All commands executed through the .NET axis interface implement an internal timeout mechanism, which uses the axis device's DefaultTimeout_ms specified in the Instrument.cfg.

4.7.21.1 Properties

4.7.21.1.1 MotionControllerConfiguration

```
public IMotionControllerConfiguration MotionControllerConfiguration { get; }
```

An interface for reading and writing various motion controller settings from/to memory/NVRAM.

4.7.21.1.2 UserDefinedProfile

```
public IUserDefinedProfile UserDefinedProfile { get; }
```

An interface for performing User Defined Profile (UDP) moves and updating the associated user defined profile data.

4.7.21.2 Public Methods

4.7.21.2.1 Disable()

```
public Task<InstrumentResult> Disable()
```

Disable / de-energize the axis motor. This will remove power from the axis so it can be moved manually. If an encoder is present, it will continue to update as the axis is manually moved.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.2 DisableTriggerOnPosition()

```
public Task<InstrumentResult> DisableTriggerOnPosition()
```

Disable trigger on position (TOP).

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.3 Enable()

```
public Task<InstrumentResult> Enable()
```

Enable / energize the axis motor. This will restore power to the axis motor (after Disable()) has been called) so it can be commanded to move.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.4 EnableTraceCapture()

```
public Task<InstrumentResult> EnableTraceCapture(TraceTrigger trigger,
    uint period,
    IReadOnlyList< TraceVariable > variables,
    ulong waitMotionStabilizeUs=0)
```

Enable capture of trace data.

Details

If the trigger is OnMotionStart, trace capture will be automatically started when the next move begins and stopped once the move completes. If the trigger is OnMotionStartNoEnd, trace capture will be automatically started when the next move begins and must be stopped by calling StopTraceCapture(). If the trigger is Immediate, trace capture will be started immediately and must be stopped by calling StopTraceCapture().

Parameters:

<code>trigger</code> <code>OnMotionStartNoEnd).</code>	Trigger to start the trace capture (e.g. Immediate, OnMotionStart or OnMotionStartDelayed).
<code>period</code> (e.g. 1 = every cycle).	Period at which the data is sampled in terms of the motion chip cycles
<code>variables</code>	A list of variables to capture. Up to 4 variables can be captured.
<code>waitMotionStabilizeUs</code>	Time in us to wait for motion to stabilize. Only applies when the trigger is OnMotionStartDelayed.

Returns:

A result `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.21.2.5 EnableTriggerOnPosition()

```
public Task<InstrumentResult> EnableTriggerOnPosition(double startPosition,
double endPosition,
double increment,
bool isBidirectional,
int pulseWidthNs)
```

Enable trigger on position (TOP). TOP will generate a GPIO pulse every 'increment' of travel while in the window specified by 'startPosition' and 'endPosition'.

Details

When using this feature with a SmartStageXY or SmartStageLinear, the `GpioConfiguration` in `Instrument.cfg` must be set to `PMDGpioConfiguration::S2` (the default).

Parameters:

<code>startPosition</code>	Position at which to start generating pulses in selected units (mm, um, or nm).
<code>endPosition</code>	Position at which to stop generating pulses in selected units (mm, um, or nm).
<code>increment</code>	Generate a pulse every 'increment' of travel in selected units (mm, um, or nm).
<code>isBidirectional</code>	If true, pulses will also be generated when moving from <code>endPosition</code> back to <code>startPosition</code> . If false, pulses will only be generated when moving from <code>startPosition</code> to <code>endPosition</code> .
<code>pulseWidthNs</code>	The width of the pulse to generate at each position in nanoseconds. The actual pulse width is a value rounded to the nearest practicable value, which must be a multiple of 50ns and less or equal to 204,800 ns. It's ignored by version 1.2 and lower of the PMD motion control firmware.

Returns:

A result `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.21.2.6 EnterHomingMode()

```
public Task<InstrumentResult> EnterHomingMode()
```

Set settings necessary before homing routine starts. Any settings replaced can be restored with `ExitHomingMode()`.

Details

Homing mode will save the `MotionCompleteMode` in buffer memory and set it to `CommandedPositionReached`. Initialization should be complete before calling this method, as it resets the homing mode.

4.7.21.2.7 ExitHomingMode()

```
public Task<InstrumentResult> ExitHomingMode()
```

Restore settings changed by EnterHomingMode() and exit homing mode.

4.7.21.2.8 GetAcceleration()

```
public InstrumentResult<double> GetAcceleration()
```

Get the maximum acceleration of the axis.

Returns:

A result InstrumentResult on completion, containing the acceleration in selected units (mm/s², um/s², or nm/s²).

4.7.21.2.9 GetActiveControlMode()

```
public InstrumentResult<ActiveControlMode> GetActiveControlMode()
```

Get the axis' active control mode, which determines how the axis is being commanded and the type of move profile in use.

Returns:

A result InstrumentResult on completion, containing the axis' active control mode.

4.7.21.2.10 GetActualPosition()

```
public InstrumentResult<double> GetActualPosition()
```

Get the actual position of the axis as reported by the encoder.

Details

On successful completion of a move, the actual position will match the commanded position (within the axis resolution). If the move failed (e.g. a position error occurred), the actual position will not match the commanded position. The commanded position will be set to the actual position when:

- A move is interrupted due to a Stop or EStop command being issued.
- While the axis is disabled and when the Enable command is issued to transition the axis from the disabled to enabled state.

Returns:

A result InstrumentResult on completion, containing the encoder position in selected units (mm, um, or nm).

4.7.21.2.11 GetCommandedPosition()

```
public InstrumentResult<double> GetCommandedPosition()
```

Get the position the axis was commanded to move to.

Details

On successful completion of a move, the actual position will match the commanded position (within the axis resolution). If the move failed (e.g. a position error occurred), the actual position will not match the commanded position. The commanded position will be set to the actual position when:

- A move is interrupted due to a Stop or EStop command being issued.

- While the axis is disabled and when the Enable command is issued to transition the axis from the disabled to enabled state.

Returns:

A result InstrumentResult on completion, containing the target position in selected units (mm, um, or nm).

4.7.21.2.12 GetDeceleration()

```
public InstrumentResult<double> GetDeceleration()
```

Get the maximum deceleration of the axis in selected units (mm/s², um/s², or nm/s²).

Returns:

A result InstrumentResult on completion, containing the deceleration in selected units (mm/s², um/s², or nm/s²).

4.7.21.2.13 GetDigitalOutputTrigger()

```
public InstrumentResult<DigitalOutputTrigger> GetDigitalOutputTrigger()
```

Get the digital output trigger in effect.

Returns:

A result InstrumentResult on completion, containing the digital output trigger in effect (see enum DigitalOutputTrigger for details).

4.7.21.2.14 GetExternalControl()

```
public InstrumentResult<double> GetExternalControl()
```

Get the currently configured move distance per pulse.

Returns:

A result InstrumentResult on completion, containing moveDistancePerPulse in selected units (mm, um, or nm). Zero indicates external control is disabled, a non-zero value indicates external control is enabled.

4.7.21.2.15 GetFlagStatus()

```
public InstrumentResult<bool> GetFlagStatus(string flag_name)
```

Returns the activation state of the given named flag

Parameters:

flag_name Named flag to query on

Returns:

A Task which returns an InstrumentResult on completion, which contains the activation state of the flag

4.7.21.2.16 GetIsEnabled()

```
public InstrumentResult<bool> GetIsEnabled()
```

Get the enabled (energized) / disabled (de-energized) state the axis motor.

Returns:

A result InstrumentResult on completion, of the enabled state of the motor (true if enabled (energized), false if disabled (de-energized)).

4.7.21.2.17 GetIsTraceCaptureEnabled()

```
public InstrumentResult<bool> GetIsTraceCaptureEnabled()
```

Is trace capture enabled for the next axis move? Does not detect immediate trace capture.

Returns:

A result InstrumentResult on completion, true if trace capture is enabled, false if not.

4.7.21.2.18 GetIsTriggerOnPositionEnabled()

```
public InstrumentResult<bool> GetIsTriggerOnPositionEnabled()
```

Get the state of the trigger on position (TOP) function.

Returns:

A result InstrumentResult on completion, true if trigger on position is enabled, false if not.

4.7.21.2.19 GetJerk()

```
public InstrumentResult<double> GetJerk()
```

Get the maximum jerk of the axis in selected units (mm/s³, um/s³, or nm/s³).

Returns:

A result InstrumentResult on completion, containing the acceleration in selected units (mm/s³, um/s³, or nm/s³).

4.7.21.2.20 GetMotorCurrent()

```
public InstrumentResult<double> GetMotorCurrent()
```

Get the RMS current being used to drive the motor at this instant.

Details

Due to communication latency with the motor drive chip, the value returned will be lagging.

Returns:

A result InstrumentResult on completion, containing the motor current in Amps.

4.7.21.2.21 GetNamedDistances()

```
public InstrumentResult<IDictionary<string, double> > GetNamedDistances()
```

Returns a complete list of named distances for this axis.

Returns:

A result InstrumentResult on completion, containing a dictionary of named distances (distanceName, distance) for this axis. Returns an empty dictionary if this axis does not have any named distances.

4.7.21.2.22 GetNamedFlags()

```
public InstrumentResult<IDictionary<string, double> > GetNamedFlags()
```

Returns a complete list of named flags for this axis.

Returns:

A result InstrumentResult on completion, containing a dictionary of named flags (flagName, flag) for this axis. Returns an empty dictionary if this axis does not have any named flags.

4.7.21.2.23 GetNamedMoveProfiles()

```
public InstrumentResult<IDictionary<string, IDictionary<string, double> > > GetNamedMoveProfiles()
```

Returns a complete list of named move profiles for this axis.

Returns:

A result InstrumentResult on completion, containing a dictionary of named move profiles (moveProfileName, (parameterName, parameterValue)) for this axis. Returns an empty dictionary if this axis does not have any named move profiles.

4.7.21.2.24 GetNamedPositions()

```
public InstrumentResult<IDictionary<string, double> > GetNamedPositions()
```

Returns a complete list of named positions for this axis.

Returns:

A result InstrumentResult on completion, containing a dictionary of named positions (positionName, position) for this axis. Returns an empty dictionary if this axis does not have any named positions.

4.7.21.2.25 GetPositionError()

```
public InstrumentResult<double> GetPositionError()
```

Get the current position error of the axis.

Details

Position error is the difference between the commanded position and the actual position of the axis, in the selected units (mm, um, or nm). A non-zero value indicates that the axis did not reach the commanded position, which can occur due to overshoot, mechanical limits, or if a move was interrupted. This function is useful for monitoring motion accuracy and detecting issues during moves.

Returns:

A result InstrumentResult on completion, containing the position error in selected units (mm, um, or nm).

4.7.21.2.26 GetRecoveryOperation()

```
public InstrumentResult<AxisRecoveryOperation> GetRecoveryOperation()
```

Get the recovery operation (if any) required to return the axis to a state where it can successfully perform moves.

Returns:

A result InstrumentResult on completion, containing an enum indicating the type of recovery operation required.

4.7.21.2.27 GetVelocity()

```
public InstrumentResult<double> GetVelocity()
```

Get the maximum velocity of the axis.

Returns:

A result InstrumentResult on completion, containing the velocity in selected units (mm/s, um/s, or nm/s).

4.7.21.2.28 MoveAbsolute()

```
public Task<InstrumentResult> MoveAbsolute(double position)
```

Moves axis to the specified absolute position in selected units (mm, um, or nm).

Parameters:

position Value of the absolute coordinate to move to in selected units (mm, um, or nm).

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.29 MoveContinuous()

```
public InstrumentResult MoveContinuous(double position,
                                       double acceleration,
                                       double velocity,
                                       double deceleration)
```

Moves axis to the specified absolute position in selected units (mm, um, or nm). Unlike other move commands, this command can be issued while there is a move in progress. It returns as soon as the move has been started or the existing move update.

Details

Parameters:

position Value of the absolute coordinate to move to in selected units (mm, um, or nm).

acceleration Maximum acceleration to be set in selected units (mm/s², um/s², or nm/s²).

velocity Maximum velocity to be set in selected units (mm/s, um/s, or nm/s).

deceleration Maximum deceleration to be set in selected units (mm/s², um/s², or nm/s²).

Returns:

An InstrumentResult indicating if the move started successfully.

4.7.21.2.30 MoveRelative()

```
public Task<InstrumentResult> MoveRelative(double distance)
```

Moves axis from the current position by the specified value in selected units (mm, um, or nm).

Parameters:

distance Amount by which to move from the current position in selected units (mm, um, or nm).

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.31 MoveRelativeToFlag()

```
public Task<InstrumentResult> MoveRelativeToFlag(double maximumDistance,
                                                  string flag_name)
```

Move axis to the given named flag searching up to a maximum distance. If the flag is reached, the axis will be brought to an abrupt stop.

Parameters:

maximumDistance Maximum distance selected units (mm, um, or nm) to search for the flag
flag_name Named flag to search for

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.32 MoveRelativeToFlagOpenLoop()

```
public Task<InstrumentResult> MoveRelativeToFlagOpenLoop(string flagName,
                                                         double offsetFromFlag)
```

Move axis to the given named flag in open loop mode. This command can only be used prior to initialization. It can be used to move an axis to a known location prior to initialization and homing.

Parameters:

flagName Named flag to move to.
offsetFromFlag An offset (+ve or -ve) in selected units (mm, um, or nm) to move away from the flag once the flag has been reached.e.g.supports backing off the positive limit (use a -ve offset) or negative limit(use a +ve offset).

4.7.21.2.33 MoveRelativeUDP()

```
public Task<InstrumentResult> MoveRelativeUDP(uint repeats=1)
```

Move axis using the currently selected time-based UDP profile.

Details

Parameters:

repeats Number of times to repeat move.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.34 NEnableTraceCapture()

```
public Task<InstrumentResult> NEnableTraceCapture(TraceTrigger trigger,
                                                  string traceProfile)
```

Enable capture of trace data using a named trace profile defined in configuration.

Details

If the trigger is OnMotionStart, trace capture will be automatically started when the next move begins and stopped once the move completes. If the trigger is OnMotionStartNoEnd, trace capture will be automatically started when the next move begins and must be stopped by calling StopTraceCapture(). If the trigger is Immediate, trace capture will be started immediately and must be stopped by calling StopTraceCapture().

Parameters:

trigger Trigger to start the trace capture (e.g. Immediate, OnMotionStart or OnMotionStartNoEnd).

`traceProfile` The name of the trace profile section defined in configuration. This profile will specify the period and up to 4 trace variables.

Returns:

A result `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.21.2.35 NEnableTriggerOnPosition()

```
public Task<InstrumentResult> NEnableTriggerOnPosition(string startPositionName,
                                                    string endPositionName,
                                                    string namedDistance,
                                                    bool isBidirectional,
                                                    int pulseWidthNs)
```

Enable trigger on position (TOP). TOP will generate a GPIO pulse every 'increment' of travel while in the window specified by 'startPosition' and 'endPosition'.

Details

When using this feature with a SmartStageXY or SmartStageLinear, the `GpioConfiguration` in `Instrument.cfg` must be set to `PMDGpioConfiguration::S2` (the default).

Parameters:

<code>startPositionName</code>	Named position at which to start generating pulses.
<code>endPositionName</code>	Named position at which to stop generating pulses.
<code>namedDistance</code>	Generate a pulse every 'namedDistance' of travel.
<code>isBidirectional</code>	If true, pulses will also be generated when moving from <code>endPosition</code> back to <code>startPosition</code> . If false, pulses will only be generated when moving from <code>startPosition</code> to <code>endPosition</code> .
<code>pulseWidthNs</code>	The width of the pulse to generate at each position in nanoseconds. The actual pulse width is a value rounded to the nearest practicable value, which must be a multiple of 50ns and less or equal to 204,800 ns. It's ignored by version 1.2 and lower of the PMD motion control firmware.

Returns:

A result `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.21.2.36 NMoveAbsolute()

```
public Task<InstrumentResult> NMoveAbsolute(string position_name)
```

Moves axis to the specified absolute position by name.

Parameters:

<code>position_name</code>	Name of the destination.
----------------------------	--------------------------

Returns:

A Task which returns an `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.21.2.37 NMoveOffset()

```
public Task<InstrumentResult> NMoveOffset(string position_name,
                                          string distance_name)
```

Move axis to a location that is a fixed named distance offset from a named absolute position.

Parameters:

position_name Name of the absolute position.

distance_name Name of the relative distance to add on to the absolute position.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.38 NMoveRelative()

```
public Task<InstrumentResult> NMoveRelative(string position_name)
```

Moves axis from the current position to destination by name.

Parameters:

position_name Name of the destination.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.39 NMoveRelativeToFlag()

```
public Task<InstrumentResult> NMoveRelativeToFlag(string maximumDistance_name,
string flag_name)
```

Move axis to the given named flag searching up to a named maximum distance. If the flag is reached, the axis will be brought to an abrupt stop.

Parameters:

maximumDistance_name Name maximum distance to search for the flag

flag_name Named flag to search for

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.40 NMoveRelativeToFlagOpenLoop()

```
public Task<InstrumentResult> NMoveRelativeToFlagOpenLoop(string flagName,
string offsetFromFlagDistanceName)
```

Move axis to the given named flag in open loop mode.

Details

This command can only be used prior to initialization. It can be used to move an axis to a known location prior to initialization and homing. Move axis to the given named flag in open loop mode.

This command can only be used prior to initialization. It can be used to move an axis to a known location prior to initialization and homing.

Parameters:

[in] flagName Named flag to move to.

[in] offsetFromFlagDistanceName A named distance offset to move away from the flag once the flag has been reached. e.g. supports backing off the positive limit (use a -ve offset) or negative limit (use a +ve offset).

flagName Named flag to move to.

offsetFromFlagDistanceName A named distance offset to move away from the flag once the flag has been reached.e.g.supports backing off the positive limit (use a -ve offset) or negative limit(use a +ve offset).

4.7.21.2.41 NResetPosition()

```
public Task<InstrumentResult> NResetPosition(string position_name)
```

Resets the current (and encoder if present) position of the motor

Parameters:

position_name New named position of the motor

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.42 ResetPosition()

```
public Task<InstrumentResult> ResetPosition()
```

Resets the commanded position to match the current encoder position.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.43 ResetPosition()

```
public Task<InstrumentResult> ResetPosition(double position)
```

Resets the current (and encoder if present) position of the motor

Parameters:

position New position of the motor in selected units (mm, um, or nm)

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.44 SaveTraceCapture()

```
public Task<InstrumentResult> SaveTraceCapture(string pathname,
int traceCount=0)
```

Save the trace data captured during the last move (see EnableTraceCapture() and NEnableTraceCapture()) as a csv file.

Parameters:

pathname The path to the file to save the data to.

traceCount The maximum number of trace samples to save to the file. If set to 0 (default), all available trace samples will be saved.

4.7.21.2.45 SelectMoveProfile()

```
public InstrumentResult SelectMoveProfile(string profileName)
```

Sets the Acceleration, Deceleration, Velocity and Jerk specified by the named profile.

Parameters:

profileName The name of the profile to select.

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.46 SetAcceleration()

```
public InstrumentResult SetAcceleration(double acceleration)
```

Set the maximum acceleration of the axis.

Parameters:

acceleration Maximum acceleration to be set in selected units (mm/s², um/s², or nm/s²).

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.47 SetDeceleration()

```
public InstrumentResult SetDeceleration(double deceleration)
```

Set the maximum deceleration of the axis.

Parameters:

deceleration Maximum deceleration to be set in selected units (mm/s², um/s², or nm/s²).

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.48 SetDigitalOutputTrigger()

```
public Task<InstrumentResult> SetDigitalOutputTrigger(DigitalOutputTrigger trigger)
```

Enable / disable a digital output (AxisOut1) trigger based on the axis motion.

Details

When using this feature with a SmartStageXY or SmartStageLinear, the GpioConfiguration in Instrument.cfg must be set to PMDGpioConfiguration::S2 (the default).

Parameters:

trigger Specifies a trigger on which the digital output will be set high or disable triggering. (see enum DigitalOutputTrigger for details).

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.49 SetExternalControl()

```
public Task<InstrumentResult> SetExternalControl(double moveDistancePerPulse)
```

Enable / disable control of the axis via an external pulse signal.

Details

When using this feature with a SmartStageXY or SmartStageLinear, the GpioConfiguration in Instrument.cfg must be set to PMDGpioConfiguration::S1.

Parameters:

`moveDistancePerPulse` Move the axis by this amount in selected units (mm, um, or nm) per positive pulse signal and by the negative amount per negative pulse signal. Setting this value to zero will disable external control.

Returns:

A result `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.21.2.50 SetJerk()

```
public InstrumentResult SetJerk(double jerk)
```

Set the maximum jerk of the axis.

Parameters:

`jerk` Maximum jerk to be set in selected units (mm/s³, um/s³, or nm/s³).

Returns:

A result `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.21.2.51 SetNamedDistances()

```
public InstrumentResult SetNamedDistances(IDictionary< string,
double > namedDistances,
bool doWriteToLocalConfig=true)
```

Update the list of named distances for this axis. The list will be persisted to configuration.

Parameters:

`namedDistances` A dictionary of named distances (distanceName, distance) for this axis.

`doWriteToLocalConfig` Whether to write updated values to local config.

Returns:

A result `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.21.2.52 SetNamedMoveProfiles()

```
public InstrumentResult SetNamedMoveProfiles(IDictionary< string,
IDictionary< string,
double >> namedMoveProfiles,
bool doWriteToLocalConfig=true)
```

Update the list of named move profiles for this axis. The list will be persisted to configuration.

Parameters:

`namedMoveProfiles` A dictionary of named move profiles (moveProfileName, (parameterName, parameterValue)) for this axis.

`doWriteToLocalConfig` Whether to write updated values to local config.

Returns:

A result `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.21.2.53 SetNamedPositions()

```
public InstrumentResult SetNamedPositions(IDictionary< string,
                                         double > namedPositions,
                                         bool doWriteToLocalConfig=true)
```

Update the list of named positions for this axis. The list will be persisted to configuration.

Parameters:

namedPositions A dictionary of named positions (positionName, position) for this axis.
doWriteToLocalConfig Whether to write updated values to local config.

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.54 SetVelocity()

```
public InstrumentResult SetVelocity(double velocity)
```

Set the maximum velocity of the axis.

Parameters:

velocity Maximum velocity to be set in selected units (mm/s, um/s, or nm/s).

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.55 Stop()

```
public Task<InstrumentResult> Stop()
```

Stop the current movement action in a controlled manner. Will stop the current profile/velocity move action and return immediately. The original move command will complete once the axis has come to a halt. If there is no movement in progress, the operation will complete without error. Wait() must still called after issuing a Stop() to wait for the axis to come to a halt. Unlike EStop(), the Stop() command does not require the axis to be re-initialized.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.56 StopTraceCapture()

```
public Task<InstrumentResult> StopTraceCapture()
```

Stop the capture of trace data. This will stop any trace capture currently in progress.

Details

If the trigger supplied to EnableTraceCapture() was OnMotionStart or OnMotionStartNoEnd and a move has not started, this will cancel the request to capture trace data on the next move.

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.21.2.57 WaitForContinuousMoveToComplete()

```
public Task<InstrumentResult> WaitForContinuousMoveToComplete()
```

Waits for the current continuous move to complete.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.22 IConfiguration Interface

```
public interface IConfiguration
```

Low-level interface providing methods to get and set configuration parameters.

4.7.22.1 Public Methods

4.7.22.1.1 GetBoolSetting()

```
public InstrumentResult<bool> GetBoolSetting(string path)
```

Read a bool setting from instrument configuration. Must be called after the instrument configuration has been loaded during Configure().

Parameters:

path The full pathname to the setting (e.g. Section.Key)

Returns:

An InstrumentResult which includes the setting on success or an error if the setting cannot be found / read, etc.

4.7.22.1.2 GetDoubleSetting()

```
public InstrumentResult<double> GetDoubleSetting(string path)
```

Read a double setting from instrument configuration. Must be called after the instrument configuration has been loaded during Configure().

Parameters:

path The full pathname to the setting (e.g. Section.Key)

Returns:

An InstrumentResult which includes the setting on success or an error if the setting cannot be found / read, etc.

4.7.22.1.3 GetIntegerSetting()

```
public InstrumentResult<int> GetIntegerSetting(string path)
```

Read an integer setting from instrument configuration. Must be called after the instrument configuration has been loaded during Configure().

Parameters:

path The full pathname to the setting (e.g. Section.Key)

Returns:

An InstrumentResult which includes the setting on success or an error if the setting cannot be found / read, etc.

4.7.22.1.4 GetSectionNames()

```
public InstrumentResult<IReadOnlyList<string> > GetSectionNames()
```

Returns a list of section names in the configuration.

Returns:

An InstrumentResult which on success contains a list of section names.

4.7.22.1.5 GetSectionSettings()

```
public InstrumentResult<IDictionary<string, string> > GetSectionSettings(string path)
```

Read an entire section of settings from instrument configuration. Must be called after the instrument configuration has been loaded during Configure().

Parameters:

path The full pathname to the section

Returns:

An InstrumentResult which includes the settings on success or an error if the section cannot be found / read, etc.

4.7.22.1.6 GetStringSetting()

```
public InstrumentResult<string> GetStringSetting(string path)
```

Read a string setting from instrument configuration. Must be called after the instrument configuration has been loaded during Configure().

Parameters:

path The full pathname to the setting (e.g. Section.Key)

Returns:

An InstrumentResult which includes the setting on success or an error if the setting cannot be found / read, etc.

4.7.22.1.7 IsSectionPresent()

```
public InstrumentResult<bool> IsSectionPresent(string section)
```

Returns whether the specified section exists in the instrument configuration. Must be called after the instrument configuration has been loaded during Configure().

Parameters:

section Section name

Returns:

An InstrumentResult which on success contains a bool flag indicating whether the section exists.

4.7.22.1.8 IsSettingPresent()

```
public InstrumentResult<bool> IsSettingPresent(string path)
```

Returns whether the specified setting exists in the instrument configuration. Must be called after the instrument configuration has been loaded during Configure().

Parameters:

path The full pathname to the setting (e.g. Section.Key)

Returns:

An InstrumentResult which on success contains a bool flag indicating whether the setting exists.

4.7.22.1.9 RemoveLocalSetting()

```
public InstrumentResult RemoveLocalSetting(string path,
                                           bool doWriteToLocalConfig=true)
```

Remove a setting from the local instrument configuration.

Parameters:

path	The full pathname to the setting (e.g. Section.Key)
doWriteToLocalConfig	Whether to persist to local config immediately.

Returns:

An InstrumentResult which can be used to determine the success or failure of the command.

4.7.22.1.10 SetBoolSetting()

```
public InstrumentResult SetBoolSetting(string path,
                                       bool value,
                                       bool doWriteToLocalConfig=true)
```

Write a bool setting to the local instrument configuration. Must be called after the instrument configuration has been loaded during Configure().

Parameters:

path	The full pathname to the setting (e.g. Section.Key)
value	The new value of the setting.
doWriteToLocalConfig	Whether to persist to local config immediately.

Returns:

An InstrumentResult which can be used to determine the success or failure of the command.

4.7.22.1.11 SetDoubleSetting()

```
public InstrumentResult SetDoubleSetting(string path,
                                         double value,
                                         bool doWriteToLocalConfig=true)
```

Write a double setting to the local instrument configuration. Must be called after the instrument configuration has been loaded during Configure().

Parameters:

path	The full pathname to the setting (e.g. Section.Key)
value	The new value of the setting.
doWriteToLocalConfig	Whether to persist to local config immediately.

Returns:

An InstrumentResult which can be used to determine the success or failure of the command.

4.7.22.1.12 SetIntegerSetting()

```
public InstrumentResult SetIntegerSetting(string path,
                                         int value,
                                         bool doWriteToLocalConfig=true)
```


Write an integer setting to the local instrument configuration. Must be called after the instrument configuration has been loaded during `Configure()`.

Parameters:

<code>path</code>	The full pathname to the setting (e.g. <code>Section.Key</code>)
<code>value</code>	The new value of the setting.
<code>doWriteToLocalConfig</code>	Whether to persist to local config immediately.

Returns:

An `InstrumentResult` which can be used to determine the success or failure of the command.

4.7.22.1.13 SetStringSetting()

```
public InstrumentResult SetStringSetting(string path,
                                       string value,
                                       bool doWriteToLocalConfig=true)
```

Write a string setting to the local instrument configuration. Must be called after the instrument configuration has been loaded during `Configure()`.

Parameters:

<code>path</code>	The full pathname to the setting (e.g. <code>Section.Key</code>)
<code>value</code>	The new value of the setting.
<code>doWriteToLocalConfig</code>	Whether to persist to local config immediately.

Returns:

An `InstrumentResult` which can be used to determine the success or failure of the command.

4.7.23 IConnectionState Interface

```
public interface IConnectionState
```

Low-level interface providing methods to get and set configuration parameters.

4.7.23.1 Events

4.7.23.1.1 ConnectionStateChanged

```
public event ConnectionStateChangedEventHandler ConnectionStateChanged
```

Event raised whenever the connection state changes (i.e. the connection between MotionSynergyAPI (running on the SBC) and the Subsystem Controller firmware (running on the Instrument hardware)).

4.7.23.2 Properties

4.7.23.2.1 ConnectionState

```
public ConnectionState ConnectionState { get; }
```

Indicates the current state of the connection between MotionSynergyAPI (running on the SBC) and the Subsystem Controller firmware (running on the Instrument hardware).

Details

A ConnectionStateChanged event will be raised whenever this state changes.

4.7.24 IDevice Interface

```
public interface IDevice
```

Interface representing a generic device. It contains properties, events, and commands common to all device types.

4.7.24.1 Properties

4.7.24.1.1 DefaultTimeoutMs

```
public uint DefaultTimeoutMs { get; }
```

The default timeout in milliseconds for commands executed on this device (e.g. 5000ms). This can be defined in configuration on a per device basis.

4.7.24.1.2 HaveAttemptedConnect

```
public bool HaveAttemptedConnect { get; }
```

True if an attempt was made to connect to the hardware, false if not.

4.7.24.1.3 IsConnected

```
public bool IsConnected { get; }
```

True if the device is successfully connected to hardware, false if not.

4.7.24.1.4 Name

```
public string Name { get; }
```

The name of the device (e.g. 'Z Axis').

4.7.24.2 Public Methods

4.7.24.2.1 Connect()

```
public Task<InstrumentResult> Connect()
```

Establish a connection to the hardware.

Details

Most commands require initialization before it can be properly used. However, a few commands only require an established connection. Calling this is not required if the hardware is already initialized.

Returns:

An InstrumentResult which can be used to determine the success or failure of the command.

4.7.24.2.2 EStop()

```
public Task<InstrumentResult> EStop()
```

Perform an emergency stop on the device. This will immediately halt any motion (e.g. an abrupt stop will be performed for an axis device) and return the device to a safe, passive state (e.g. for an

axis device this means the operating mode is set to disable the axis, motor output, current control, position loop, and trajectory).

Details

For an axis device, this will perform an abrupt stop. `Initialize()` must be called on this device before it will accept further commands.

Returns:

A Task which returns an `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.24.2.3 Initialize()

```
public Task<InstrumentResult> Initialize()
```

Initialize the device. A device must be initialized before most other commands can be performed, except for a function which establishes connection.

Details

In the case of an axis this involves:

- Establishing a connection to the hardware.
- Performing Analog Calibration.
- Performing Commutation.

Returns:

A Task which returns an `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.24.2.4 Shutdown()

```
public Task<InstrumentResult> Shutdown()
```

Shutdown the device in preparation for power off. Once a device has been shutdown, it will accept no further commands. If there is a connection associated with this device and this is the last device using the connection, the connection will be closed.

Returns:

A Task which returns an `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.25 IDeviceEmulator Interface

```
public interface IDeviceEmulator
```

Interface representing a generic device emulator. This interface is used to control the behaviour of the emulator and inject various states for testing purposes. It contains properties, events, and commands common to all device emulators.

Details

TBD - The specific commands, properties and events will be detailed in a later stage.

4.7.26 IDiagnostics Interface

```
public interface IDiagnostics
```

Low-level interface providing methods to produce diagnostic information.

4.7.26.1 Properties

4.7.26.1.1 CurrentLogLevel

```
public LogLevel CurrentLogLevel { get; }
```

Returns the current log level.

4.7.26.1.2 LogFoldername

```
public string LogFoldername { get; }
```

Path to the folder used to store log files generated by MotionSynergyAPI.

4.7.26.2 Public Methods

4.7.26.2.1 LogString()

```
public InstrumentResult LogString(LogLevel level,
                                  string logline)
```

Logs the provided string to the diagnostic log at debug level.

Parameters:

level The log level (Debug, Info, Warning, Error, Fatal)

logline String to log

4.7.26.2.2 SetLogLevel()

```
public InstrumentResult SetLogLevel(LogLevel level,
                                     bool persist)
```

Sets the logging level for the application.

Details

Use this method to adjust the verbosity of log messages at runtime. For example, setting the level to LogLevel.Error will ensure that only error messages are logged.

Parameters:

level The desired logging level. This determines the minimum severity of log messages that will be recorded.

persist Persist the log level to the configuration file if true.

Returns:

An InstrumentResult indicating the success or failure of the operation.

4.7.26.2.3 SubscribeForLogEntries()

```
public InstrumentResult<uint> SubscribeForLogEntries(LogLevel minimumLogLevel,
                                                       LogEntriesPublishedCallback callback)
```

Subscribe for notification whenever a new log entry is created with a log level higher or equal to the minimum log level.

Parameters:

`minimumLogLevel` The minimum log level, e.g. setting this to Info will show all log entries except for Debug log entries.

`callback` Callback to invoke each time one or more new log entries matching the supplied filter are available.

Returns:

A result `InstrumentResult` on completion, containing a unique `subscriberId` which can be used to Unsubscribe (see `UnsubscribeForLogEntries(uint)`).

4.7.26.2.4 UnsubscribeForLogEntries()

```
public InstrumentResult UnsubscribeForLogEntries(uint logEntrySubscriberId)
```

Unsubscribe a subscriber from log entry publications.

Details

All subscribers must be unsubscribed before shutting down `MotionSynergyAPI`.

Parameters:

`logEntrySubscriberId` The unique `subscriberId` returned when `SubscribeForLogEntries(LogLevel, LogEntriesPublishedCallback)` was first called.

Returns:

A result `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.27 IDigitalInput Interface

```
public interface IDigitalInput : IDevice
```

Interface to control a `DigitalInput` device, a device capable of monitoring a digital input signal (e.g. an opto or microswitch) and providing a binary (i.e. true / false) representation of the signal.

4.7.27.1 Events

4.7.27.1.1 DigitalInputChanged

```
public event DigitalInputChangedEventHandler DigitalInputChanged
```

Event raised whenever the digital input state changes and asynchronous change notifications are enabled.

4.7.27.2 Public Methods

4.7.27.2.1 DisableAsynchChangeUpdates()

```
public InstrumentResult DisableAsynchChangeUpdates()
```

Disable asynchronous change notifications for this digital input.

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.27.2.2 EnableAsynchChangeUpdates()

```
public InstrumentResult EnableAsynchChangeUpdates()
```

Enable asynchronous change notifications for this digital input.

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.27.2.3 GetBit()

```
public InstrumentResult<bool> GetBit(byte bit)
```

Gets the current state of a bit of the digital input

Returns:

The current state of the bit of the digital input

4.7.27.2.4 GetRegister()

```
public InstrumentResult<byte> GetRegister()
```

Gets the current state of all input bits as a uint8_t

Returns:

The current state of the digital input

4.7.27.2.5 IsChangeNotificationEnabled()

```
public InstrumentResult<bool> IsChangeNotificationEnabled()
```

Check if asynchronous change notifications is currently enabled for this digital input.

Returns:

Are asynchronous change notifications currently enabled for this digital input.

4.7.28 IMotionControllerConfiguration Interface

```
public interface IMotionControllerConfiguration
```

An interface for reading and writing various motion controller settings from/to memory/NVRAM.

4.7.28.1 Public Methods

4.7.28.1.1 GetBiquadFilters()

```
public InstrumentResult<BiquadFilterSettings> GetBiquadFilters()
```

Read the biquad filter settings from the motion controller's memory.

Returns:

A result InstrumentResult on completion, containing the settings.

4.7.28.1.2 GetCommunication()

```
public InstrumentResult<CommunicationSettings> GetCommunication()
```

Read the current communication settings. These will correspond to the settings in the motion controller's memory and NVRAM, unless there has been a call to SetCommunication. In this case, it will return the updated settings supplied to SetCommunication, which are buffered, pending a call to WriteToNVRAM.

Returns:

A result InstrumentResult on completion, containing the settings.

4.7.28.1.3 GetCommutationParameter1()

```
public InstrumentResult<int> GetCommutationParameter1()
```

Get the current value associated with commutation parameter 1 from the PMD controller.

Details

Note that this value is equal to the PhaseAngle * Prescalar factor.

Returns:

A result InstrumentResult on completion, containing the value of commutation parameter 1.

4.7.28.1.4 GetCurrentLoop()

```
public InstrumentResult<CurrentLoopSettings> GetCurrentLoop()
```

Read the current loop settings from the motion controller's memory.

Returns:

A result InstrumentResult on completion, containing the settings.

4.7.28.1.5 GetElectricalProperties()

```
public InstrumentResult<ElectricalProperties> GetElectricalProperties()
```

Get various electrical properties from the PMD controller.

4.7.28.1.6 GetFirmwareVersion()

```
public InstrumentResult<FirmwareVersionInfo> GetFirmwareVersion()
```

Get firmware version from the PMD controller.

Returns:

A result InstrumentResult on completion, containing the version info.

4.7.28.1.7 GetMotionError()

```
public InstrumentResult<MotionErrorSettings> GetMotionError()
```

Read the current motion error settings from the motion controller's memory.

Returns:

A result InstrumentResult on completion, containing the settings.

4.7.28.1.8 GetMotionTracking()

```
public InstrumentResult<MotionTrackingSettings> GetMotionTracking()
```

Read the current motion tracking settings from the motion controller's memory.

Returns:

A result `InstrumentResult` on completion, containing the settings.

4.7.28.1.9 `GetPhaseAngle()`

```
public InstrumentResult<PhaseAngleInfo> GetPhaseAngle()
```

Get the current phase angle from the PMD controller.

Returns:

A result `InstrumentResult` on completion, containing the phase angle information.

4.7.28.1.10 `GetPositionLoop()`

```
public InstrumentResult<PositionLoopSettings> GetPositionLoop()
```

Read the position loop settings from the motion controller's memory.

Returns:

A result `InstrumentResult` on completion, containing the settings.

4.7.28.1.11 `Restore()`

```
public Task<InstrumentResult> Restore(string pathname)
```

Restore from a PMD configuration script to the Magellan IC's working memory.

Details

Note that the axis needs to be de-energized (disabled) before calling this function.

Parameters:

`pathname` The path to the PMD configuration script file to read.

Returns:

A result `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.28.1.12 `RestoreToNVRAM()`

```
public Task<InstrumentResult> RestoreToNVRAM(string pathname)
```

Restore from a PMD configuration script to the Magellan IC's NVRAM configuration.

Details

Note that the axis needs to be de-energized (disabled) before calling this function.

Parameters:

`pathname` The path to the PMD configuration script file to read.

Returns:

A result `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.28.1.13 `Save()`

```
public Task<InstrumentResult> Save(string pathname)
```

Save the Magellan IC's NVRAM configuration as a PMD configuration script.

Details

Note that the axis needs to be de-energized (disabled) before calling this function.

Parameters:

pathname The path to the PMD configuration script to write.

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.28.1.14 SetBiquadFilters()

```
public Task<InstrumentResult> SetBiquadFilters(BiquadFilterSettings biquadFilterSettings)
```

Set the biquad filter settings in the motion controller's memory.

Parameters:

biquadFilterSettings The new biquad filter settings to use.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.28.1.15 SetCommunication()

```
public Task<InstrumentResult> SetCommunication(CommunicationSettings communicationSettings)
```

Set the communication settings to be written to NVRAM (note the motion controller's memory is NOT updated).

Parameters:

communicationSettings The new communication settings to apply when WriteToNVRAM is called.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.28.1.16 SetCommutationParameter1()

```
public Task<InstrumentResult> SetCommutationParameter1(int commutationParameter1)
```

Set the commutation parameter 1 value to be written to the PMD controller.

Details

Note that this value should be equal to the PhaseAngle * Prescalar factor.

Parameters:

commutationParameter1 The value to set.

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.28.1.17 SetCurrentLoop()

```
public Task<InstrumentResult> SetCurrentLoop(CurrentLoopSettings currentLoopSettings)
```

Set the current loop settings in the motion controller's memory.

Parameters:

`currentLoopSettings` The new current loop settings to use.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.28.1.18 SetMotionError()

```
public Task<InstrumentResult> SetMotionError(MotionErrorSettings motionErrorSettings)
```

Set the motion error settings in the motion controller's memory.

Parameters:

`motionErrorSettings` The new motion error settings to use.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.28.1.19 SetMotionTracking()

```
public Task<InstrumentResult> SetMotionTracking(MotionTrackingSettings motionTrackingSettings)
```

Set the motion tracking settings in the motion controller's memory.

Parameters:

`motionTrackingSettings` The new motion tracking settings to use.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.28.1.20 SetPhaseAngle()

```
public Task<InstrumentResult> SetPhaseAngle(UInt16 rawPhaseAngle)
```

Set the phase angle to be written to the PMD controller.

Parameters:

`rawPhaseAngle` The raw phase angle to set.

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.28.1.21 SetPositionLoop()

```
public Task<InstrumentResult> SetPositionLoop(PositionLoopSettings positionLoopSettings)
```

Set the position loop settings in the motion controller's memory.

Parameters:

`positionLoopSettings` The new position loop settings to use.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.28.1.22 WriteToNVRAM()

```
public Task<InstrumentResult> WriteToNVRAM(IReadOnlyList< MotionControllerConfigurationType >
configurationType)
```

Transfer the specified types of configuration settings from the motion controller's memory to NVRAM so they will persist across power cycles.

Details

Note that the axis needs to be de-energized (disabled) before calling this function. If the active communication settings are changed, the communications link needs to be re-established (i.e. disconnect followed by a connect or initialize).

Parameters:

configurationType The types of motion controller configuration settings to write.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.29 IMotionSynergyAPI Interface

```
public interface IMotionSynergyAPI
```

The primary interface provided by MotionSynergyAPI. This interface supports use cases such as startup (configure, connect, initialize) and shutdown. It also provides access to additional interfaces supporting more specialized functions such as Configuration, Diagnostics, Scripting, Alerts and access to Axis devices.

4.7.29.1 Events

4.7.29.1.1 InstrumentStateChanged

```
public event InstrumentStateChangedEventHandler InstrumentStateChanged
```

Event raised whenever the high-level state of MotionSynergyAPI changes.

4.7.29.2 Properties

4.7.29.2.1 Alerts

```
public IAlerts Alerts { get; }
```

Provides access to the Alerts interface.

4.7.29.2.2 AxisList

```
public IReadOnlyList<IAxis> AxisList { get; }
```

Returns a list of all the axis which make up the hardware. This allows individual axis to be monitored and controlled.

4.7.29.2.3 Configuration

```
public IConfiguration Configuration { get; }
```

Provides access to the Configuration interface.

4.7.29.2.4 Connection

```
public IConnectionState Connection { get; }
```

Provides access to the ConnectionStaTe interface allowing subscription to connection state change events.

4.7.29.2.5 Devices

```
public IReadOnlyList<IDevice> Devices { get; }
```

Returns a list of all the devices which make up the instrument hardware. This allows individual devices to be monitored and controlled.

Details

Warning: Device instances retrieved through this property will be invalidated when Initialize() is called. After Initialize() is called, this property should be used to retrieve new device instances.

4.7.29.2.6 Diagnostics

```
public IDiagnostics Diagnostics { get; }
```

Provides access to the Diagnostics interface.

4.7.29.2.7 DigitalInputList

```
public IReadOnlyList<IDigitalInput> DigitalInputList { get; }
```

Returns a list of all the digital inputs which make up the hardware. This allows individual general purpose inputs to be monitored.

4.7.29.2.8 InstrumentState

```
public InstrumentState InstrumentState { get; }
```

Indicates the current high-level state of MotionSynergyAPI.

Details

An InstrumentStateChanged event will be raised whenever this state changes.

4.7.29.2.9 Scripting

```
public IScripting Scripting { get; }
```

Provides access to scripting interface.

4.7.29.3 Public Methods

4.7.29.3.1 Configure()

```
public Task<InstrumentResult> Configure(InstrumentSettings instrumentSettings)
```

Configure the MotionSynergyAPI infrastructure ready for use.

Parameters:

instrumentSettings Settings which control the instrument behaviour.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.29.3.2 Connect()

```
public Task<InstrumentResult> Connect()
```

Establish a connection to the instrument hardware. Connection to all devices of the hardware will be established.

Details

Most commands require initialization before it can be properly used. However, a few commands only require an established connection. Calling this is not required if the hardware is already initialized.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.29.3.3 ConnectDevice()

```
public Task<InstrumentResult> ConnectDevice(string deviceName)
```

Establish a connection to the instrument hardware. Only the connection to the specified device will be established. Any previous connections will be cleared.

Details

This command is like Connect, but only for a single device.

Parameters:

deviceName The name of the device to connect.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.29.3.4 Disconnect()

```
public InstrumentResult Disconnect()
```

Close the connection to the instrument hardware (i.e. Process Controller, Optics Processor and barcode scanner).

Details

All subscribers must be unsubscribed before calling Disconnect.

Returns:

An InstrumentResult which can be used to determine the success or failure of the command.

4.7.29.3.5 GetFirstProduct()

```
public FirstProduct GetFirstProduct()
```

Determine the first product listed in the configuration file and its associate axis/axes.

Returns:

The product type and a list of associated axes on success, 'Undefined' and an empty list on failure.

4.7.29.3.6 Initialize()

```
public Task<InstrumentResult> Initialize(bool runInitializeScript,  
                                         IProgress< ProgressUpdate > progress)
```

Initialize the instrument hardware in preparation for running a test. This includes establishing a connection, initializing all hardware devices, and running the power on self-test workflow.

Details

This command will automatically re-establish a connection to the instrument hardware (i.e. perform a disconnect, followed by a connect).

Warning: Individual device instances retrieved through the Devices property must be discarded. Initialize will recreate all devices, rendering the old instances stale.

Parameters:

`runInitializeScript` Run the script specified in configuration by `Instrument.InitializeScript` during initialization

`progress` An instance to receive progress notifications during execution. Initialization is a long running command and progress notifications can be used to report progress to the user. If progress notifications are not required, this parameter may be set to null.

Returns:

A Task which returns an `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.29.3.7 Initialize()

```
public Task<InstrumentResult> Initialize()
```

Initialize the instrument hardware in preparation for running a test. This includes establishing a connection, initializing all hardware devices, and running the power on self-test workflow.

Details

This command calls `Initialize(bool, System.IProgress<MotionSynergyAPI.ProgressUpdate>)` with `runInitializeScript=true` and `progress=null`.

Warning: Individual device instances retrieved through the Devices property must be discarded. Initialize will recreate all devices, rendering the old instances stale.

Returns:

A Task which returns an `InstrumentResult` on completion, which can be used to determine the success or failure of the command.

4.7.29.3.8 ServiceInitialize()

```
public Task<InstrumentResult> ServiceInitialize(bool runInitializeScript,
                                              IProgress< ProgressUpdate > progress)
```

Initialize the instrument hardware. This only differs from the `MotionSynergyAPI Initialize` command in that it doesn't automatically perform a disconnect / connect prior to initialization.

Parameters:

`runInitializeScript` Run the script specified in configuration by `Instrument.InitializeScript` during initialization

`progress` An instance to receive progress notifications during execution. Initialization is a long running command and progress notifications can be used to report progress to the user. If progress notifications are not required, this parameter may be set to null. The following progress update steps are provided for `Initialize`.

- `ServiceInitialize`: indicates instrument hardware initialization.
- `InitializeComplete`: indicates instrument initialization has completed.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.29.3.9 Shutdown()

```
public InstrumentResult Shutdown()
```

Prepares MotionSynergyAPI for Power Off. Disconnects from the instrument hardware and performs any necessary software clean up and persistence of state.

Returns:

An InstrumentResult which can be used to determine the success or failure of the command.

4.7.30 IScripting Interface

```
public interface IScripting
```

Interface which supports executing a Lua script.

4.7.30.1 Public Methods

4.7.30.1.1 ExecuteScript()

```
public Task<InstrumentResult<IDictionary<string, string> > > ExecuteScript(string scriptFilename,
                                                                    IReadOnlyList< ScriptParameter >
                                                                    parameters,
                                                                    IProgress< ProgressUpdate >
                                                                    progress,
                                                                    CancellationToken?
                                                                    cancellationToken)
```

Execute a Lua script file.

Parameters:

scriptFilename	Path to the Lua script file.
parameters	A list of parameters to pass to the script. If parameters are not required, a null can be passed in.
progress	An instance to receive progress notifications during execution. Executing a script may be a long running command and progress notifications can be used to report progress to the user. If progress notifications are not required, this parameter may be set to null.
cancellationToken	Provides the caller with a means of cancelling the command. This parameter may be set to null if the means to cancel is not required.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command. The result also includes a dictionary containing any output parameters returned by the script in the form of string key/value pairs.

4.7.31 IUserDefinedProfile Interface

```
public interface IUserDefinedProfile
```

An interface for performing User Defined Profile (UDP) moves and updating the associated user defined profile data.

4.7.31.1 Public Methods

4.7.31.1.1 GetCurrentDataStreamIndex()

```
public InstrumentResult<uint> GetCurrentDataStreamIndex()
```

Get the current data stream index being used by the motion controller to calculate the target position of the axis during a UDP move.

Returns:

A result InstrumentResult on completion, containing the current data stream index.

4.7.31.1.2 MoveRelativeUDP()

```
public Task<InstrumentResult> MoveRelativeUDP(uint repeats=1)
```

Move axis using the currently selected time-based UDP profile.

Details

Parameters:

repeats Number of times to repeat move.

Returns:

A Task which returns an InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.31.1.3 SelectSineWaveMoveProfile()

```
public Task<InstrumentResult> SelectSineWaveMoveProfile(double frequencyHz,
                                                         double amplitude,
                                                         double angularAcceleration)
```

Select the sine wave UDP profile and setup to be ready for a relative UDP move.

Details

Parameters:

frequencyHz The frequency of the sine wave in Hz.

amplitude The amplitude of the sine wave in selected units (mm, um, or nm).

angularAcceleration The angular acceleration in Hz per sec.

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.31.1.4 UpdateProfileData()

```
public Task<InstrumentResult> UpdateProfileData(uint index,
                                                  double position)
```

Update a single position at the specified index in the currently selected user defined profile.

Details

Parameters:

index Zero-based index into the profile data array.

position The new position in selected units (mm, um, or nm) to write to this index in the profile data array.

Returns:

A result InstrumentResult on completion, which can be used to determine the success or failure of the command.

4.7.32 BiquadFilterDefinition Struct

```
public struct BiquadFilterDefinition
```

Defines a single Biquad Filter

4.7.32.1 Properties

4.7.32.1.1 A1

```
public int A1 { get; set; }
```

A1 Coefficient

4.7.32.1.2 A2

```
public int A2 { get; set; }
```

A2 Coefficient

4.7.32.1.3 AlphaOrBeta

```
public double AlphaOrBeta { get; set; }
```

The alpha or beta parameter used to generate the filter coefficients. Only used in Lead or Lag filters. Must be less than 1 in the Lead Alpha case, and greater than 1 in the Lag Beta case.

4.7.32.1.4 B0

```
public int B0 { get; set; }
```

B0 Coefficient

4.7.32.1.5 B1

```
public int B1 { get; set; }
```

B1 Coefficient

4.7.32.1.6 B2

```
public int B2 { get; set; }
```

B2 Coefficient

4.7.32.1.7 FilterType

```
public BiquadFilterType FilterType { get; set; }
```

The type of filter.

4.7.32.1.8 Frequency

```
public double Frequency { get; set; }
```

The frequency parameter used to generate the filter coefficients. Must be greater than zero.

4.7.32.1.9 IsActive

```
public bool IsActive { get; }
```

Returns true if a valid filter is in use, false if not.

4.7.32.1.10 IsAlphaOrBetaValid

```
public bool IsAlphaOrBetaValid { get; }
```

Gets a value indicating whether the AlphaOrBeta> property is valid for a lead / lag filter.

4.7.32.1.11 IsValid

```
public bool IsValid { get; }
```

Returns true if the filter type, frequency and qualify factor are valid, false if not.

4.7.32.1.12 K

```
public int K { get; set; }
```

K Coefficient

4.7.32.1.13 QualityFactor

```
public double QualityFactor { get; set; }
```

The quality factor parameter used to generate the filter coefficients. Must be greater than zero.

4.7.32.2 Public Methods

4.7.32.2.1 Constructor: BiquadFilterDefinition()

```
public BiquadFilterDefinition(BiquadFilterType filterType,
    double frequency,
    double qualityFactor,
    double alphaOrBeta,
    bool isValid,
    bool isActive,
    bool isAlphaOrBetaValid,
    bool isLeadLag,
    int b0,
    int b1,
    int b2,
    int a1,
    int a2,
    int k)
```

Constructor used during native deserialization.

Details

" Not used Not used Not used Not used B0 Coefficient B1 Coefficient B2 Coefficient A1 Coefficient A2 Coefficient K Coefficient

Parameters:

filterType	The type of filter.
frequency	The frequency parameter used to generate the filter coefficients. Must be greater than zero.
qualityFactor	The quality factor parameter used to generate the filter coefficients. Must be greater than zero.
alphaOrBeta	The alpha or beta parameter used to generate the filter coefficients. Only used in Lead or Lag filters.

4.7.32.3 Public Variables

4.7.32.3.1 IsLeadLag

```
bool IsLeadLag=> FilterType ==BiquadFilterType.Lead ||FilterType ==BiquadFilterType.Lag
```

Returns true if the filter is a Lead or Lag filter, false if not.

4.7.33 BiquadFilterSettings Struct

```
public struct BiquadFilterSettings
```

Defines biquad filter parameters for an axis.

4.7.33.1 Properties

4.7.33.1.1 Filter1

```
public BiquadFilterDefinition Filter1 { get; set; }
```

Defines the first Biquad Filter (may be none if Filter 1 is not used)

4.7.33.1.2 Filter2

```
public BiquadFilterDefinition Filter2 { get; set; }
```

Defines the second Biquad Filter (may be none if Filter 2 is not used)

4.7.33.2 Public Methods

4.7.33.2.1 Constructor: BiquadFilterSettings()

```
public BiquadFilterSettings(BiquadFilterDefinition filter1,
    BiquadFilterDefinition filter2)
```

Constructor

Parameters:

filter1 Defines the first Biquad Filter (may be none if Filter 1 is not used)

filter2 Defines the second Biquad Filter (may be none if Filter 2 is not used)

4.7.34 CurrentLoopSettings Struct

```
public struct CurrentLoopSettings
```

Defines current loop parameters for an axis.

4.7.34.1 Properties

4.7.34.1.1 IlimitCurrent

```
public UInt16 IlimitCurrent { get; set; }
```

Integrator Limit.

4.7.34.1.2 IsCurrentLoopTuningAvailable

```
public bool IsCurrentLoopTuningAvailable { get; set; }
```

Whether current loop tuning is available for the current motor type. As a result from the get function, the parameters are valid only if this is true. Can be ignored when setting/saving parameters.

4.7.34.1.3 KiCurrent

```
public UInt16 KiCurrent { get; set; }
```

Integrator Gain.

4.7.34.1.4 KpCurrent

```
public UInt16 KpCurrent { get; set; }
```

Proportional Gain.

4.7.34.2 Public Methods

4.7.34.2.1 Constructor: CurrentLoopSettings()

```
public CurrentLoopSettings(UInt16 kpCurrent,
    UInt16 kiCurrent,
    UInt16 ilimitCurrent,
    bool isCurrentLoopTuningAvailable)
```

Constructor

Parameters:

kpCurrent	Proportional Gain.
kiCurrent	Integrator Gain.
ilimitCurrent	Integrator Limit.
isCurrentLoopTuningAvailable	Whether current loop tuning is available for the current motor type.

4.7.34.2.2 AreParamsEqual()

```
public bool AreParamsEqual(CurrentLoopSettings settingsToCompare)
```

Check if parameters of this current loop settings are equal to another.

Parameters:

settingsToCompare

4.7.35 ElectricalProperties Struct

```
public struct ElectricalProperties
```

Electrical properties retrieved from the PMD controller.

4.7.35.1 Properties

4.7.35.1.1 BusVoltageV

```
public double BusVoltageV { get; set; }
```

Voltage provided by the power supply being used.

4.7.35.1.2 EncoderResolutionNm

```
public double EncoderResolutionNm { get; set; }
```

The encoder resolution for the axis.

4.7.35.1.3 SampleTimeUs

```
public double SampleTimeUs { get; set; }
```

Sampling clock frequency based on the Servo Rate (20kHz = 51.2, 4kHz = 256).

4.7.35.2 Public Methods

4.7.35.2.1 Constructor: ElectricalProperties()

```
public ElectricalProperties(double busVoltageV,
                           double encoderResolutionNm,
                           double sampleTimeUs)
```

Constructor

Parameters:

busVoltageV	Voltage provided by the power supply being used.
encoderResolutionNm	The encoder resolution for the axis.
sampleTimeUs	Sampling clock frequency based on the Servo Rate (20kHz = 51.2, 4kHz = 256).

4.7.36 FirmwareVersionInfo Struct

```
public struct FirmwareVersionInfo
```

Firmware version information of a PMD Controller.

4.7.36.1 Properties

4.7.36.1.1 Full

```
public uint Full { get; set; }
```

Full version number.

Details

This is all version numbers combined into a single 32-bit number. It includes additional information such as the product family, motor type and numbers of axes.

4.7.36.1.2 Major

```
public uint Major { get; set; }
```

Major version number.

4.7.36.1.3 Minor

```
public uint Minor { get; set; }
```

Minor version number.

4.7.36.1.4 Special

```
public uint Special { get; set; }
```

Special version number.

4.7.36.2 Public Methods

4.7.36.2.1 Constructor: FirmwareVersionInfo()

```
public FirmwareVersionInfo(uint major,
                           uint minor,
                           uint special,
                           uint full)
```

Constructor

Parameters:

```
major
minor
special
full
```

4.7.36.2.2 ToString()

```
public override string ToString()
```

Get version info appropriately presented as a string.

4.7.37 MotionErrorSettings Struct

```
public struct MotionErrorSettings
```

Defines the motion error settings (i.e. the position error limit and the action to take if a motion error occurs)

4.7.37.1 Properties

4.7.37.1.1 ActionOnError

```
public MotionErrorAction ActionOnError { get; set; }
```

The action to perform when a motion error occurs.

4.7.37.1.2 PositionErrorLimit

```
public double PositionErrorLimit { get; set; }
```

The position error limit in selected units (mm, um, or nm). If the actual position deviates from the commanded position by more than this value, a motion error will be generated.

4.7.37.2 Public Methods

4.7.37.2.1 Constructor: MotionErrorSettings()

```
public MotionErrorSettings(double positionErrorLimit,
                           MotionErrorAction actionOnError)
```

Constructor

Parameters:

```
positionErrorLimit The position error limit in selected units (mm, um, or nm).
actionOnError      The action to perform when a motion error occurs.
```

4.7.38 MotionTrackingSettings Struct

```
public struct MotionTrackingSettings
```

Defines when an axis move is determined to be complete by the motion controller.

4.7.38.1 Properties

4.7.38.1.1 Mode

```
public MotionCompleteMode Mode { get; set; }
```

Determines how the motion controller determines a move is complete.

4.7.38.1.2 SettleTimeMs

```
public double SettleTimeMs { get; set; }
```

The length of time an axis must remain within the settle window before the move is deemed to be complete. This setting is only applicable when the motion complete mode is set to 'MotionSettles'.

4.7.38.1.3 SettleWindow

```
public double SettleWindow { get; set; }
```

The size of the settle window in selected units (mm, um, or nm). This setting is only applicable when the motion complete mode is set to 'MotionSettles'.

4.7.38.2 Public Methods

4.7.38.2.1 Constructor: MotionTrackingSettings()

```
public MotionTrackingSettings(MotionCompleteMode mode,
    double settleTimeMs,
    double settleWindow)
```

Constructor

Parameters:

mode Determines how the motion controller determines a move is complete.

settleTimeMs The length of time an axis must remain within the settle window before the move is deemed to be complete.

settleWindow The size of the settle window in selected units (mm, um, or nm).

4.7.39 PhaseAngleInfo Struct

```
public struct PhaseAngleInfo
```

Phase angle information from the PMD controller.

4.7.39.1 Properties

4.7.39.1.1 IsValid

```
public bool IsValid { get; set; }
```

Whether the phase angle is valid.

4.7.39.1.2 PhaseAngleDeg

```
public uint PhaseAngleDeg { get; set; }
```

Phase angle in degrees.

4.7.39.1.3 PhaseAngleRaw

```
public uint PhaseAngleRaw { get; set; }
```

Raw phase angle value.

4.7.39.2 Public Methods

4.7.39.2.1 Constructor: PhaseAngleInfo()

```
public PhaseAngleInfo(uint raw,
                      uint deg,
                      bool isValid)
```

Constructor.

Parameters:

```
raw
deg
isValid
```

4.7.40 PositionLoopSettings Struct

```
public struct PositionLoopSettings
```

Defines position loop parameters for an axis.

4.7.40.1 Properties

4.7.40.1.1 DerivativeTime

```
public int DerivativeTime { get; set; }
```

PID Derivative Time

4.7.40.1.2 Ilimit

```
public int Ilimit { get; set; }
```

PID Integrator Limit

4.7.40.1.3 IsPositionLoopTuningAvailable

```
public bool IsPositionLoopTuningAvailable { get; set; }
```

Whether position loop tuning is available for the current motor type. As a result from the get function, the position loop parameters are valid only if this is true. Can be ignored when setting/saving parameters.

4.7.40.1.4 Kaff

```
public int Kaff { get; set; }
```

Acceleration Feedforward Gain

4.7.40.1.5 Kd

```
public int Kd { get; set; }
```

PID Derivative Gain

4.7.40.1.6 Ki

```
public int Ki { get; set; }
```

PID Integrator Gain

4.7.40.1.7 Kout

```
public int Kout { get; set; }
```

PID Output Gain, without scaling.

4.7.40.1.8 Kp

```
public int Kp { get; set; }
```

PID Proportional Gain

4.7.40.1.9 Kvff

```
public int Kvff { get; set; }
```

Velocity Feedforward Gain

4.7.40.1.10 this[string key]

```
public int this[string key] { get; set; }
```

Get and set position loop settings using index operator.

4.7.40.2 Public Methods

4.7.40.2.1 Constructor: PositionLoopSettings()

```
public PositionLoopSettings(int kp,
                             int ki,
                             int ilimit,
                             int kd,
                             int derivativeTime,
                             int kout,
                             int kvff,
                             int kaff,
                             bool isPositionLoopTuningAvailable)
```

Constructor

Parameters:

kp	PID Proportional Gain
ki	PID Integrator Gain
ilimit	PID Integrator Limit
kd	PID Derivative Gain
derivativeTime	PID Derivative Time
kout	PID Output Gain, without scaling.
kvff	Velocity Feedforward Gain
kaff	Acceleration Feedforward Gain
isPositionLoopTuningAvailable	Whether position loop tuning is available for the current motor type.

4.7.40.2.2 AreParamsEqual()

```
public bool AreParamsEqual(PositionLoopSettings settingsToCompare)
```

Check if parameters of this position loop settings are equal to another.

Parameters:

settingsToCompare

4.7.40.2.3 GetPercentKout()

```
public double GetPercentKout(int numDecimalPoints)
```

Get Kout in percentage form with variable number of decimal points.

Parameters:

numDecimalPoints Number of decimal points for the percentage value.

Returns:

Kout in percentage form with the specified number of decimal points.

Exceptions:

ArgumentException Thrown when the percentage is not between 0 and 100.

4.7.40.2.4 GetPercentKout()

```
public double GetPercentKout()
```

Get Kout in percentage form.

Returns:

Kout in percentage form.

Exceptions:

ArgumentException Thrown when the percentage is not between 0 and 100.

4.7.40.2.5 GetSettingNames()

```
public static IList<string> GetSettingNames()
```

Names of all position loop parameters.

Details

The order of setting names is the same as the parameter encoding value for Get/SetPositionLoop Motion Control IC command.

Returns:

List of position loop parameter names

4.7.40.2.6 SetPercentKout()

```
public void SetPercentKout(double percent)
```

Set Kout from a percentage value.

Parameters:

percent Percentage representing Kout value.

Exceptions:

ArgumentException Thrown when the percentage is not between 0 and 100.

4.7.41 ScriptParameter Struct

```
public struct ScriptParameter
```

Represents a parameter which can be passed to a script.

4.7.41.1 Public Methods

4.7.41.1.1 Constructor: ScriptParameter()

```
public ScriptParameter(string name,
                      string value,
                      ScriptParameterType type)
```

Construct a script parameter.

Parameters:

name The parameter name.

value The parameter value in string form.

type The parameter type (e.g. bool, string, int, etc).

4.7.41.2 Public Variables

4.7.41.2.1 Name

```
readonly string Name
```

The parameter name.

4.7.41.2.2 Type

```
readonly ScriptParameterType Type
```

The parameter type (e.g. bool, string, int, etc).

4.7.41.2.3 Value

```
readonly string Value
```

The parameter value in string form.

4.7.42 MotionSynergyAPI Namespace

4.7.42.1 Delegates

4.7.42.1.1 ConnectionStateChangedEventHandler()

```
public delegate void ConnectionStateChangedEventHandler(object sender,
                                                       ConnectionStateChangedEventArgs message)
```

Delegate to receive IConnectionState.ConnectionStateChanged events.

Parameters:

sender This will be the IMotionSynergyAPI instance.

message Contains details of the change in connection state.

4.7.42.1.2 DigitalInputChangedEventHandler()

```
public delegate void DigitalInputChangedEventHandler(object sender,
                                                       DigitalInputEventArgs message)
```

Delegate to receive IDigitalInput.DigitalInputChanged events.

Parameters:

sender This will be the IDigitalInput device instance.
message Contains details of the change in the digital input device register.

4.7.42.1.3 InstrumentStateChangedEventHandler()

```
public delegate void InstrumentStateChangedEventHandler(object sender,
                                                    InstrumentStateChangedEventArgs message)
```

Delegate to receive IMotionSynergyAPIBase.InstrumentStateChanged events.

Parameters:

sender This will be the IMotionSynergyAPI instance.
message Contains details of the change in instrument state.

4.7.42.1.4 LogEntriesPublishedCallback()

```
public delegate void LogEntriesPublishedCallback(LogEntry logEntry)
```

Defines a delegate to invoke when publishing one or more log entries (see IDiagnostics.SubscribeForLogEntries(LogLevel, LogEntriesPublishedCallback)).

Parameters:

logEntry A new log entry.

4.7.42.2 Enumerations

4.7.42.2.1 ActiveControlMode

```
public enum ActiveControlMode
{
    Normal, // The axis is controlled in the normal manner, using normal move profiles and move
    commands.

    UDPTIME, // The axis is controlled using a User Defined Profile (UDP) time-based profile.

    UDPCoordinatedMotion, // The axis is controlled using a User Defined Profile (UDP) master / driven
    axis profile, where this axis is operating as the driven axis.

    UDPSineWave, // The axis is controlled using a User Defined Profile (UDP) sine wave profile.

    RME, // The axis is controlled using a Repetitive Motion Enhancement (RME) profile.

    ExternalControl, // The axis is controlled via an external step/direction electronic signal.
}
```

Defines the axis' active control mode, which determines how the axis is being commanded and the type of move profile it's using.

4.7.42.2.2 AxisRecoveryOperation

```
public enum AxisRecoveryOperation
{
    NoneRequired, // The axis is already in a state where it can perform moves and no recovery action is
    required.

    ResetPosition, // The axis has encountered a position error and ResetPosition() must be called to
    reset its commanded position to match its actual position.

    ResetPositionAndEnable, // The axis has encountered a position error and the motor has been disabled
    (de-energized). ResetPosition() must be called to reset its commanded position to match its actual
    position, followed by Enable() to re-energize the motor.
}
```

```

    Initialize, // The axis has encountered a general error and must be re-initialized by calling
    Initialize().
}

```

Defines the recovery operation required return the axis to a state where it can performs moves.

4.7.42.2.3 BiquadFilterType

```

public enum BiquadFilterType
{
    None, // The biquad filter is not in use.

    Unknown, // The type of filter is unknown. The filter is in use on the motion controller but there
    is no filter in configuration with a matching set of coefficients.

    Notch, // A Notch filter

    LowPass, // A Low Pass filter

    Lead, // The Lead component of a Lead/Lag filter setup

    Lag, // The Lag component of a Lead/Lag filter setup

    UpstreamFilterNotInUse, // The upstream filter is not in use. The upstream filter needs to be
    enabled first, and then this filter can be used.
}

```

Defines the different types of biquad filters available.

4.7.42.2.4 CanBaudRate

```

public enum CanBaudRate
{
    Baud1000000, // CAN Baud Rate 0 = 1 Mbit/s

    Baud800000, // CAN Baud Rate 1 = 800 Kbit/s

    Baud500000, // CAN Baud Rate 2 = 500 Kbit/s

    Baud250000, // CAN Baud Rate 3 = 250 Kbit/s

    Baud125000, // CAN Baud Rate 4 = 125 Kbit/s

    Baud50000, // CAN Baud Rate 5 = 50 Kbit/s

    Baud20000, // CAN Baud Rate 6 = 20 Kbit/s

    Baud10000, // CAN Baud Rate 7 = 10 Kbit/s
}

```

Defines the set of baud rates that can be used for CAN communications.

4.7.42.2.5 ConnectionState

```

public enum ConnectionState
{
    Disconnected, // There is no connection between MotionSynergyAPI and the Subsystem Controller
    firmware.

    Connecting, // MotionSynergyAPI is in the process of trying to establish a connection to the
    Subsystem Controller firmware.

    Connected, // There is a connection between MotionSynergyAPI and the Subsystem Controller firmware.
}

```

Represents the state of the connection between MotionSynergyAPI (running on the SBC) and the Subsystem Controller firmware (running on the Instrument hardware).

4.7.42.2.6 DigitalOutputTrigger

```
public enum DigitalOutputTrigger
{
    Disabled, // Digital output is set low and triggers are disabled (i.e. the digital output state is
              // not affected by the axis state).

    InMotion, // The digital output will be set high whenever the axis is in motion.

    MotionComplete, // The digital output will be set high whenever the axis completes a move.

    AtMaxVelocity, // The commanded velocity output from the profile generator.

    Immediate, // The digital output will be set high immediately (i.e. the digital output is directly
               // controlled by software).
}
```

Defines the set of triggers which can be used to set the AxisOut1 general purpose output (GPO).

4.7.42.2.7 InstrumentState

```
public enum InstrumentState
{
    Uninitialized, // Initial state when Motion Synergy API is first created.

    Initializing, // Instrument is in the process of initializing (i.e. the Initialize command is
                 // running).

    Ready, // The instrument is idle and ready to run a test or perform maintenance.

    Running, // The instrument is running a test.

    Error, // The instrument has encountered an error and cannot run a test. It needs to be re-
           // initialized or powered off and on.

    FatalError, // The instrument software has encountered an unexpected error and must be restarted
               // (i.e. powered off and on). The instrument can only be restarted from this state.

    Shutdown, // The instrument has been shut down (i.e. the Shutdown command has been called) in
              // preparation for power off. The instrument can only be restarted from this state.
}
```

Describes the overall state of the instrument.

4.7.42.2.8 LogLevel

```
public enum LogLevel
{
    Trace, // Trace level for verbose code path evaluation and low level comms.

    Debug, // Debug level for verbose results and values.

    Info, // Info level for significant events.

    Warning, // Warning level for potential/handled error cases.

    Error, // Error level for non-fatal error cases.

    Fatal, // Fatal level for application closure cases.
}
```

Represents the log level on a log entry.

4.7.42.2.9 MotionCompleteMode

```
public enum MotionCompleteMode
{
    CommandedPositionReached, // A move is complete as soon as the commanded position has been reached.
                             // The axis settle time and window is ignored.
}
```

```

    MotionSettles, // A move is completed once the axis has reached the commanded position and has
    maintained position within the settle window for the length of the settle time.

```

```

}

```

Defines when the motion controller determines a move has completed.

4.7.42.2.10 MotionControllerConfigurationType

```

public enum MotionControllerConfigurationType
{
    None, // Invalid motion controller configuration type.

    MotionTracking, // The motion tracking configuration settings (MotionTrackingSettings).

    MotionError, // The motion error configuration settings (MotionErrorSettings).

    PositionLoop, // The position loop configuration settings (PositionLoopSettings).

    ActiveCommunication, // The active communication configuration settings (CommunicationSettings).
    This requires the communications link to be re-established (i.e. disconnect / connect).

    InactiveCommunication, // The inactive communication configuration settings (CommunicationSettings).
    Does not affect the current communications link.

    CurrentLoop, // The current loop settings (CurrentLoopSettings).

    BiquadFilter, // The biquad filter configuration settings (BiquadFilterSettings).
}

```

Defines the different types of motion controller configuration settings which can be written to NVRAM using the IMotionControllerConfiguration.WriteToNVRAM method.

4.7.42.2.11 MotionErrorAction

```

public enum MotionErrorAction
{
    None, // Motion errors are ignored and no action is taken.

    AbruptStop, // The axis is brought to an abrupt stop.

    SmoothStop, // The axis is brought to a smooth stop using the current deceleration setting.

    DisableVelocityLoop, // Disable velocity loop and higher modules. Note this is currently not
    supported by SmartStage.

    DisablePositionLoop, // Disable position loop and higher modules.

    DisableCurrentLoop, // Disable current loop and higher modules.

    DisableMotor, // The motor is disabled (de-energized).

    AbruptStopWithPosErrorClear, // Abrupt stop with position error clear.

    PassiveBraking, // Passive braking. Note this is currently not supported by SmartStage.
}

```

Defines the action performed by the motion controller when a motion error (i.e. position error) is detected.

4.7.42.2.12 SelectableMotionErrorAction

```

public enum SelectableMotionErrorAction
{
    None, // Motion errors are ignored and no action is taken.

    AbruptStop, // The axis is brought to an abrupt stop.
}

```

```
SmoothStop, // The axis is brought to a smooth stop using the current deceleration setting.
DisableMotor, // The motor is disabled (de-energized).
}
```

Like MotionErrorAction, but only ones which can be selected from the application. Must be a subset of MotionErrorAction.

4.7.42.2.13 ScriptParameterType

```
public enum ScriptParameterType
{
    UnknownParam, // A script parameter of unknown type - treat it as a simple ascii string.
    BooleanParam, // A script parameter representing a true / false value
    IntegerParam, // A script parameter representing an integer
    DoubleParam, // A script parameter representing a double precision floating point
    StringParam, // A script parameter representing a simple ascii string
}
```

Represents the type of a script parameter

4.7.42.2.14 SerialBaudRate

```
public enum SerialBaudRate
{
    Baud1200, // Serial Baud Rate 1.2 Kbps
    Baud2400, // Serial Baud Rate 2.4 Kbps
    Baud9600, // Serial Baud Rate 9.6 Kbps
    Baud19200, // Serial Baud Rate 19.2 Kbps
    Baud57600, // Serial Baud Rate 57.6 Kbps
    Baud115200, // Serial Baud Rate 115.2 Kbps
    Baud230400, // Serial Baud Rate 230.4 Kbps
    Baud460800, // Serial Baud Rate 460.8 Kbps
}
```

Defines the set of baud rates that can be used for Serial communications.

4.7.42.2.15 SerialProtocol

```
public enum SerialProtocol
{
    Point2Point, // RS232 point-to-point
    MultiDropUsingIdleLineDetection, // RS485 Multi-drop using idle-line detection
}
```

Defines the protocol to use for serial communications.

4.7.42.2.16 TraceTrigger

```
public enum TraceTrigger
{
    Immediate, // Trace data capture begins immediately and continues until StopTraceCapture() is called.
    OnMotionStart, // Trace data capture begins at the start of the next move and ends when the move completes.
}
```



```

    OnMotionStartDelayed, // Trace data capture begins at a delayed time after the start of the next
                           move and ends when the move completes. This is only supported by the UDP sine wave move.

    OnMotionStartNoEnd, // Trace data capture begins at the start of the next move and continues until
                        StopTraceCapture() is called.
}

```

Defines the trigger to begin (end) a trace data capture.

4.7.42.2.17 TraceVariable

```

public enum TraceVariable
{
    None, // Used to terminate the list of trace variables to capture.

    PositionError, // The difference between the actual and commanded position.

    CommandedPosition, // The commanded position output from the profile generator.

    CommandedVelocity, // The commanded velocity output from the profile generator.

    CommandedAcceleration, // The commanded acceleration output from the profile generator.

    ActualPosition, // The actual position of the motor.

    ActualVelocity, // The actual velocity (calculated using a simple low-pass filter).

    ActiveMotorCommand, // The instantaneous motor command.

    MotionControlICTime, // The motion control IC time (units of servo cycles).

    PositionCaptureRegister, // The contents of the high-speed Capture register.

    PositionLoopIntegratorSum, // The integral value used in the position loop PID filter.

    PositionLoopIntegratorContribution, // The contribution of the integral portion of the PID filter.

    PositionLoopDerivative, // The derivative value of the position loop PID filter.

    Biquad1Input, // The value input to biquad 1 filter.

    Biquad2Input, // The value input to biquad 2 filter.

    EventStatusRegister, // The Event Status register.

    ActivityStatusRegister, // The Activity Status register.

    SignalStatusRegister, // The Signal Status register.

    DriveStatusRegister, // The Drive Status register.

    DriveFaultStatusRegister, // The Drive Fault Status register.

    PhaseAngle, // The motor phase angle.

    PhaseOffset, // The phase offset.

    PhaseACommand, // The output command for Phase A.

    PhaseBCommand, // The output command for Phase B.

    PhaseCCommand, // The output command for Phase C.

    AnalogInput0, // The most recently read value from the Analog0 signal.

    AnalogInput1, // The most recently read value from the Analog1 signal.

    AnalogInput2, // The most recently read value from the Analog2 signal.

    AnalogInput3, // The most recently read value from the Analog3 signal.
}

```

AnalogInput4, // The most recently read value from the Analog4 signal.

AnalogInput5, // The most recently read value from the Analog5 signal.

AnalogInput6, // The most recently read value from the Analog6 signal.

AnalogInput7, // The most recently read value from the Analog7 signal.

PhaseAngleScaled, // The normalized phase angle, scaled from 0 to 360 deg (0 to 0x7fff) rather than in encoder counts.

PhaseAReference, // The current Loop reference for phase A.

PhaseAError, // The current Loop Error for phase A.

PhaseAActualCurrent, // The current Loop actual current for phase A. Actual current for single phase (DC Brush) motors.

PhaseAIntegratorSum, // The current Loop integral sum for phase A.

PhaseAIntegratorContribution, // The current Loop integrator contribution to PI filter for phase A.

CurrentLoopAOutput, // The current Loop output for phase A.

PhaseBReference, // The current Loop reference for phase B.

PhaseBError, // The current Loop Error for phase B.

PhaseBActualCurrent, // The current Loop actual current for phase B.

PhaseBIntegratorSum, // The current Loop integral sum for phase B.

PhaseBIntegratorContribution, // The current Loop integrator contribution to PI filter for phase B.

CurrentLoopBOutput, // The current Loop output for phase B.

FOCDReference, // The FOC reference for D (direct) Loop.

FOCDError, // The FOC D (direct) Loop error.

FOCDFeedback, // The FOC D (direct) feedback current. Actual current for 3 phase motors (Brushless DC).

FOCDIntegratorSum, // The FOC integral for D (direct) Loop.

FOCDIntegratorContribution, // The FOC integrator contribution for D (direct) Loop.

FOCDOutput, // The FOC output for D (direct) Loop.

FOCQReference, // The FOC reference for Q (quadrature) Loop.

FOCQError, // The FOC Q (quadrature) Loop error.

FOCQFeedback, // The FOC Q (quadrature) feedback current.

FOCQIntegratorSum, // The FOC integral for Q (quadrature) Loop.

FOCQIntegratorContribution, // The FOC integrator contribution for Q (quadrature) Loop.

FOCQOutput, // The FOC output Q (quadrature) Loop.

FOCAlphaOutput, // The FOC output for phase Alpha.

FOCBetaOutput, // The FOC output for phase Beta.

BusVoltage, // The bus voltage.

Temperature, // The temperature of the drive's output stage.

FoldbackEnergy, // The i2t total foldback energy.

LegACurrent, // *Current1A reading*

LegBCurrent, // *Current1B reading*

LegCCurrent, // *Current1C reading*

LegDCurrent, // *Current1D reading*

PWMOutputA, // *Phase A PWM output*

PWMOutputB, // *Phase B PWM output (used with Brushless DC motors only)*

PWMOutputC, // *Phase C PWM output (used with Brushless DC and step motors only)*

ContourOutput, // *UDP active profile output based on current master source index and corresponding position in profile table.*

ContourOffset, // *UDP active profile offset (add to ContourOutput to calculate axis target position).*

SignalStatus_EncoderA, // *Signal Status bit 0*

SignalStatus_EncoderB, // *Signal Status bit 1*

SignalStatus_EncoderIndex, // *Signal Status bit 2*

SignalStatus_CaptureInput, // *Signal Status bit 3*

SignalStatus_Gate, // *Signal Status bit 3. Alternative name for TOP use.*

SignalStatus_EncoderHome, // *Signal Status bit 3. Treated as a flag.*

SignalStatus_PositiveLimit, // *Signal Status bit 4*

SignalStatus_NegativeLimit, // *Signal Status bit 5*

SignalStatus_AxisIn, // *Signal Status bit 6*

SignalStatus_AxisOut, // *Signal Status bit 10*

SignalStatus_FaultOut, // *Signal Status bit 14*

SignalStatus_GPI1, // *Signal Status - Encoder A (bit 0, axis 2)*

SignalStatus_GPI2, // *Signal Status - Encoder B (bit 1, axis 2)*

SignalStatus_Trigger, // *Signal Status - Trigger (bit 2 and 3, axis 2)*

ActivityStatus_PhasingInitialized, // *Activity Status - PhasingInitialized (bit 0)*

ActivityStatus_AtMaximumVelocity, // *Activity Status - AtMaximumVelocity (bit 1)*

ActivityStatus_Tracking, // *Activity Status - Tracking (bit 2)*

ActivityStatus_CurrentProfileMode, // *Activity Status - Current Profile Mode (bit 3-5)*

ActivityStatus_Reserved, // *Activity Status - Reserved*

ActivityStatus_AxisSettled, // *Activity Status - Axis Settled (bit 7)*

ActivityStatus_MotorOn, // *Activity Status - Position Loop Enabled (bit 8)*

ActivityStatus_PositionCapture, // *Activity Status - Position Capture (bit 9)*

ActivityStatus_InMotion, // *Activity Status - In-motion (bit 10)*

ActivityStatus_InPositiveLimit, // *Activity Status - In Positive Limit (bit 11)*

ActivityStatus_InNegativeLimit, // *Activity Status - In Negative Limit (bit 12)*

ActivityStatus_ProfileSegment, // *Activity Status - Profile Segment (bit 13-15)*

```
}
```

Defines the set of trace variables which can be captured during a move. See `EnableTraceCapture()`.

4.7.43 Alerts Namespace

4.7.43.1 Delegates

4.7.43.1.1 AlertRaisedEventHandler()

```
public delegate void MotionSynergyAPI.AlertRaisedEventHandler(object sender,
                                                             AlertRaisedEventArgs message)
```

Delegate to receive `IAleris.AlertRaised` events.

Parameters:

sender This will be the `IMotionSynergyAPI` instance.

message Contains details of the Alert being raised.

4.7.43.1.2 AlertsClearedEventHandler()

```
public delegate void MotionSynergyAPI.AlertsClearedEventHandler(object sender,
                                                                AlertsClearedEventArgs message)
```

Delegate to receive `IAleris.AlertsCleared` events.

Parameters:

sender This will be the `IMotionSynergyAPI` instance.

message Contains details of the Alerts being cleared.

4.7.43.2 Enumerations

4.7.43.2.1 AlertCode

```
public enum AlertCode
{
    Success, // Indicates a command has completed successfully. This will not result in an alert being
             // raised. (Severity = Low)

    AxisDeviceAlertInvalidConfiguration, // Invalid configuration detected for axis device (Severity =
    High)

    AxisDeviceAlertUnsupportedModelType, // Unsupported axis device model type (Severity = High)

    AxisDeviceAlertAlreadyConfigured, // Detected attempt to configure the axis device more than once
    (Severity = High)

    AxisDeviceAlertPositionError, // Position error encountered on the axis device (Severity = High)

    AxisDeviceAlertGeneralError, // General axis error detected (Severity = High)

    AxisDeviceAlertSettingError, // Bad axis setting encountered (Severity = High)

    AxisDeviceAlertBump, // Axis bump detected (Severity = High)

    AxisDeviceAlertHalt, // Axis halted (Severity = High)

    AxisDeviceAlertEstop, // Axis device ESTOPPED (Severity = High)

    AxisDeviceAlertUninitialized, // Axis device not initialized (Severity = High)

    AxisDeviceAlertBusy, // Operation requested on a busy axis device (Severity = High)
```

```

AxisDeviceAlertOutOfRange, // Out of range error detected for axis (Severity = High)

AxisDeviceAlertCallInhibited, // Call to axis inhibited (Severity = High)

AxisDeviceAlertOverTemperature, // The axis motor temperature has exceeded the temperature limit
(Severity = High)

AxisDeviceAlertCurrentFoldback, // The axis motor amplifier current output has gone into a foldback
condition (Severity = High)

AxisDeviceAlertOverVoltage, // The axis motor bus voltage has exceeded the maximum bus voltage limit
(Severity = High)

AxisDeviceAlertUnderVoltage, // The axis motor bus voltage has gone below the minimum bus voltage
limit (Severity = High)

AxisDeviceCommunicationsFailure, // Communications has been lost to the axis (Severity = High)

StdException, // Indicates the command failed due to a C++ std::exception being thrown (Severity =
High)

BoostException, // Indicates the command failed due to a C++ boost::exception being thrown (Severity
= High)

UnknownException, // Indicates the command failed due to an unknown exception being thrown (Severity
= SafetyCritical)

ManagedException, // Indicates the command failed due to a managed exception being thrown (Severity
= High)

ExecuteScriptError, // The script has encountered an error during execution. Refer to the log for
more details. (Severity = High)

PropertyErrorHigh, // An unknown High severity error has occurred while reading an interface
property. The software will need to be re-initialized. (Severity = High)

PropertyErrorSafetyCritical, // An unknown Safety Critical severity error has occurred while reading
an interface property. The software will need to be restarted. (Severity = SafetyCritical)

UnhandledExceptionOnDispatcher, // An unhandled exception has been caught by a Dispatcher thread
causing it to exit. (Severity = SafetyCritical)

FailedToInitialize, // Initialization of the hardware subsystem failed. The software will need to be
re-initialized. (Severity = High)

ScriptNotFoundError, // A script file matching the supplied script name does not exist. (Severity =
Medium)

ScriptAborted, // A script was aborted during execution. (Severity = Medium)
}

```

Unique Id for each type of alert that can be raised by InstrumentControl.

Details

This enum is automatically generated from the alerts configuration file.

4.7.43.2.2 AlertSeverity

```

public enum AlertSeverity
{
    Unknown, // Default value indicating an unknown / uninitialized state. Must be set to one of the
following valid values.

    Low, // Alert is for information purposes only. Non-fatal, no action required by the software.

    Medium, // Alert provides warning information. Non-fatal, no action required by the software but
could indicate an intermittent failure or something which will require action in the future.

    High, // Alert indicates an error has occurred and MotionSynergyAPI is now in the Error state. As a

```

minimum, MotionSynergyAPI must be reinitialized before running the next test. May indicate a hardware error which requires a service call to correct.

SafetyCritical, // Alert indicates MotionSynergyAPI has encountered an unexpected error and is now in the Error state. MotionSynergyAPI must be restarted.

}

The severity of the Alert dictates the type of action the software must take. Low and Medium severity Alerts are non-fatal and serve as information or warnings. High and SafetyCritical indicate a fatal error has occurred and require the software to be re-initialized, restarted or a service call to rectify.

4.7.44 Alert Codes

Code	Name	Severity	Description
0	Success	Low	Indicates a command has completed successfully. This will not result in an alert being raised.
10001	Axis Device Alert Invalid Configuration	High	Invalid configuration detected for axis device
10002	Axis Device Alert Unsupported Model Type	High	Unsupported axis device model type
10003	Axis Device Alert Already Configured	High	Detected attempt to configure the axis device more than once
10004	Axis Device Alert Position Error	High	Position error encountered on the axis device
10005	Axis Device Alert General Error	High	General axis error detected
10006	Axis Device Alert Setting Error	High	Bad axis setting encountered
10007	Axis Device Alert Bump	High	Axis bump detected
10008	Axis Device Alert Halt	High	Axis halted
10009	Axis Device Alert Estop	High	Axis device ESTOPPED
10010	Axis Device Alert Uninitialized	High	Axis device not initialized
10011	Axis Device Alert Busy	High	Operation requested on a busy axis device
10012	Axis Device Alert Out Of Range	High	Out of range error detected for axis

10013	Axis Device Alert Call Inhibited	High	Call to axis inhibited
10014	Axis Device Alert Over Temperature	High	The axis motor temperature has exceeded the temperature limit
10015	Axis Device Alert Current Foldback	High	The axis motor amplifier current output has gone into a foldback condition
10016	Axis Device Alert Over Voltage	High	The axis motor bus voltage has exceeded the maximum bus voltage limit
10017	Axis Device Alert Under Voltage	High	The axis motor bus voltage has gone below the minimum bus voltage limit
10018	Axis Device Communications Failure	High	Communications has been lost to the axis
30000	Std Exception	High	Indicates the command failed due to a C++ std::exception being thrown
30001	Boost Exception	High	Indicates the command failed due to a C++ boost::exception being thrown
30002	Unknown Exception	Safety Critical	Indicates the command failed due to an unknown exception being thrown
30003	Managed Exception	High	Indicates the command failed due to a managed exception being thrown
30004	Execute Script Error	High	The script has encountered an error during execution. Refer to the log for more details.
30005	Property Error High	High	An unknown High severity error has occurred while reading an interface property. The software will need to be re-initialized.
30006	Property Error Safety Critical	Safety Critical	An unknown Safety Critical severity error has occurred while reading an interface property. The software will need to be restarted.
30007	Unhandled Exception On Dispatcher	Safety Critical	An unhandled exception has been caught by a Dispatcher thread causing it to exit.
30008	Failed To Initialize	High	Initialization of the hardware subsystem failed. The software will need to be re-initialized.
30009	Script Not Found Error	Medium	A script file matching the supplied script name does not exist.
30010	Script Aborted	Medium	A script was aborted during execution.

5 Python Software Integration

The *Motion Synergy API* can be called from Python 3.x scripts using [Python.NET](#). It's supported on both Windows and Linux.

5.1 Running the example Python Console Application

An example Python console application has been provided which demonstrates how to use the *Motion Synergy API* from python. It's located in the following folder:

```
\SourceCode\Examples\Python\ConsoleApplication
```

You can run this application as follows:

1. Ensure the MotionSynergyGUI application has been run and you have selected your product type and communications settings. You should have tested your setup to ensure you can control your Dover Motion product.
2. On Linux you will need make sure you have installed Python, Pip and the .NET 8.0 SDK (not required for Windows).

- a. Setup Python & Pip.

```
sudo apt update
sudo apt install python-is-python3
sudo apt install pip
```

- b. Add the Microsoft Package Repository so the .NET 8.0 SDK can be found.

```
# Install pre-requisite packages.
sudo apt install -y wget apt-transport-https software-properties-common
# Download the Microsoft repository GPG keys
wget -q "https://packages.microsoft.com/config/ubuntu/$(lsb_release -rs)/packages-microsoft-prod.deb"
# Register the Microsoft repository GPG keys
sudo dpkg -i packages-microsoft-prod.deb
# Delete the the Microsoft repository GPG keys file
rm packages-microsoft-prod.deb
# Update the list of packages after we added packages.microsoft.com
sudo apt update
```

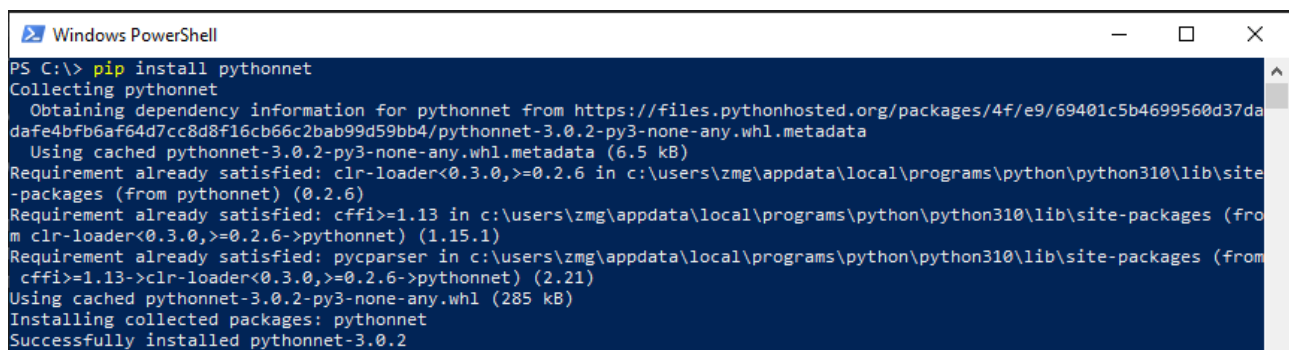
- c. Install the .NET 8.0 SDK

```
sudo apt install -y dotnet-sdk-8.0
```

3. Pip install Python.NET.

```
pip install pythonnet
```

e.g.

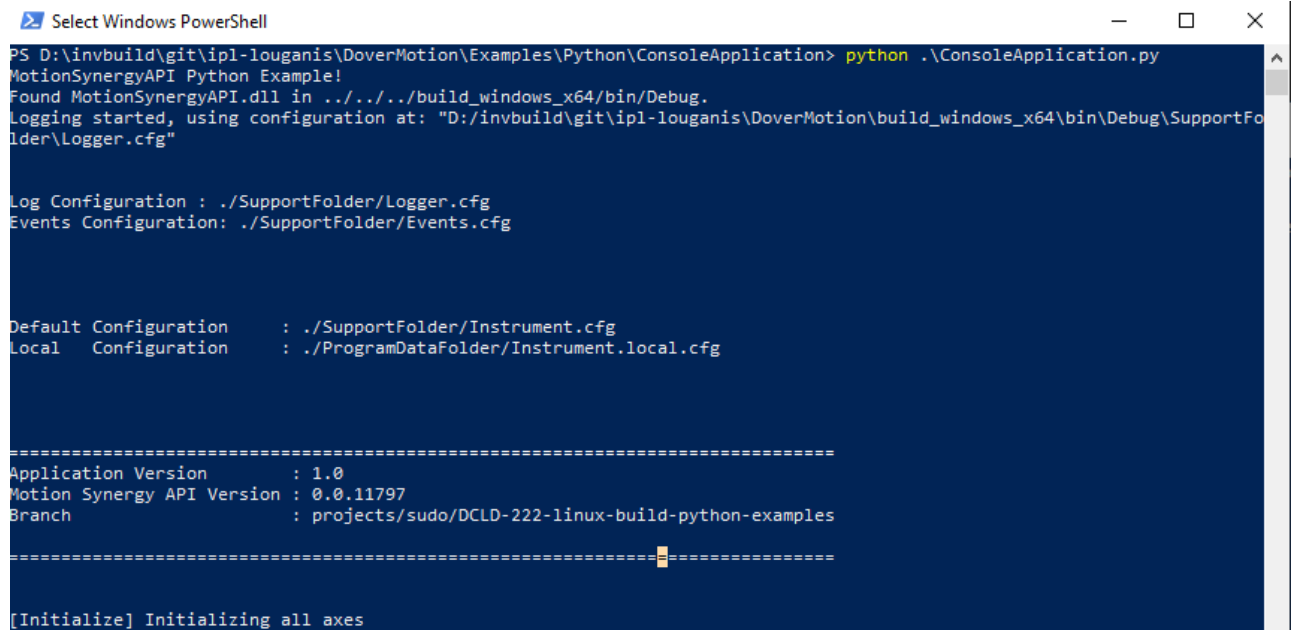


```
Windows PowerShell
PS C:\> pip install pythonnet
Collecting pythonnet
  Obtaining dependency information for pythonnet from https://files.pythonhosted.org/packages/4f/e9/69401c5b4699560d37da
d4fe4bf66af64d7cc8d8f16cb66c2bab99d59bb4/pythonnet-3.0.2-py3-none-any.whl.metadata
  Using cached pythonnet-3.0.2-py3-none-any.whl.metadata (6.5 kB)
Requirement already satisfied: clr-loader<0.3.0,>=0.2.6 in c:\users\zmg\appdata\local\programs\python\python310\lib\site
-packages (from pythonnet) (0.2.6)
Requirement already satisfied: cffi>=1.13 in c:\users\zmg\appdata\local\programs\python\python310\lib\site-packages (fro
m clr-loader<0.3.0,>=0.2.6->pythonnet) (1.15.1)
Requirement already satisfied: pycparser in c:\users\zmg\appdata\local\programs\python\python310\lib\site-packages (from
 cffi>=1.13->clr-loader<0.3.0,>=0.2.6->pythonnet) (2.21)
Using cached pythonnet-3.0.2-py3-none-any.whl (285 kB)
Installing collected packages: pythonnet
Successfully installed pythonnet-3.0.2
```

4. Run the example application from Python:


```
python .\ConsoleApplication.py
```

You should see output similar to the following:



```

PS D:\invbuild\git\ipl-louganis\DoverMotion\Examples\Python\ConsoleApplication> python .\ConsoleApplication.py
MotionSynergyAPI Python Example!
Found MotionSynergyAPI.dll in ../../../../build_windows_x64/bin/Debug.
Logging started, using configuration at: "D:/invbuild/git/ipl-louganis/DoverMotion/build_windows_x64/bin/Debug/SupportFolder/Logger.cfg"

Log Configuration : ./SupportFolder/Logger.cfg
Events Configuration: ./SupportFolder/Events.cfg

Default Configuration : ./SupportFolder/Instrument.cfg
Local Configuration : ./ProgramDataFolder/Instrument.local.cfg

=====
Application Version : 1.0
Motion Synergy API Version : 0.0.11797
Branch : projects/sudo/DCLD-222-linux-build-python-examples
=====

[Initialize] Initializing all axes

```

5.2 Python API

When calling the *Motion Synergy API* from Python you are using the .NET interface. This is documented in the section describing '*.NET Software Integration*'.

5.2.1 Python and Async Methods

Most of the methods on the axis interface are asynchronous. When you call one of these methods from Python, you will receive a reference to an async task. To wait for the task to complete, you need to access the *Result* property of the task. e.g.

```

# Begin a move on each axis to a named position (defined in Instrument.cfg)
yAxisTask = yAxis.NMoveAbsolute("PositiveLimit")
xAxisTask = xAxis.NMoveAbsolute("PositiveLimit")

# Both moves are now executing in parallel.

# Wait for the X Axis to complete its move.
result = xAxisTask.Result
if not result.Success:
    sys.exit(f"{xAxis.Name} failed: {result}")

# Wait for the Y Axis to complete its move.
result = yAxisTask.Result
if not result.Success:
    sys.exit(f"{yAxis.Name} failed: {result}")

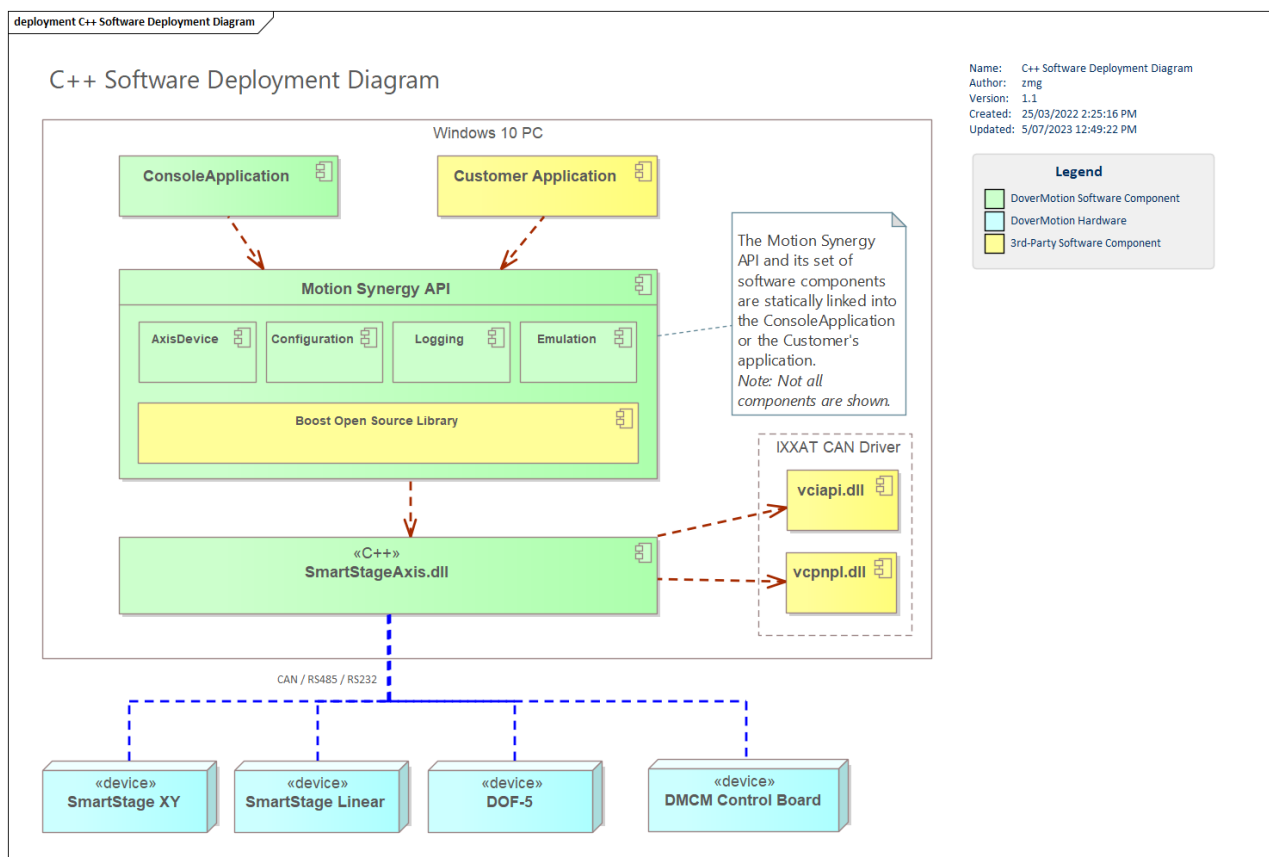
```

6 C++ Software Integration

Dover Motion provides a complete set of C++ source code files packaged as static libraries for integration with native applications or other native 3rd-party applications.

The source code implements communication and control of the *Magellan* motion control chip(s) via one of the supported communication links (RS232, RS485 or CAN). It also includes support for basic infrastructure (i.e. Configuration, Logging, Error Reporting, Device Emulation). The individual software components are wrapped by an *MotionSynergyAPI* class.

The following diagram illustrates how the software is deployed:



6.1 Building and Running the C++ Console Application

There is an example which demonstrates how to integrate the C++ source code into a console application. It's located in the following folder:

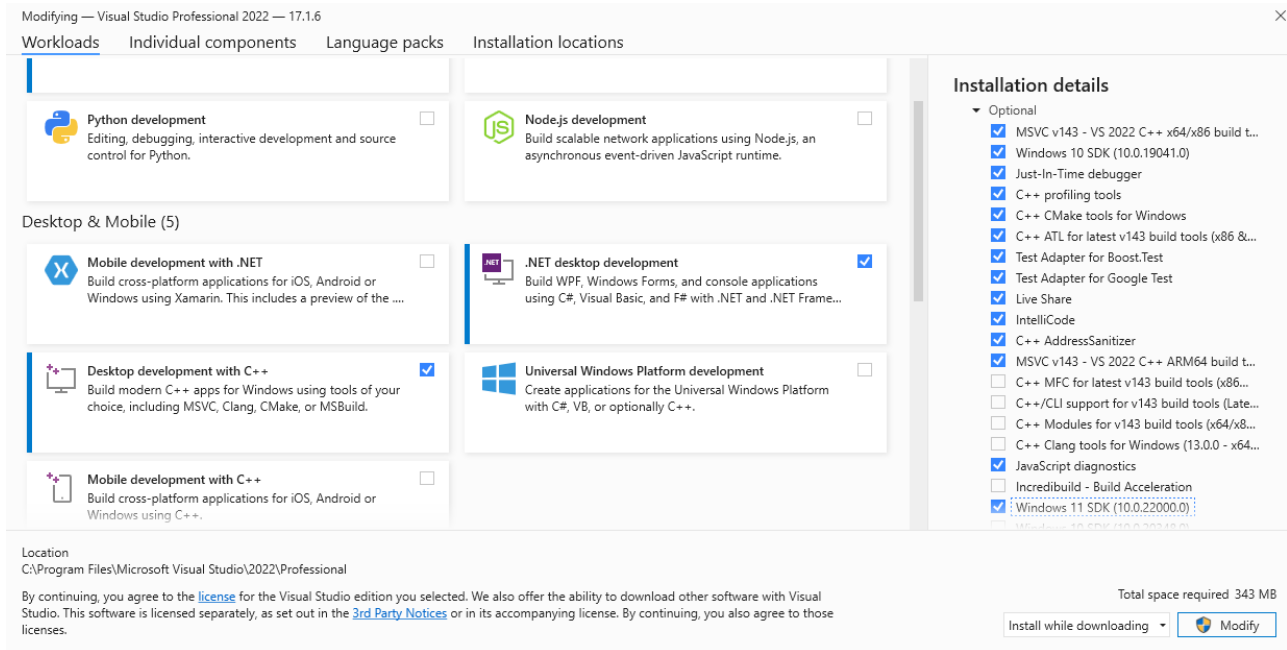
```
\SourceCode\Examples\CPlusPlus\ConsoleApplication
```

Important: You must have first run the *MotionSynergyGUI* and selected your product and communication settings before building and running the example applications.

6.1.1 Windows

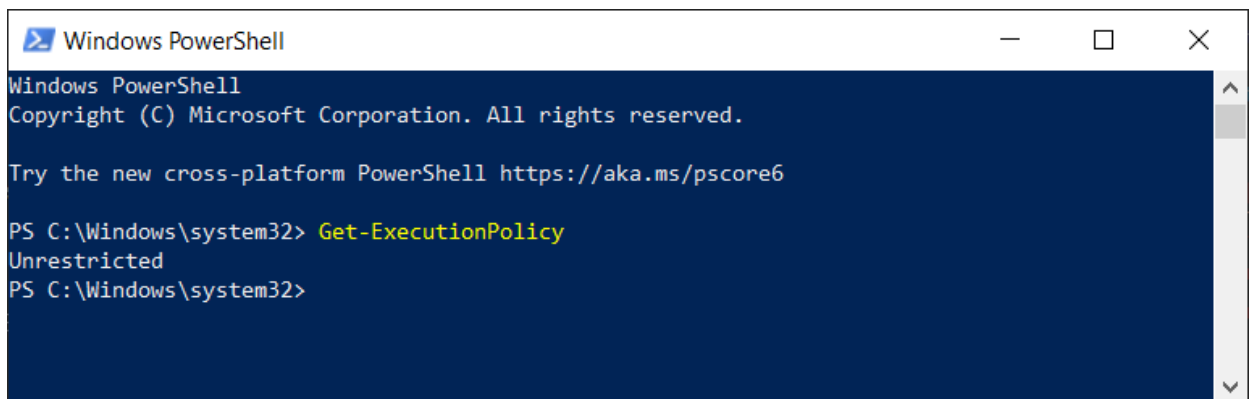
Before you can build this application you must have installed the following tools:

- CMake v3.25 or later. See <https://cmake.org/download/>. Make sure you select "Add to PATH for current user" as part of the installation process.
- Visual Studio 2022 v17.14.21 (or above).
- Windows 11 SDK (10.0.22621.0), which is backward compatible with Windows 10. This can be selected from the Visual Studio Installer.



The source code is accompanied by a set of CMake files. Rather than loading these directly into Visual Studio, a solution and set of project files must be generated from the CMake files, as follows:

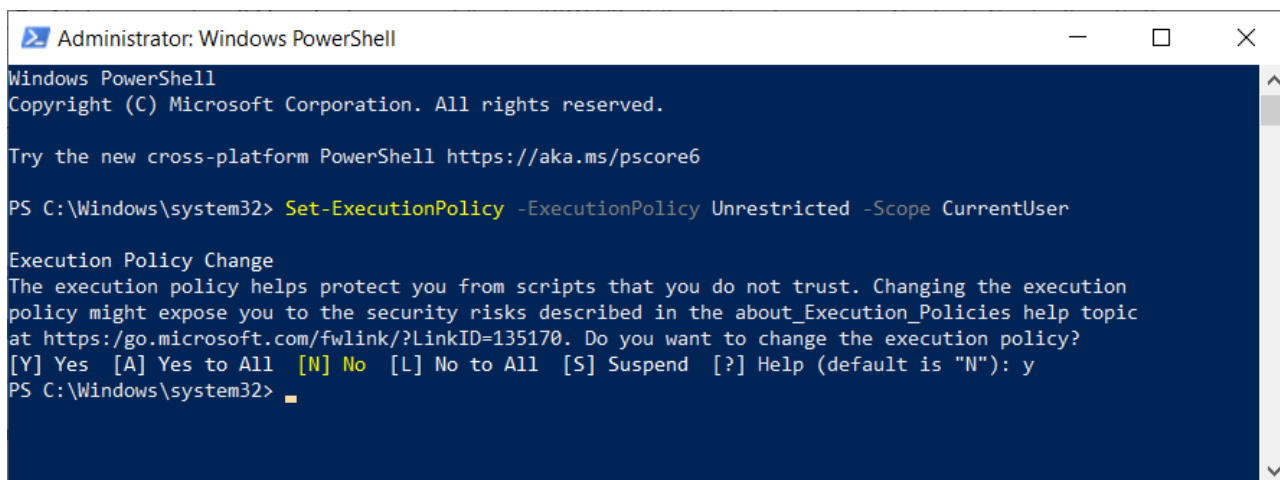
1. Check you have the appropriate permissions to run a *PowerShell* script.
2. Open a *PowerShell* Window and enter *Get-ExecutionPolicy*



3. If it's set to *Unrestricted* or *RemoteSigned* you don't have to do anything further. If it's not, you will need to update the execution policy as follows:
4. Run PowerShell as an Administrator

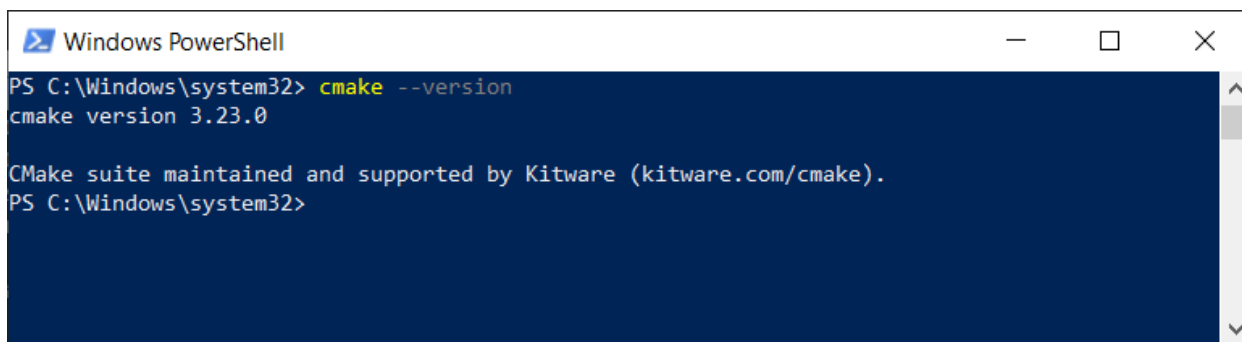


5. Enter *Set-ExecutionPolicy -ExecutionPolicy Unrestricted -Scope CurrentUser*



You can now close the Administrator *PowerShell* Window and proceed as follows.

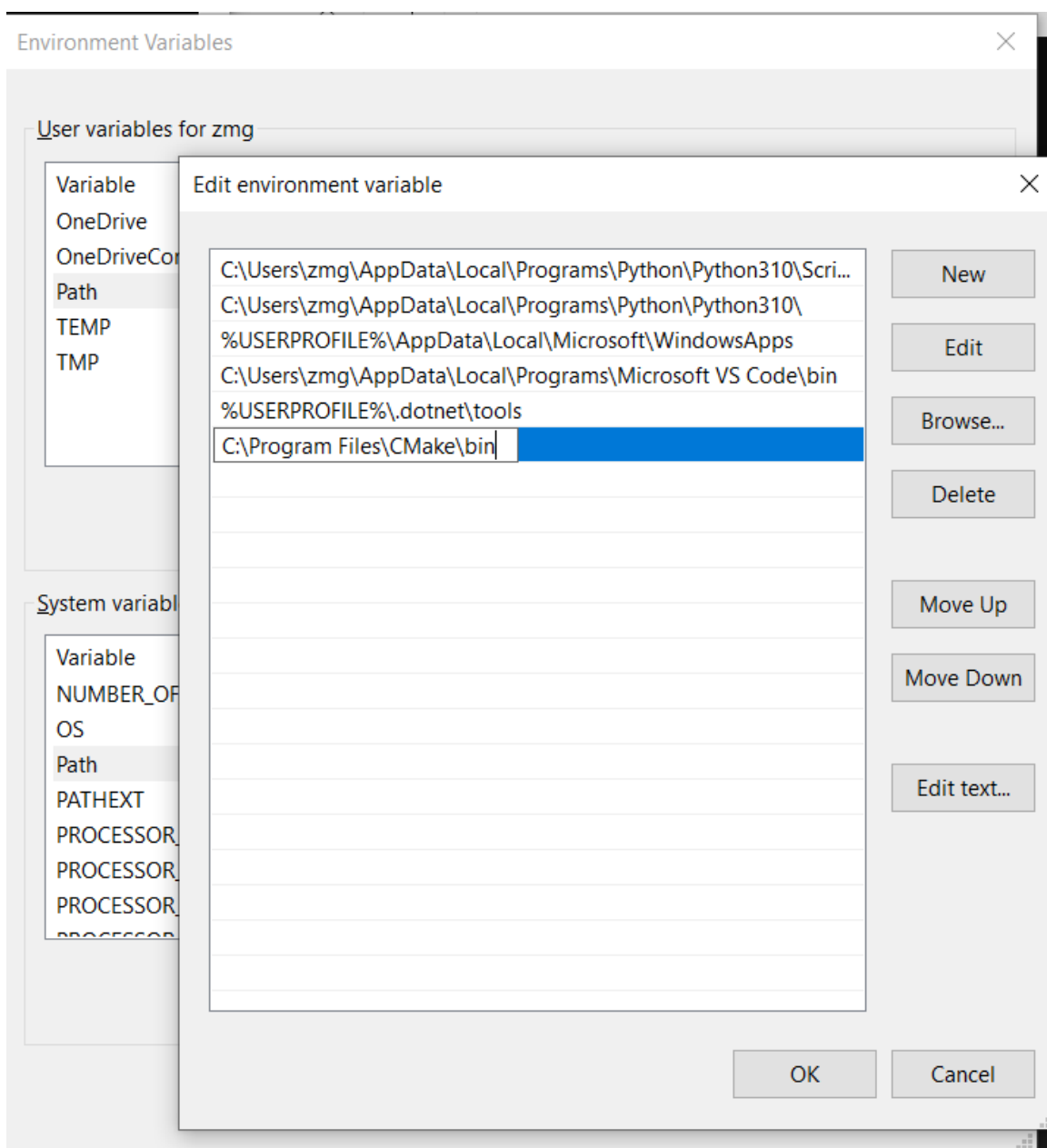
6. Open a *PowerShell* Window.
7. Enter *CMake --version* to check you have *CMake* installed and it can be found.



```
Windows PowerShell
PS C:\Windows\system32> cmake --version
cmake version 3.23.0

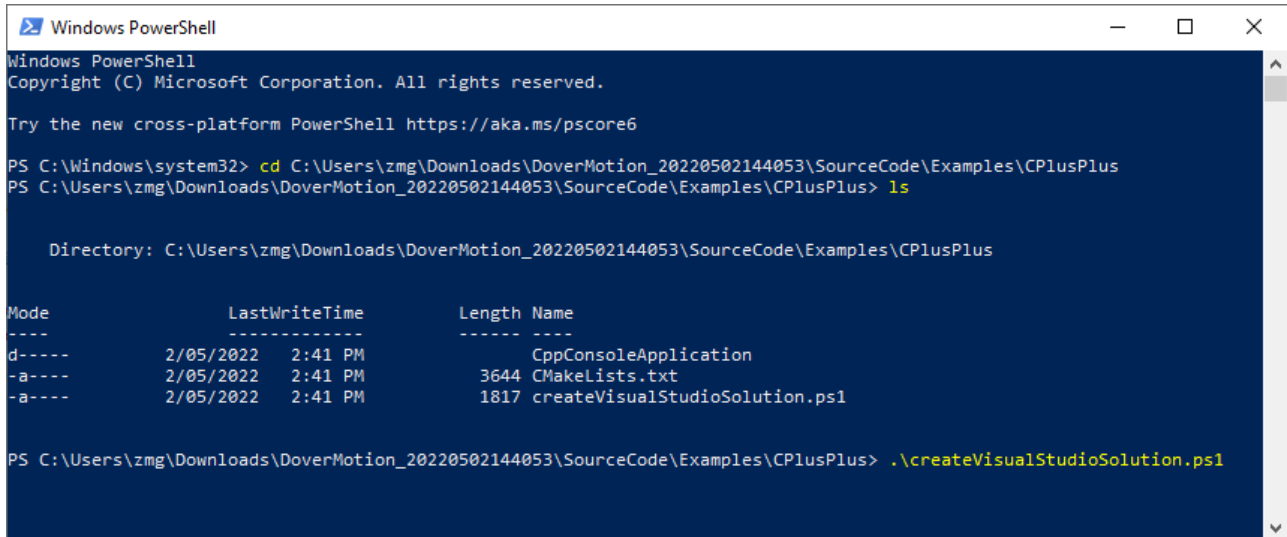
CMake suite maintained and supported by Kitware (kitware.com/cmake).
PS C:\Windows\system32>
```

8. If there is an error indicating it can't find *CMake*, you will need to add a path to CMake in your Environment Variables. Open the Windows *Environment Variables* and add the path *C:\Program Files\CMake\bin* to the *PATH* variable.



You may need to close the PowerShell Window and re-open it for the new path to take effect.

9. Change directory to the `\SourceCode\Examples\CPlusPlus\` folder.
10. Run the `createVisualStudioSolution.ps1` script. It will create a solution for Visual Studio 2022.
Note: You must have access to the internet when running this command as it uses [vcpkg](#) to download 3rd-party libraries (e.g. boost).



```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

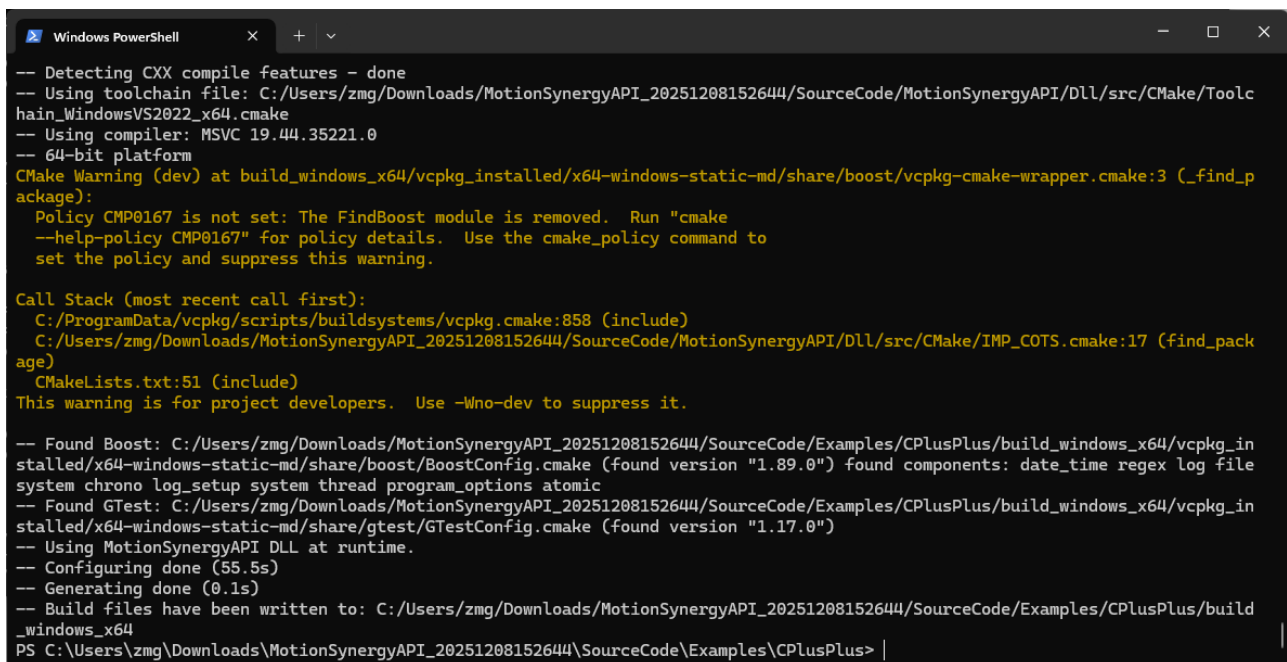
PS C:\Windows\system32> cd C:\Users\zmg\Downloads\DoverMotion_20220502144053\SourceCode\Examples\CPlusPlus
PS C:\Users\zmg\Downloads\DoverMotion_20220502144053\SourceCode\Examples\CPlusPlus> ls

    Directory: C:\Users\zmg\Downloads\DoverMotion_20220502144053\SourceCode\Examples\CPlusPlus

Mode                LastWriteTime         Length Name
----                -
d-----          2/05/2022   2:41 PM             CppConsoleApplication
-a-----          2/05/2022   2:41 PM             3644 CMakeLists.txt
-a-----          2/05/2022   2:41 PM             1817 createVisualStudioSolution.ps1

PS C:\Users\zmg\Downloads\DoverMotion_20220502144053\SourceCode\Examples\CPlusPlus> .\createVisualStudioSolution.ps1
  
```

The output from the command should look similar to the following:



```

-- Detecting CXX compile features - done
-- Using toolchain file: C:/Users/zmg/Downloads/MotionSynergyAPI_20251208152644/SourceCode/MotionSynergyAPI/DLL/src/CMake/Toolchain_WindowsVS2022_x64.cmake
-- Using compiler: MSVC 19.44.35221.0
-- 64-bit platform
CMake Warning (dev) at build_windows_x64/vcpkg_installed/x64-windows-static-md/share/boost/vcpkg-cmake-wrapper.cmake:3 (_find_package):
  Policy CMP0167 is not set: The FindBoost module is removed. Run "cmake --help-policy CMP0167" for policy details. Use the cmake_policy command to set the policy and suppress this warning.

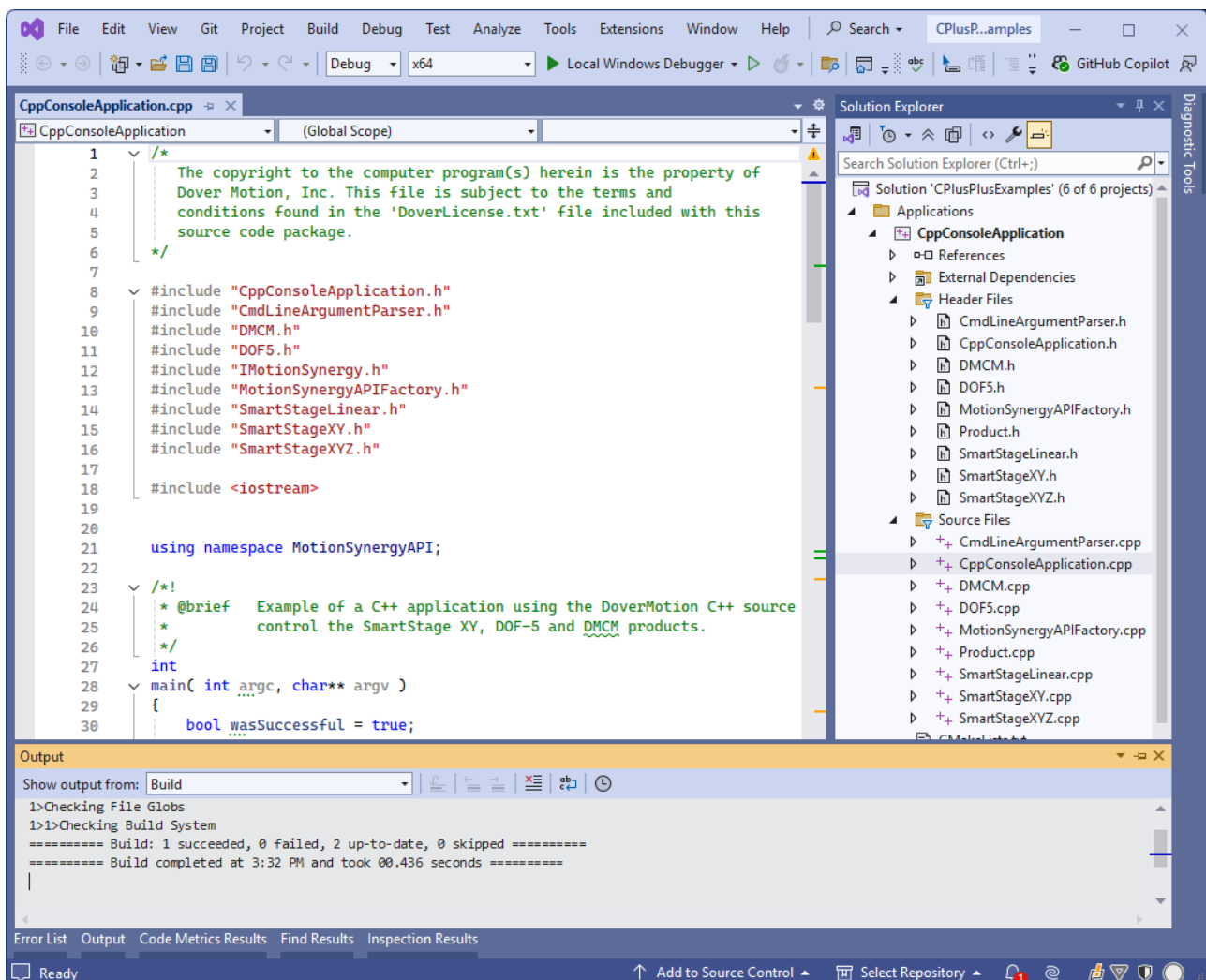
Call Stack (most recent call first):
  C:/ProgramData/vcpkg/scripts/buildsystems/vcpkg.cmake:858 (include)
  C:/Users/zmg/Downloads/MotionSynergyAPI_20251208152644/SourceCode/MotionSynergyAPI/DLL/src/CMake/IMP_COTS.cmake:17 (find_package)
  CMakeLists.txt:51 (include)
This warning is for project developers. Use -Wno-dev to suppress it.

-- Found Boost: C:/Users/zmg/Downloads/MotionSynergyAPI_20251208152644/SourceCode/Examples/CPlusPlus/build_windows_x64/vcpkg_installed/x64-windows-static-md/share/boost/BoostConfig.cmake (found version "1.89.0") found components: date_time regex log file system chrono log_setup system thread program_options atomic
-- Found GTest: C:/Users/zmg/Downloads/MotionSynergyAPI_20251208152644/SourceCode/Examples/CPlusPlus/build_windows_x64/vcpkg_installed/x64-windows-static-md/share/gtest/GTestConfig.cmake (found version "1.17.0")
-- Using MotionSynergyAPI DLL at runtime.
-- Configuring done (55.5s)
-- Generating done (0.1s)
-- Build files have been written to: C:/Users/zmg/Downloads/MotionSynergyAPI_20251208152644/SourceCode/Examples/CPlusPlus/build_windows_x64
PS C:\Users\zmg\Downloads\MotionSynergyAPI_20251208152644\SourceCode\Examples\CPlusPlus> |
  
```

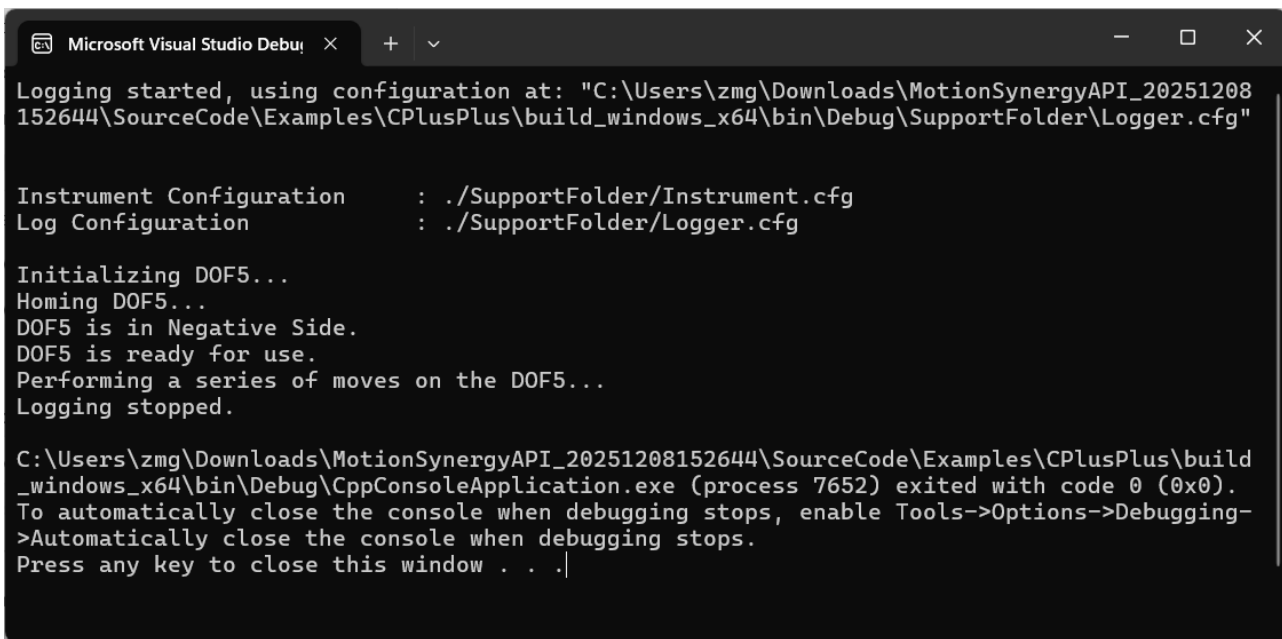
The Visual Studio solution is called `CPlusPlusExamples.sln` and is output to the `build_windows_x64` folder:

SourceCode > Examples > CPlusPlus > build_windows_x64			
Name	Date modified	Type	S
CCP	2/05/2022 4:30 PM	File folder	
CMakeFiles	2/05/2022 4:30 PM	File folder	
CppConsoleApplication	2/05/2022 4:30 PM	File folder	
ALL_BUILD.vcxproj	2/05/2022 4:30 PM	VC++ Project	
ALL_BUILD.vcxproj.filters	2/05/2022 4:30 PM	VC++ Project Filters File	
cmake_install.cmake	2/05/2022 4:30 PM	CMake Source File	
CMakeCache.txt	2/05/2022 4:30 PM	TXT File	
CPlusPlusExamples.sln	2/05/2022 4:30 PM	Visual Studio Solution	
INSTALL.vcxproj	2/05/2022 4:30 PM	VC++ Project	
INSTALL.vcxproj.filters	2/05/2022 4:30 PM	VC++ Project Filters File	
ZERO_CHECK.vcxproj	2/05/2022 4:30 PM	VC++ Project	
ZERO_CHECK.vcxproj.filters	2/05/2022 4:30 PM	VC++ Project Filters File	

You can now load the solution with Visual Studio and build the solution. Once the build is complete you will see output similar to the following:



When you run the application, you should see it home the hardware and perform a series of moves. Diagnostic logging is output to a console window:



```
Microsoft Visual Studio Debug Console
Logging started, using configuration at: "C:\Users\zmg\Downloads\MotionSynergyAPI_20251208152644\SourceCode\Examples\CPlusPlus\build_windows_x64\bin\Debug\SupportFolder\Logger.cfg"

Instrument Configuration      : ./SupportFolder/Instrument.cfg
Log Configuration           : ./SupportFolder/Logger.cfg

Initializing DOF5...
Homing DOF5...
DOF5 is in Negative Side.
DOF5 is ready for use.
Performing a series of moves on the DOF5...
Logging stopped.

C:\Users\zmg\Downloads\MotionSynergyAPI_20251208152644\SourceCode\Examples\CPlusPlus\build_windows_x64\bin\Debug\CppConsoleApplication.exe (process 7652) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Details of the Application behavior are documented in the source code itself.

6.1.1.1 Building the MotionSynergyAPI from source code

By default, the example application builds and links to the prebuilt *MotionSynergyAPINative.dll*. If you need to integrate the MotionSynergyAPI directly from source code, you must first request a source code release from Dover Motion. Once you have the source code, you can build MotionSynergyAPI in two ways:

Including MotionSynergyAPI into a single solution

In this mode the example application builds and links the MotionSynergyAPI libraries as a single solution.

1. Run the *createVisualStudioSolution.ps1* script and pass it a *-apiMode source* argument. e.g.
`.\createVisualStudioSolution.ps1 -apiMode source`


```

Windows PowerShell
-- Detecting CXX compile features - done
-- Using toolchain file: C:/Users/zmg/Downloads/MotionSynergyAPI_SourceCode_20251208152644/SourceCode/MotionSynergyAPI/Impl/native/CMake/Toolchain_WindowsVS2022_x64.cmake
-- Using compiler: MSVC 19.44.35221.0
-- 64-bit platform
CMake Warning (dev) at build_windows_x64/vcpkg_installed/x64-windows-static-md/share/boost/vcpkg-cmake-wrapper.cmake:3 (_find_package):
  Policy CMP0167 is not set: The FindBoost module is removed. Run "cmake --help-policy CMP0167" for policy details. Use the cmake_policy command to set the policy and suppress this warning.

Call Stack (most recent call first):
  C:/ProgramData/vcpkg/scripts/buildsystems/vcpkg.cmake:858 (include)
  C:/Users/zmg/Downloads/MotionSynergyAPI_SourceCode_20251208152644/SourceCode/MotionSynergyAPI/Impl/native/CMake/IMP_COTS.cmake:17 (find_package)
  CMakeLists.txt:51 (include)
This warning is for project developers. Use -Wno-dev to suppress it.

-- Found Boost: C:/Users/zmg/Downloads/MotionSynergyAPI_SourceCode_20251208152644/SourceCode/Examples/CPlusPlus/build_windows_x64/vcpkg_installed/x64-windows-static-md/share/boost/BoostConfig.cmake (found version "1.89.0") found components: date_time regex log filesystem chrono log_setup system thread program_options atomic
-- Found GTest: C:/Users/zmg/Downloads/MotionSynergyAPI_SourceCode_20251208152644/SourceCode/Examples/CPlusPlus/build_windows_x64/vcpkg_installed/x64-windows-static-md/share/gtest/GTestConfig.cmake (found version "1.17.0")
-- Building MotionSynergyAPI from source.
-- Configuring done (63.7s)
-- Generating done (0.5s)
-- Build files have been written to: C:/Users/zmg/Downloads/MotionSynergyAPI_SourceCode_20251208152644/SourceCode/Examples/CPlusPlus/build_windows_x64
PS C:\Users\zmg\Downloads\MotionSynergyAPI_SourceCode_20251208152644\SourceCode\Examples\CPlusPlus>

```

Using MotionSynergyAPI as an SDK

It's also possible to pre-build the MotionSynergyAPI as a separate set of libraries and header files and then link to these directly from your application. This may be more convenient as you no longer need to rebuild the MotionSynergyAPI libraries every time you rebuild your application and you do not need to mix in MotionSynergyAPI projects with your application projects.

To use MotionSynergyAPI as an SDK:

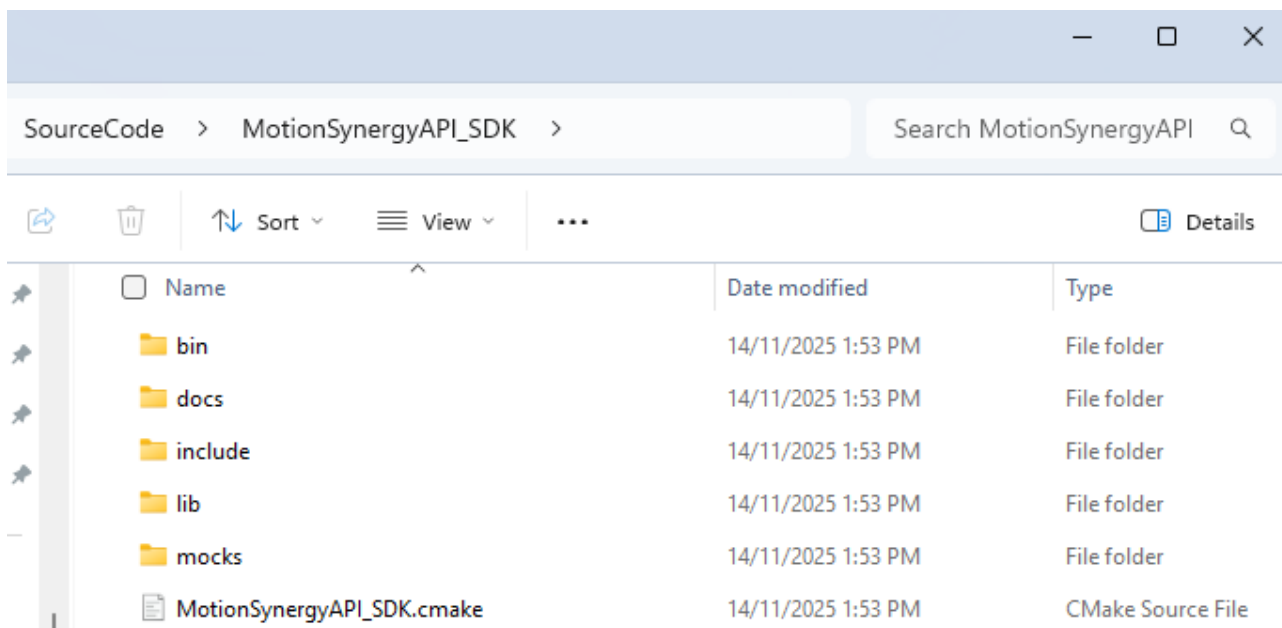
1. Create a MotionSynergyAPI standalone solution.
2. Build and install the MotionSynergyAPI solution. This will create a *MotionSynergyAPI_SDK* folder in *\SourceCode*
3. You can now configure and build the example application using this SDK.

Create a MotionSynergyAPI standalone solution

1. Change directory to the *\SourceCode\MotionSynergyAPI* folder.
2. Run the *createVisualStudioSolution.ps1* script. It will create a solution for Visual Studio 2022. Note: You must have access to the internet when running this command as it uses [vcpkg](#) to download 3rd-party libraries (e.g. boost).

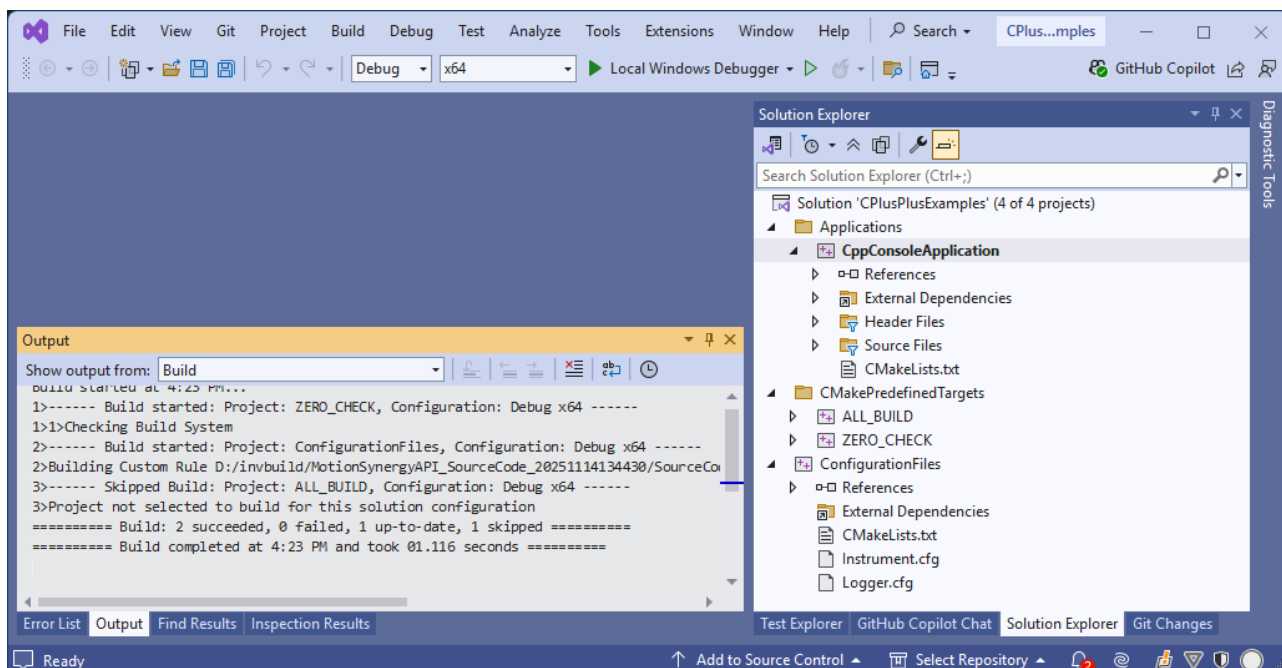
Build and install the MotionSynergyAPI solution

1. Run the *buildAndInstallSdk.ps1* script. This will build both Debug and Release builds of the SDK and then run a CMake 'install' to copy the library files and headers to the *\SourceCode\MotionSynergyAPI_SDK* folder. The contents should look similar to the following:



Configure and build the example application using the SDK

1. You can now configure the example application to link against this SDK. Change to the `\SourceCode\Examples\CPlusPlus\` folder.
2. Run the `createVisualStudioSolution.ps1` script and pass it a `-apiMode sdk` argument. e.g. `.\createVisualStudioSolution.ps1 -apiMode sdk`
3. You can now load the solution with Visual Studio and build the solution. The solution should look similar to the following:



6.1.2 Linux

The example application has been tested on Ubuntu 22.04 LTS. Before building the C++ application you will need to install the following packages.

CMake 3.25 (or above)

```
sudo snap install cmake --classic
```

Python and Pip

```
sudo apt update
sudo apt install python-is-python3
sudo apt install pip
```

Optional - Install ARM64 cross compiler (e.g. Raspberry Pi builds)

```
sudo apt install gcc-aarch64-linux-gnu g++-aarch64-linux-gnu binutils-aarch64-linux-gnu
```

To build the example C++ Console Application, enter the following commands:

```
cd SourceCode/Examples/CPlusPlus/
./createLinux.sh
./buildLinux.sh
```

To run the example C++ Console Application once it's been built, enter the following commands:

```
cd build_linux_x64_debug/bin
./CppConsoleApplication
```

```
root@INVMEILLT0353:~/test2/SourceCode/Examples/CPlusPlus/build_linux_x64_debug/bin# ./CppConsoleApplication
Logging started, using configuration at: "/root/test2/SourceCode/Examples/CPlusPlus/build_linux_x64_debug/bin/SupportFolder/Logger.cfg"

Instrument Configuration      : ./SupportFolder/Instrument.cfg
Log Configuration           : ./SupportFolder/Logger.cfg

Initializing SmartStageXY ...
Homing SmartStageXY ...
SmartStageXY is ready for use.
Performing a series of moves on the SmartStageXY...
Logging stopped.
root@INVMEILLT0353:~/test2/SourceCode/Examples/CPlusPlus/build_linux_x64_debug/bin#
```

To build the example C++ Console Application for the ARM64 platform, specify the platform argument when creating and building the application:

```
cd SourceCode/Examples/CPlusPlus/
./createLinux.sh --platform arm64
./buildLinux.sh --platform arm64
```

```
[ 99%] Building CXX object CppConsoleApplication/CMakeFiles/CppConsoleApplication.dir/src/Product.cpp.o
[100%] Building CXX object CppConsoleApplication/CMakeFiles/CppConsoleApplication.dir/src/SmartStageXY.cpp.o
[100%] Building CXX object CppConsoleApplication/CMakeFiles/CppConsoleApplication.dir/src/SmartStageLinear.cpp.o
[100%] Linking CXX executable ../bin/CppConsoleApplication
/usr/lib/gcc-cross/aarch64-linux-gnu/11/../../../../aarch64-linux-gnu/bin/ld: missing --end-group; added as last command line option
[100%] Built target CppConsoleApplication
~/test2/SourceCode/Examples/CPlusPlus
root@INVMEILLT0353:~/test2/SourceCode/Examples/CPlusPlus# ls -l build_linux_arm64_debug/bin/
total 323624
-rwxr-xr-x 1 root root 62704784 Sep 20 09:33 CppConsoleApplication
drwxr-xr-x 2 root root 4096 Sep 20 09:31 SupportFolder
-rw-r--r-- 1 root root 12796118 Sep 20 09:32 libAlerts.a
-rw-r--r-- 1 root root 12299120 Sep 20 09:32 libAlertsTest.a
```

6.1.2.1 Building the MotionSynergyAPI from source code

By default, the example application builds and links to the prebuilt *libMotionSynergyAPINative.so*. If you need to integrate the MotionSynergyAPI directly from source code, you must first request a source code release from Dover Motion. Once you have the source code, you can build MotionSynergyAPI in two ways:

Including MotionSynergyAPI into a single make file

In this mode the example application builds and links the MotionSynergyAPI libraries as a single make file.

1. Change to the `\SourceCode\Examples\CPlusPlus\` folder.
2. Run the `createLinux.sh` script and pass it a `--source` argument. e.g.
`./createLinux.sh --source --config Debug`
3. Build the application.
`./buildLinux.sh --config Debug`
4. You can also create and build the Release configuration as follows:

```
./createLinux.sh --source --config Release
./buildLinux.sh --config Release
```

Using MotionSynergyAPI as an SDK on Linux

It's also possible to pre-build the MotionSynergyAPI as a separate set of libraries and header files and then link to these directly from your application.

To use MotionSynergyAPI as an SDK on Linux:

1. Create and build the MotionSynergyAPI for Debug and Release configuration.
2. Run CMake install. This will create a `MotionSynergyAPI_SDK` folder in `\SourceCode`
3. You can now configure and build the example application to use this SDK.

Create a MotionSynergyAPI standalone solution

1. Change directory to the `\SourceCode\MotionSynergyAPI\` folder.
2. Run the following commands to create and build Debug and Release configurations. Note: You must have access to the internet when running this command as it uses [vcpkg](#) to download 3rd-party libraries (e.g. boost).

```
./createLinux.sh --config Debug
./buildLinux.sh --config Debug
./createLinux.sh --config Release
./buildLinux.sh --config Release
```

Install the MotionSynergyAPI solution

- Run the following CMake 'install' commands to copy the library files and headers to the `\SourceCode\MotionSynergyAPI_SDK\` folder.

```
cmake --install ../build_linux_x64_debug --config Debug
cmake --install ../build_linux_x64_release --config Release
```

Configure and build the example application using the SDK

1. You can now configure the example application to link against this SDK. Change to the `\SourceCode\Examples\CPlusPlus\` folder.
2. Run the `createLinux.sh` script and pass it a `--sdk` argument. e.g.
`./createLinux.sh --sdk --config Debug`
3. Build the application.
`./buildLinux.sh --config Debug`
4. You can also create and build the Release configuration as follows:

```
./createLinux.sh --sdk --config Release
./buildLinux.sh --config Release
```

6.2 Public Interface

6.2.1 **imp::CommunicationSettings Class**

```
#include <CommunicationSettings.h>
```

```
class CommunicationSettings
```

Defines the Serial and CAN communication settings for an axis.

6.2.1.1 Public Methods

6.2.1.1.1 ToString()

```
std::string ToString() const
```

6.2.1.2 Public Variables

6.2.1.2.1 CanAddress

```
uint16_t CanAddress
```

CAN Node ID address (0-127).

6.2.1.2.2 CanPortBaudRate

```
CanBaudRate CanPortBaudRate
```

Baud rate to use for CAN mode (0-7).

6.2.1.2.3 SerialAddress

```
uint16_t SerialAddress
```

Multi-drop address for use with RS485 (0-31).

6.2.1.2.4 SerialPortBaudRate

```
SerialBaudRate SerialPortBaudRate
```

Baud rate to use for serial mode (1,200-460,800)

6.2.1.2.5 SerialPortProtocol

```
SerialProtocol SerialPortProtocol
```

Protocol to use for serial mode.

6.2.2 **imp::IAxisDevice Class**

```
#include <IAxisDevice.h>
```

```
class IAxisDevice : public virtual imp::ICCPDevice
```

Shorthand for creating shared pointers of IAxisDevice.

Details

Defines the interface an Axis must adhere to for building motion control systems within CCP

6.2.2.1 Public Methods

6.2.2.1.1 Disable()

```
virtual void Disable()=0
```

Disable / de-energize the axis motor.

Details

This will remove power from the axis so it can be moved manually. If an encoder is present, it will continue to update as the axis is manually moved.

6.2.2.1.2 DisableTriggerOnPosition()

```
virtual void DisableTriggerOnPosition()=0
```

Disable trigger on position (TOP).

6.2.2.1.3 Enable()

```
virtual void Enable()=0
```

Enable / energize the axis motor.

Details

This will restore power to the axis motor (after Disable() has been called) so it can be commanded to move.

6.2.2.1.4 EnableTraceCapture()

```
virtual void EnableTraceCapture(TraceTrigger trigger,
                                uint16_t period,
                                const std::vector< TraceVariable > &variables,
                                uint64_t waitMotionStabilizeUs=0)=0
```

Enable capture of trace data.

Details

If the trigger is OnMotionStart, trace capture will be automatically started when the next move begins and stopped once the move completes. If the trigger is OnMotionStartNoEnd, trace capture will be automatically started when the next move begins and must be stopped by calling StopTraceCapture(). If the trigger is Immediate, trace capture will be started immediately and must be stopped by calling StopTraceCapture().

Parameters:

[in] trigger	Trigger to start the trace capture (e.g. Immediate or OnMotionStart).
[in] period (e.g. 1 = every cycle).	Period at which the data is sampled in terms of the motion chip cycles
[in] variables	A list of variables to capture. Up to 4 variables can be captured.
[in] waitMotionStabilizeUs	Time in us to wait for motion to stabilize. Only applies when the trigger is OnMotionStartDelayed.

6.2.2.1.5 EnableTriggerOnPosition()

```
virtual void EnableTriggerOnPosition(double startPosition,
                                      double endPosition,
                                      double increment,
                                      bool isBidirectional,
                                      int pulseWidthNs)=0
```

Enable trigger on position (TOP). TOP will generate a GPIO pulse every 'increment' of travel while in the window specified by 'startPosition' and 'endPosition'.

Details

When using this feature with a SmartStageXY or SmartStageLinear, the GpioConfiguration in Instrument.cfg must be set to PMDGpioConfiguration::S2 (the default).

Parameters:

- [in] startPosition Position at which to start generating pulses in selected units (mm, um, or nm).
- [in] endPosition Position at which to stop generating pulses in selected units (mm, um, or nm).
- [in] increment Generate a pulse every 'increment' of travel in selected units (mm, um, or nm).
- [in] isBidirectional If true, pulses will also be generated when moving from endPosition back to startPosition. If false, pulses will only be generated when moving from startPosition to endPosition.
- [in] pulseWidthNs The width of the pulse to generate at each position in nanoseconds. The actual pulse width is a value rounded to the nearest practicable value, which must be a multiple of 50ns and less or equal to 204,800 ns. It's ignored by version 1.2 and lower of the PMD motion control firmware.

6.2.2.1.6 EnterHomingMode()

```
virtual void EnterHomingMode()=0
```

Set settings necessary before homing routine starts. Any settings replaced can be restored with ExitHomingMode().

Details

Homing mode will save the MotionCompleteMode in buffer memory and set it to CommandedPositionReached. Initialization should be complete before calling this method, as it resets the homing mode.

Exceptions:

std::runtime_error if homing mode is already active.

6.2.2.1.7 ExitHomingMode()

```
virtual void ExitHomingMode()=0
```

Restore settings changed by EnterHomingMode() and exit homing mode.

Exceptions:

std::runtime_error if homing mode is not active.

6.2.2.1.8 GetAcceleration()

```
virtual double GetAcceleration(bool forceRefresh=true)=0
```

Get the maximum acceleration of the axis.

Parameters:

- [in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

Maximum acceleration in selected units (mm/s², um/s², or nm/s²).

6.2.2.1.9 GetActiveControlMode()

```
virtual ActiveControlMode GetActiveControlMode()=0
```

Get the active control mode of the axis.

Returns:

Active control mode of the axis (Normal, UDPTIME, UDPSlave, RCP, or ExternalControl).

6.2.2.1.10 GetActualPosition()

```
virtual double GetActualPosition(bool forceRefresh=true)=0
```

Get the actual position of the axis as reported by the encoder.

Details

On successful completion of a move, the actual position will match the commanded position (within the axis resolution). If the move failed (e.g. a position error occurred), the actual position will not match the commanded position. The commanded position will be set to the actual position when: A move is interrupted due to a Stop or EStop command being issued. While the axis is disabled and when the Enable command is issued to transition the axis from the disabled to enabled state.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

Encoder position in selected units (mm, um, or nm).

6.2.2.1.11 GetCommandedPosition()

```
virtual double GetCommandedPosition(bool forceRefresh=true)=0
```

Get the position the axis was commanded to move to.

Details

On successful completion of a move, the actual position will match the commanded position (within the axis resolution). If the move failed (e.g. a position error occurred), the actual position will not match the commanded position. The commanded position will be set to the actual position when: A move is interrupted due to a Stop or EStop command being issued. While the axis is disabled and when the Enable command is issued to transition the axis from the disabled to enabled state.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

Commanded position in selected units (mm, um, or nm).

6.2.2.1.12 GetDeceleration()

```
virtual double GetDeceleration(bool forceRefresh=true)=0
```

Get the maximum deceleration of the axis.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

Maximum deceleration in selected units (mm/s², um/s², or nm/s²).

6.2.2.1.13 GetDigitalOutputTrigger()

```
virtual DigitalOutputTrigger GetDigitalOutputTrigger(bool forceRefresh=true)=0
```

Get the digital output trigger in effect.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

The DigitalOutputTrigger currently in effect (see enum DigitalOutputTrigger for details).

6.2.2.1.14 GetExternalControl()

```
virtual double GetExternalControl(bool forceRefresh=true)=0
```

Get the currently configured move distance per pulse.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

moveDistancePerPulse in selected units (mm, um, or nm). Zero indicates external control is disabled, a non-zero value indicates external control is enabled.

6.2.2.1.15 GetFlagStatus()

```
virtual bool GetFlagStatus(const std::string &flagName,
                           bool forceRefresh=true)=0
```

Returns the activation state of the given named flag.

Parameters:

[in] flagName Named flag to query on

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

The activation state of the flag

6.2.2.1.16 GetIsEnabled()

```
virtual bool GetIsEnabled(bool forceRefresh=true)=0
```

Get the enabled (energized) / disabled (de-energized) state the axis motor.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

true if enabled (energized), false if disabled (de-energized).

6.2.2.1.17 GetIsTraceCaptureEnabled()

```
virtual bool GetIsTraceCaptureEnabled(bool forceRefresh=true)=0
```

Is trace capture enabled for the next axis move? Does not detect immediate trace capture.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

true if trace capture is enabled for the next axis move, false if not enabled for the next axis move or was enabled using the Immediate trigger.

6.2.2.1.18 GetIsTriggerOnPositionEnabled()

```
virtual bool GetIsTriggerOnPositionEnabled(bool forceRefresh=true)=0
```

Get the state of the trigger on position (TOP) function.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

true if enabled, false if disabled.

6.2.2.1.19 GetJerk()

```
virtual double GetJerk(bool forceRefresh=true)=0
```

Get the maximum jerk of the axis.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

Maximum jerk in selected units (mm/s^3, um/s^3, or nm/s^3).

6.2.2.1.20 GetMotionControllerConfiguration()

```
virtual IMotionControllerConfiguration& GetMotionControllerConfiguration()=0
```

Get an interface for reading and writing various motion controller settings from/to memory/NVRAM.

Returns:

The above interface.

6.2.2.1.21 GetMotorCurrent()

```
virtual double GetMotorCurrent(bool forceRefresh=true)=0
```

Get the RMS current being used to drive the motor at this instant.

Details

Due to communication latency with the motor drive chip, the value returned will be lagging.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

Motor current in Amps.

6.2.2.1.22 GetNamedDistances()

```
virtual NamedDistancesMap GetNamedDistances() const =0
```

Returns a complete list of named distances for this axis.

Returns:

A map of named distances (distanceName, distance) for this axis. Returns an empty map if this axis does not have any named distances.

6.2.2.1.23 GetNamedFlags()

```
virtual NamedFlagsMap GetNamedFlags() const =0
```

Returns a complete list of named flags for this axis.

Returns:

A map of named flags (flagName, index) for this axis. Returns an empty map if this axis does not have any named flags.

6.2.2.1.24 GetNamedMoveProfiles()

```
virtual NamedMoveProfilesMap GetNamedMoveProfiles() const =0
```

Returns a complete list of named move profiles for this axis.

Returns:

A map of named move profiles (moveProfileName, (parameterName, parameterValue)) for this axis. Returns an empty map if this axis does not have any named move profiles.

6.2.2.1.25 GetNamedPositions()

```
virtual NamedPositionsMap GetNamedPositions() const =0
```

Returns a complete list of named positions for this axis.

Returns:

A map of named positions (positionName, position) for this axis. Returns an empty map if this axis does not have any named positions.

6.2.2.1.26 GetPositionError()

```
virtual double GetPositionError(bool forceRefresh=true)=0
```

Get the current position error of the axis.

Details

Position error is the difference between the commanded position and the actual position of the axis, in the selected units (mm, um, or nm). A non-zero value indicates that the axis did not reach the commanded position. This can occur due to overshoot, mechanical limits, or if a move was interrupted. The position error is useful for monitoring motion accuracy and detecting issues during moves.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to hardware and wait for value (blocking).

Returns:

Position error in selected units (mm, um, or nm).

6.2.2.1.27 GetRecoveryOperation()

```
virtual AxisRecoveryOperation GetRecoveryOperation(bool forceRefresh=true)=0
```

Get the recovery operation (if any) required to return the axis to a state where it can successfully perform moves.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware (to get the current enabled/disable state of the motor). false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

An enum indicating the type of recovery operation required.

6.2.2.1.28 GetUserDefinedProfile()

```
virtual IUserDefinedProfile& GetUserDefinedProfile()=0
```

Get an interface for performing User Defined Profile (UDP) moves and updating the associated user defined profile data.

Returns:

The above interface.

6.2.2.1.29 GetVelocity()

```
virtual double GetVelocity(bool forceRefresh=true)=0
```

Set the maximum velocity of the axis.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

Maximum velocity in selected units (mm/s, um/s, or nm/s).

6.2.2.1.30 MoveAbsolute()

```
virtual void MoveAbsolute(double position)=0
```

Move axis to absolute position.

Details

Moves axis to the specified absolute position.

Parameters:

[in] position Value of the absolute coordinate to move to in selected units (mm, um, or nm).

6.2.2.1.31 MoveContinuous()

```
virtual void MoveContinuous(double position,
                             double acceleration,
                             double velocity,
                             double deceleration)=0
```

Moves axis to the specified absolute position in selected units (mm, um, or nm). Unlike other move commands, this command can be issued while there is a move in progress.

Details

It uses the supplied trajectory parameters (velocity, acceleration, deceleration), not the currently selected move profile. The trajectory parameters can also be modified during a move. Note that only trapezoidal moves are supported for continuous moves, not s-curve moves, hence the lack of a jerk parameter.

Parameters:

- [in] position Value of the absolute coordinate to move to in selected units (mm, um, or nm).
- [in] acceleration Maximum acceleration to be set in selected units (mm/s², um/s², or nm/s²).
- [in] velocity Maximum velocity to be set in selected units (mm/s, um/s, or nm/s).
- [in] deceleration Maximum deceleration to be set in selected units (mm/s², um/s², or nm/s²).

6.2.2.1.32 MoveRelative()

```
virtual void MoveRelative(double distance)=0
```

Move axis to relative position.

Details

Moves axis from the current position by the specified value.

Parameters:

- [in] distance Amount by which to move from the current position in selected units (mm, um, or nm).

6.2.2.1.33 MoveRelativeToFlag()

```
virtual void MoveRelativeToFlag(double maximumDistance,
                                const std::string &flagName)=0
```

Move axis to the given named flag searching up to a maximum distance.

Details

Moves axis to the flag if it can be detected, or the maximum distance which would raise an exception. If the flag is reached, the axis will be brought to an abrupt stop.

Parameters:

- [in] maximumDistance Maximum distance to search for the flag in selected units (mm, um, or nm)
- [in] flagName Named flag to search for

6.2.2.1.34 MoveRelativeToFlagOpenLoop()

```
virtual void MoveRelativeToFlagOpenLoop(const std::string &flagName,
                                         double offsetFromFlag)=0
```

Move axis to the given named flag in open loop mode.

Details

This command can only be used prior to initialization. It can be used to move an axis to a known location prior to initialization and homing.

Parameters:

- [in] flagName Named flag to move to.

[in] offsetFromFlag An offset (+ve or -ve) in selected units (mm, um, or nm) to move away from the flag once the flag has been reached. e.g. supports backing off the positive limit (use a -ve offset) or negative limit (use a +ve offset).

6.2.2.1.35 MoveRelativeUDP()

```
virtual void MoveRelativeUDP(uint32_t repeat)=0
```

Move axis using the currently selected time-based UDP profile. A time-based UDP profile must have first been selected by calling SelectMoveProfile(). If a time-based UDP profile is not active, this method will return an error.

Details

This method is obsolete. Call the equivalent IUserDefinedProfile.MoveRelativeUDP instead.

Parameters:

[in] repeat Number of times to repeat move.

6.2.2.1.36 NEnableTraceCapture()

```
virtual void NEnableTraceCapture(TraceTrigger trigger,
                                const std::string &traceProfile)=0
```

Enable capture of trace data using a named trace profile defined in configuration.

Details

If the trigger is OnMotionStart, trace capture will be automatically started when the next move begins and stopped once the move completes. If the trigger is OnMotionStartNoEnd, trace capture will be automatically started when the next move begins and must be stopped by calling StopTraceCapture(). If the trigger is Immediate, trace capture will be started immediately and must be stopped by calling StopTraceCapture().

Parameters:

[in] trigger Trigger to start the trace capture (e.g. Immediate or OnMotionStart).

[in] traceProfile The name of the trace profile section defined in configuration. This profile will specify the period and up to 4 trace variables.

6.2.2.1.37 NEnableTriggerOnPosition()

```
virtual void NEnableTriggerOnPosition(const std::string &startPositionName,
                                      const std::string &endPositionName,
                                      const std::string &namedDistance,
                                      bool isBidirectional,
                                      int pulseWidthNs)=0
```

Enable trigger on position (TOP). TOP will generate a GPIO pulse every 'increment' of travel while in the window specified by 'startPosition' and 'endPosition'.

Details

When using this feature with a SmartStageXY or SmartStageLinear, the GpioConfiguration in Instrument.cfg must be set to PMDGpioConfiguration::S2 (the default).

Parameters:

[in] startPositionName Named position at which to start generating pulses.

[in] endPositionName Named position at which to stop generating pulses.

[in] namedDistance Generate a pulse every 'namedDistance' of travel.

[in] isBidirectional If true, pulses will also be generated when moving from endPosition back to

startPosition. If false, pulses will only be generated when moving from startPosition to endPosition.

[in] pulseWidthNs The width of the pulse to generate at each position in nanoseconds. The actual pulse width is a value rounded to the nearest practicable value, which must be a multiple of 50ns and less or equal to 204,800 ns. It's ignored by version 1.2 and lower of the PMD motion control firmware.

6.2.2.1.38 NMoveAbsolute()

```
virtual void NMoveAbsolute(const std::string &position_name)=0
```

Move axis to absolute position by name.

Details

Moves axis to the specified absolute position in um.

Parameters:

[in] position_name Name of the absolute coordinate to move to.

6.2.2.1.39 NMoveOffset()

```
virtual void NMoveOffset(const std::string &position_name,
                        const std::string &distance_name)=0
```

Move axis to a location that is a fixed named distance offset from a named absolute position.

Details

Moves axis to the resultant offset position in um.

Parameters:

[in] position_name Name of the absolute position to base the movement on.

[in] distance_name Name of the relative distance to add on to the absolute position.

6.2.2.1.40 NMoveRelative()

```
virtual void NMoveRelative(const std::string &distance_name)=0
```

Move axis to relative position by name.

Details

Moves axis from the current position by the specified value in um.

Parameters:

[in] distance_name Name of distance by which to move from the current position in um.

6.2.2.1.41 NMoveRelativeToFlag()

```
virtual void NMoveRelativeToFlag(const std::string &maximumDistanceName,
                                const std::string &flagName)=0
```

Move axis to the given named flag searching up to a named maximum distance.

Details

Moves axis to the flag if it can be detected, or the maximum distance which would raise an exception. If the flag is reached, the axis will be brought to an abrupt stop.

Parameters:

[in] maximumDistanceName Name maximum distance to search for the flag

[in] flagName Named flag to search for

6.2.2.1.42 NMoveRelativeToFlagOpenLoop()

```
virtual void NMoveRelativeToFlagOpenLoop(const std::string &flagName,
                                         const std::string &offsetFromFlagDistanceName)=0
```

Move axis to the given named flag in open loop mode.

Details

This command can only be used prior to initialization. It can be used to move an axis to a known location prior to initialization and homing.

Parameters:

[in] flagName Named flag to move to.

[in] offsetFromFlagDistanceName A named distance offset to move away from the flag once the flag has been reached. e.g. supports backing off the positive limit (use a -ve offset) or negative limit (use a +ve offset).

6.2.2.1.43 NResetPosition()

```
virtual void NResetPosition(const std::string &positionName)=0
```

Resets the current (and possibly encoder) position of the motor, to the named value.

Parameters:

[in] positionName New named position of the motor

6.2.2.1.44 ResetPosition()

```
virtual void ResetPosition()=0
```

Resets the commanded position to match the current encoder position.

6.2.2.1.45 ResetPosition()

```
virtual void ResetPosition(double position)=0
```

Resets the current (and possibly encoder) position of the motor, to the specified position.

Parameters:

[in] position New position of the motor in selected units (mm, um, or nm).

6.2.2.1.46 SaveTraceCapture()

```
virtual void SaveTraceCapture(const std::string &pathname,
                             std::int32_t traceCount)=0
```

Save the trace data captured during the last move (see EnableTraceCapture() and NEnableTraceCapture()) as a csv file.

Parameters:

[in] pathname The path to the file to save the data to.

[in] traceCount The maximum number of trace samples to save to the file.

6.2.2.1.47 SelectMoveProfile()

```
virtual void SelectMoveProfile(const std::string &profileName)=0
```

Sets the Acceleration, Deceleration, Velocity and Jerk specified by the named profile. Wait must be called before issuing another command to the device.

Exceptions:

std::runtime_error if the move profile does not exist.

6.2.2.1.48 SetAcceleration()


```
virtual void SetAcceleration(double acceleration)=0
```

Set the maximum acceleration of the axis.

Parameters:

[in] acceleration Maximum acceleration to be set in selected units (mm/s², um/s², or nm/s²).

6.2.2.1.49 SetDeceleration()

```
virtual void SetDeceleration(double deceleration)=0
```

Set the maximum deceleration of the axis.

Parameters:

[in] deceleration Maximum deceleration to be set in selected units (mm/s², um/s², or nm/s²).

6.2.2.1.50 SetDigitalOutputTrigger()

```
virtual void SetDigitalOutputTrigger(DigitalOutputTrigger trigger)=0
```

Enable / disable a digital output (AxisOut1) trigger based on the axis motion.

Details

When using this feature with a SmartStageXY or SmartStageLinear, the GpioConfiguration in Instrument.cfg must be set to PMDGpioConfiguration::S2 (the default).

Parameters:

[in] trigger Specifies a trigger on which to set the digital output (see enum DigitalOutputTrigger for details).

6.2.2.1.51 SetExternalControl()

```
virtual void SetExternalControl(double moveDistancePerPulse)=0
```

Enable / disable control of the axis via an external pulse signal.

Details

When using this feature with a SmartStageXY or SmartStageLinear, the GpioConfiguration in Instrument.cfg must be set to PMDGpioConfiguration::S1.

Parameters:

[in] moveDistancePerPulse Move the axis by this amount in selected units (mm, um, or nm) per positive pulse signal and by the negative amount per negative pulse signal. Setting this value to zero will disable external control.

6.2.2.1.52 SetIdleErrorCallback()

```
virtual void SetIdleErrorCallback(std::function< void(DeviceResponseType)> axisErrorCb)=0
```

Set callback to be invoked when in idle or electronic gearing and an error is observed.

Parameters:

[in] axisErrorCb Callback to be invoked when an error is detected. This will be invoked on a different thread to the main thread.

6.2.2.1.53 SetJerk()

```
virtual void SetJerk(double jerk)=0
```

Set the maximum jerk of the axis.

Parameters:

[in] jerk Maximum jerk to be set in selected units (mm/s³, um/s³, or nm/s³).

6.2.2.1.54 SetNamedDistances()

```
virtual void SetNamedDistances(const NamedDistancesMap &namedDistances,
                               bool doWriteToLocalConfig=true)=0
```

Update the list of named distances for this axis. The list will be persisted to configuration.

Parameters:

[in] namedDistances A map of named distances (distanceName, distance) for this axis.
 [in] doWriteToLocalConfig Whether to write updated values to local config.

6.2.2.1.55 SetNamedMoveProfiles()

```
virtual void SetNamedMoveProfiles(const NamedMoveProfilesMap &namedMoveProfiles,
                                   bool doWriteToLocalConfig=true)=0
```

Update the list of named move profiles for this axis. The list will be persisted to configuration.

Parameters:

[in] namedMoveProfiles A map of named move profiles (moveProfileName, (parameterName, parameterValue)) for this axis.
 [in] doWriteToLocalConfig Whether to write updated values to local config.

6.2.2.1.56 SetNamedPositions()

```
virtual void SetNamedPositions(const NamedPositionsMap &namedPositions,
                               bool doWriteToLocalConfig=true)=0
```

Update the list of named positions for this axis. The list will be persisted to configuration.

Parameters:

[in] namedPositions A map of named positions (positionName, position) for this axis.
 [in] doWriteToLocalConfig Whether to write updated values to local config.

6.2.2.1.57 SetVelocity()

```
virtual void SetVelocity(double velocity)=0
```

Set the maximum velocity of the axis.

Parameters:

[in] velocity Maximum velocity to be set in selected units (mm/s, um/s, or nm/s).

6.2.2.1.58 Stop()

```
virtual void Stop()=0
```

Stop the current movement action in a controlled manner.

Details

Will stop the current profile/velocity move action and return immediately. The original move command will complete once the axis has come to a halt. If there is no movement in progress, the operation will complete without error. Wait() must still be called after issuing a Stop() to wait for the axis to come to a halt. Unlike EStop(), the Stop() command does not require the axis to be re-initialized.

6.2.2.1.59 StopTraceCapture()

```
virtual void StopTraceCapture()=0
```

Stop the capture of trace data. This will stop any trace capture currently in progress.

Details

If the trigger supplied to `EnableTraceCapture()` was `OnMotionStart` or `OnMotionStartNoEnd` and a move has not started, this will cancel the request to capture trace data on the next move.

6.2.2.1.60 `SubscribeToMovementStarted()`

```
virtual SubscriptionId SubscribeToMovementStarted(const std::shared_ptr< IDispatcher > &pDispatcher,
MovementStartedCallback callback)=0
```

Subscribe for callback on start of axis movement.

Parameters:

[in] pDispatcher The dispatcher through which the callback is to be invoked

[in] callback The callback to invoke

Returns:

A unique identifier allowing the subscription to be unsubscribed

6.2.2.1.61 `UnsubscribeFromMovementStarted()`

```
virtual bool UnsubscribeFromMovementStarted(SubscriptionId subscriptionId)=0
```

Unsubscribe for callback on start of axis movement.

Parameters:

[in] subscriptionId The unique identifier of the subscription as returned by `SubscribeToMovementStarted`

Returns:

Whether the subscription with the given ID was successfully unsubscribed

6.2.3 `imp::ICCPDevice` Class

```
#include <ICCPDevice.h>
```

```
class ICCPDevice
```

Defines the interface any device must adhere to for integration with the CCP.

6.2.3.1 Public Methods

6.2.3.1.1 `BeginPreventWait()`

```
virtual void BeginPreventWait()=0
```

Indicates the start of a period where `Wait()` should not block. If a `Wait()` is currently in progress, it will be unblocked and the call will return immediately.

Details

This does not change the state of the device. `Wait()` must be called again (after `FinishPreventWait()`) to ensure the asynchronous action has completed. This method must be called from another thread.

6.2.3.1.2 `Configure()`

```
virtual void Configure(IConfigurationSPtr pConfig)=0
```

Configure the device.

Parameters:

[in] pConfig The instrument configuration used to configure the device.

6.2.3.1.3 Connect()

```
virtual void Connect()=0
```

Establish a connection to the hardware.

Details

Most commands require initialization before it can be properly used. However, a few commands only require an established connection. Calling this is not required if the hardware is already initialized.

6.2.3.1.4 DefaultTimeout()

```
virtual uint32_t DefaultTimeout() const =0
```

Returns the default timeout which can be used in Wait() calls.

Returns:

The default timeout (in milliseconds)

6.2.3.1.5 EStop()

```
virtual void EStop()=0
```

Perform an emergency stop of the device. This will immediately halt any motion (e.g. an abrupt stop will be performed for an axis device) and return the device to a safe, passive state (e.g. for an axis device this means the operating mode is set to disable the axis, motor output, current control, position loop, and trajectory).

Details

The next call to Wait() on this device will throw an exception, indicating the device has been EStopped and the command in progress was interrupted. Alternatively, EStopWait() can be called to wait for the estop to complete without throwing an exception. Initialize() must be called on this device before it will accept further commands.

6.2.3.1.6 EStopWait()

```
virtual void EStopWait()=0
```

Wait for up to the default timeout for the previous EStop to complete.

Details

Waits up to the default timeout for any active asynchronous action to complete. Does not throw an exception if the EStop was successful.

Exceptions:

DeviceException if;

- The EStop action itself fails (An action prematurely terminated due to an EStop will not result in an exception)
- The default timeout is reached

6.2.3.1.7 EStopWait()

```
virtual void EStopWait(uint32_t timeout_ms)=0
```

Wait for the previous EStop to complete.

Details

Waits for any active asynchronous action to complete. Does not throw an exception if the EStop was successful.

Parameters:

[in] timeout_ms The length of time to wait before timing out.

Exceptions:

DeviceException if;

- The EStop action itself fails (An action prematurely terminated due to an EStop will not result in an exception)
- Timeout is reached
- timeout_ms is > the maximum action timeout (default of 2 minutes)

6.2.3.1.8 FinishPreventWait()

```
virtual void FinishPreventWait()=0
```

Indicates the completion of a period where Wait() should not block. The next call to Wait() will block until the outstanding asynchronous action has completed.

Details

This method can be called from any thread.

6.2.3.1.9 GetIsConnected()

```
virtual bool GetIsConnected(bool forceRefresh=true)=0
```

Gets the connected state of the device.

Parameters:

[in] forceRefresh Force the device to do a deep call to hardware. false = Return locally cached value. true = Send request to h/w and wait for value (blocking).

Returns:

True if the device is successfully connected to hardware, false if not.

6.2.3.1.10 HaveAttemptedConnect()

```
virtual bool HaveAttemptedConnect() const =0
```

Was a connection to the device attempted?

Returns:

True if an attempt was made to connect to the hardware, false if not.

6.2.3.1.11 Initialize()

```
virtual void Initialize()=0
```

Initialize the device. A device must be initialized before most other commands can be performed.

Details

In the case of an axis this involves:

- Establishing a connection to the hardware.
- Performing Analog Calibration.
- Performing Commutation.

6.2.3.1.12 IsActionInProgress()

```
virtual bool IsActionInProgress()=0
```

Gets the action progress state of the device.

Returns:

True if an action is in progress.

6.2.3.1.13 MaxTimeout()

```
virtual uint32_t MaxTimeout() const =0
```

Returns the maximum timeout which can be used in Wait() calls.

Returns:

The maximum timeout (in milliseconds)

6.2.3.1.14 Name()

```
virtual std::string Name() const =0
```

Retrieve the name of the device.

Returns:

The name of the device.

6.2.3.1.15 Shutdown()

```
virtual void Shutdown()=0
```

Shutdown the device in preparation for power off. Once a device has been shutdown, it will accept no further commands.

Details

If there is a connection associated with this device and this is the last device using the connection, the connection will be closed.

6.2.3.1.16 SubscribeCompletionCallback()

```
virtual SubscriptionId SubscribeCompletionCallback(const IDispatcherSPtr &pDispatcher,
                                                    CompletionCallback callback)=0
```

Subscribes for callback upon asynchronous action completion.

Parameters:

[in] pDispatcher Pointer to the dispatcher the callback is to be invoked on.

[in] callback The callback to be invoked upon asynchronous action completion.

Exceptions:

Exception if;

- pDispatcher is nullptr

6.2.3.1.17 Type()

```
virtual std::string Type() const =0
```

The type of device.

Returns:

The type of device.

6.2.3.1.18 UnsubscribeCompletionCallback()

```
virtual void UnsubscribeCompletionCallback(SubscriptionId subscriptionId)=0
```

Unsubscribes for callback upon asynchronous action completion.

Parameters:

[in] subscriptionId Identifier of the subscribed callback.

6.2.3.1.19 Wait()

```
virtual void Wait()=0
```

Wait for up to the default timeout for the previous asynchronous action to complete.

Details

Waits up to the default timeout for any active asynchronous action to complete.

Exceptions:

DeviceException if;

- Any action or attribute access results in a failure of any type
- An EStop was performed on the device.
- The default timeout is reached

6.2.3.1.20 Wait()

```
virtual void Wait(uint32_t timeout_ms)=0
```

Wait for the previous asynchronous action to complete.

Details

Waits for any active asynchronous action to complete.

Parameters:

[in] timeout_ms The length of time to wait before timing out.

Exceptions:

DeviceException if;

- Any action or attribute access results in a failure of any type
- An EStop was performed on the device.
- Timeout is reached
- timeout_ms is > the maximum action timeout (default of 2 minutes)

6.2.3.2 Public Variables

6.2.3.2.1 InvalidSubscriptionId

```
static const SubscriptionId InvalidSubscriptionId
```

6.2.4 imp::IDigitalInputDevice Class

```
#include <IDigitalInputDevice.h>

class IDigitalInputDevice : public imp::IDigitalIOReader
```

Defines the interface a Digital Input device must adhere to for building motion control systems within CCP.

Details

Using virtual inheritance here caused issues when using `dynamic_cast` to this type from `ICCPDevice` ; see the `GetDevice` method of `LuaDevice` for details.

6.2.5 imp::IDigitalIOPublisher Class

```
#include <IDigitalIOPublisher.h>

class IDigitalIOPublisher : public virtual imp::ICCPDevice
```

Defines the interface a DigitalIO device must adhere to for allowing clients to subscribe for change notifications.

Details

We inherit from `ICCPDevice` here because `LuaDevice` had issues when multiple/virtual inheritance was used in derived classes; see `IDigitalInputDevice` and `IDigitalOutputDevice` for more details.

6.2.5.1 Public Methods

6.2.5.1.1 ChangeNotificationEnabled()

```
virtual bool ChangeNotificationEnabled()=0
```

Queries whether change notification is enabled.

Returns:

true if change notification is enabled

Exceptions:

`std::runtime_error` if the device does not support change notification

6.2.5.1.2 ChangeNotificationSupported()

```
virtual bool ChangeNotificationSupported() const =0
```

Queries whether the device supports change notification.

Returns:

true if the device supports change notification

6.2.5.1.3 DisableChangeNotification()

```
virtual void DisableChangeNotification()=0
```

Disables notification of change.

Details

Note A Wait() call should follow this to determine if the notification was successfully disabled

Exceptions:

std::runtime_error if the device does not support change notification

6.2.5.1.4 EnableChangeNotification()

```
virtual void EnableChangeNotification()=0
```

Enables notification of change.

Details

Note A Wait() call should follow this to determine if the notification was successfully enabled

Exceptions:

std::runtime_error if the device does not support change notification

6.2.5.1.5 SubscribeChangeNotification()

```
virtual SubscriptionId SubscribeChangeNotification(const DigitalIOChangeSubscription &subscription)=0
```

Subscribe for notification of change.

Parameters:

[in] subscription details of the DigitalIO change subscription being made

Returns:

A unique identifier allowing the subscription to be unsubscribed

Exceptions:

std::runtime_error if the device does not support change notification

6.2.5.1.6 UnsubscribeChangeNotification()

```
virtual bool UnsubscribeChangeNotification(const SubscriptionId &subscriptionId)=0
```

Unsubscribe for notification of change.

Parameters:

[in] subscriptionId The unique identifier of the subscription as returned by
SubscribeChangeNotification

Returns:

true if the a subscription with the given ID was successfully unsubscribed

Exceptions:

std::runtime_error if the device does not support change notification

6.2.6 imp::IDigitalIOReader Class

```
#include <IDigitalIOReader.h>
```

```
class IDigitalIOReader : public virtual imp::IDigitalIOPublisher
```

Defines the interface a DigitalIO device must adhere to for allowing clients to read its digital information.

6.2.6.1 Public Methods

6.2.6.1.1 GetBitState()

```
virtual DigitalIOBitState GetBitState(DigitalIOBit bit,
                                     bool forceRefresh=false)=0
```

Gets the state of bit addressed by the bit index.

Parameters:

[in] bit Enum representing the DigitalIOBit

[in] forceRefresh Forces retrieval of value from the remote device as opposed to from a local cache.

Returns:

State of the DigitalIOBit

6.2.6.1.2 GetRegister()

```
virtual uint8_t GetRegister(bool forceRefresh=false)=0
```

Gets the state of all the bits as one byte.

Parameters:

[in] forceRefresh Forces retrieval of value from the remote device as opposed to from a local cache.

Returns:

Byte reflecting the states of the bits.

6.2.7 imp::IMotionControllerConfiguration Class

```
#include <IMotionControllerConfig.h>
```

```
class IMotionControllerConfiguration
```

An interface for reading and writing various motion controller settings from/to memory/NVRAM.

6.2.7.1 Public Methods

6.2.7.1.1 GetBiquadFilters()

```
virtual BiquadFilterSettings GetBiquadFilters()=0
```

Read the biquad filter settings from the motion controller's memory.

Returns:

The biquad filter settings.

6.2.7.1.2 GetCommunication()

```
virtual CommunicationSettings GetCommunication()=0
```

Read the current communication settings.

Details

These will correspond to the settings in the motion controller's memory and NVRAM, unless there has been a call to SetCommunication(). In this case, it will return the updated settings supplied to SetCommunication(), which are buffered, pending a call to WriteToNVRAM().

Returns:

The current communication settings.

6.2.7.1.3 GetCommutationParameter1()

```
virtual int32_t GetCommutationParameter1()=0
```

Get the current value associated with commutation parameter 1 from the PMD controller.

Details

Note that this value is equal to the PhaseAngle * Prescalar factor.

Returns:

The commutation parameter 1 value.

6.2.7.1.4 GetCurrentLoop()

```
virtual CurrentLoopSettings GetCurrentLoop()=0
```

Read the current loop settings from the motion controller's memory.

Returns:

The current loop settings.

6.2.7.1.5 GetElectricalProperties()

```
virtual ElectricalProperties GetElectricalProperties()=0
```

Get various electrical properties from the PMD controller.

Returns:

The electrical properties information.

6.2.7.1.6 GetFirmwareVersion()

```
virtual FirmwareVersionInfo GetFirmwareVersion()=0
```

Get firmware version from the PMD controller.

Returns:

The firmware version.

6.2.7.1.7 GetMotionError()

```
virtual MotionErrorSettings GetMotionError()=0
```

Read the current motion error settings from the motion controller's memory.

Returns:

The current motion error settings.

6.2.7.1.8 GetMotionTracking()

```
virtual MotionTrackingSettings GetMotionTracking()=0
```

Read the current motion tracking settings from the motion controller's memory.

Returns:

The current motion tracking settings.

6.2.7.1.9 GetPhaseAngle()

```
virtual PhaseAngleInfo GetPhaseAngle()=0
```

Get the current phase angle from the PMD controller.

Returns:

The phase angle information.

6.2.7.1.10 GetPositionLoop()

```
virtual PositionLoopSettings GetPositionLoop()=0
```

Read the position loop settings from the motion controller's memory.

Returns:

The position loop settings.

6.2.7.1.11 Restore()

```
virtual void Restore(std::string pathname)=0
```

Restore from a PMD configuration script to the Magellan IC's working memory. Note that the axis needs to be de-energized (disabled) before calling this function.

Parameters:

[in] pathname The path to the PMD configuration script file to read.

6.2.7.1.12 RestoreToNVRAM()

```
virtual void RestoreToNVRAM(std::string pathname)=0
```

Restore from a PMD configuration script to the Magellan IC's NVRAM configuration. Note that the axis needs to be de-energized (disabled) before calling this function.

Parameters:

[in] pathname The path to the PMD configuration script file to read.

6.2.7.1.13 Save()

```
virtual void Save(std::string pathname)=0
```

Save the Magellan IC's NVRAM configuration as a PMD configuration script. Note that the axis needs to be de-energized (disabled) before calling this function.

Parameters:

[in] pathname The path to the PMD configuration script to write.

6.2.7.1.14 SetBiquadFilters()

```
virtual void SetBiquadFilters(const BiquadFilterSettings &biquadFilterSettings)=0
```

Set the biquad filter settings in the motion controller's memory.

Parameters:

[in] `biquadFilterSettings` The new biquad filter settings to apply.

6.2.7.1.15 SetCommunication()

```
virtual void SetCommunication(const CommunicationSettings &communicationSettings)=0
```

Set the communication settings to be written to NVRAM (note the motion controller's memory is NOT updated).

Parameters:

[in] `communicationSettings` The new communication settings to apply when `WriteToNVRAM()` is called.

6.2.7.1.16 SetCommutationParameter1()

```
virtual void SetCommutationParameter1(int32_t commutationParameter1)=0
```

Set the commutation parameter 1 value to be written to the PMD controller.

Details

Note that this value should be equal to the `PhaseAngle` * `Prescalar` factor.

Parameters:

[in] `commutationParameter1` The value to set.

6.2.7.1.17 SetCurrentLoop()

```
virtual void SetCurrentLoop(const CurrentLoopSettings &currentLoopSettings)=0
```

Set the current loop settings in the motion controller's memory.

Parameters:

[in] `currentLoopSettings` The new current loop settings to apply.

6.2.7.1.18 SetMotionError()

```
virtual void SetMotionError(const MotionErrorSettings &motionErrorSettings)=0
```

Set the motion error settings in the motion controller's memory.

Parameters:

[in] `motionErrorSettings` The new motion error settings to apply.

6.2.7.1.19 SetMotionTracking()

```
virtual void SetMotionTracking(const MotionTrackingSettings &motionTrackingSettings)=0
```

Set the motion tracking settings in the motion controller's memory.

Parameters:

[in] `motionTrackingSettings` The new motion tracking settings to apply.

6.2.7.1.20 SetPhaseAngle()

```
virtual void SetPhaseAngle(uint16_t rawPhaseAngle)=0
```

Set the phase angle to be written to the PMD controller.

Parameters:

[in] `rawPhaseAngle` The raw phase angle to set.

6.2.7.1.21 SetPositionLoop()

```
virtual void SetPositionLoop(const PositionLoopSettings &positionLoopSettings)=0
```

Set the position loop settings in the motion controller's memory.

Parameters:

[in] positionLoopSettings The new position loop settings to apply.

6.2.7.1.22 WriteToNVRAM()

```
virtual void WriteToNVRAM(std::vector< MotionControllerConfigurationType > configurationTypes)=0
```

Transfer the specified types of configuration settings from the motion controller's memory to NVRAM so they will persist across power cycles.

Details

Note that the axis needs to be de-energized (disabled) before calling this function. If the active communication settings are changed, the communications link needs to be re-established (i.e. disconnect followed by a connect or initialize).

Parameters:

[in] configurationTypes The types of configuration settings to persist.

6.2.8 imp::MotionErrorSettings Class

```
#include <MotionErrorSettings.h>

class MotionErrorSettings
```

Defines the motion error settings (i.e. the position error limit and the action to take if a motion error occurs).

6.2.8.1 Public Methods**6.2.8.1.1 ToString()**

```
std::string ToString() const
```

6.2.8.2 Public Variables**6.2.8.2.1 ActionOnError**

```
MotionErrorAction ActionOnError= MotionErrorAction::None
```

The action to perform when a motion error occurs.

6.2.8.2.2 PositionErrorLimit

```
double PositionErrorLimit= 0.0
```

The position error limit in selected units (mm, um, or nm). If the actual position deviates from the commanded position by more than this value, a motion error will be generated.

6.2.9 imp::MotionTrackingSettings Class

```
#include <MotionTrackingSettings.h>

class MotionTrackingSettings
```

Defines when an axis move is determined to be complete by the motion controller.

6.2.9.1 Public Methods

6.2.9.1.1 ToString()

```
std::string ToString() const
```

6.2.9.2 Public Variables

6.2.9.2.1 Mode

```
MotionCompleteMode Mode
```

Determines how the motion controller determines a move is complete.

6.2.9.2.2 SettleTimeMs

```
double SettleTimeMs
```

The length of time an axis must remain within the settle window before the move is deemed to be complete. This setting is only applicable when the motion complete mode is set to 'MotionSettles'.

6.2.9.2.3 SettleWindow

```
double SettleWindow
```

The size of the settle window in selected units (mm, um, or nm). This setting is only applicable when the motion complete mode is set to 'MotionSettles'.

6.2.10 MotionSynergyAPI Class

```
#include <MotionSynergyAPI.h>
```

```
class MotionSynergyAPI : public IMotionSynergy
```

A simplified version of MotionSynergyAPI for C++ Applications responsible for creating the infrastructure (e.g. configuration, logging, etc) necessary to control one or more Axis devices.

Details

This class uses the RAII pattern to ensure the infrastructure is destroyed when the object instance goes out of scope.

6.2.10.1 Public Methods

6.2.10.1.1 Destructor: ~MotionSynergyAPI()

```
~MotionSynergyAPI() override
```

Destructor calls Shutdown() to ensure the infrastructure is destroyed.

6.2.10.1.2 Configure()

```
void Configure(const std::string &instrumentConfigPathname,
               const std::string &logConfigPathname,
               const std::string &logFolderPathname) override
```

Create the necessary infrastructure, read the instrument configuration and create the axis device instances.

Parameters:

[in] instrumentConfigPathname Path to the main 'instrument' configuration file.

[in] logConfigPathname Path to the logging configuration file.

[in] logFolderPathname Path to write log files to. *

Exceptions:

Various exceptions will be thrown in the case an error is encountered (e.g. cannot load the configuration).

6.2.10.1.3 GetAxis()

```
imp::IAxisDevice& GetAxis(const std::string &axisName) const override
```

Return a reference to an axis.

Details

Configure() must have been called first.

Parameters:

[in] axisName Name of the axis to return.

Exceptions:

std::runtime_error if the axis cannot be found (i.e. axisName doesn't match an axis defined in configuration).

6.2.10.1.4 GetConfiguration()

```
IMotionSynergyConfig& GetConfiguration() override
```

6.2.10.1.5 GetDigitalInput()

```
imp::IDigitalInputDevice& GetDigitalInput(const std::string &digitalInputName) const override
```

Return a reference to a digital input.

Details

Configure() must have been called first.

Parameters:

[in] digitalInputName Name of the digital input to return.

Exceptions:

std::runtime_error if the digital input cannot be found (i.e. digitalInputName doesn't match a digital input defined in configuration).

6.2.10.1.6 GetDispatcher()

```
imp::IDispatcherSPtr GetDispatcher() override
```

Get a pointer to the main dispatcher interface. Creates the dispatcher and associated thread on the first call to this method. Subsequent calls will return the same instance.

Returns:

The main dispatcher.

6.2.10.1.7 GetFirstProduct()

```
std::string GetFirstProduct(std::vector< std::string > &productAxesNames) override
```


Determine the first product listed in the configuration file and its associate axis/axes.

Parameters:

[out] productAxesNames Output parameter which will contain a list of axes associated with this product.

Returns:

The product type and a list of associated axes on success, 'Undefined' and an empty list on failure.

6.2.10.1.8 GetLogger()

```
imp::ILogger& GetLogger() override
```

6.2.10.1.9 GetProductList()

```
std::vector<ProductInfo> GetProductList() override
```

6.2.10.1.10 IsDigitalInputEnabled()

```
bool IsDigitalInputEnabled(const std::string &digitalInputName) const override
```

Check if a digital input has been enabled in configuration and is available for use.

Details

Configure() must have been called first.

Parameters:

[in] digitalInputName Name of the digital input to check.

Returns:

true if the digital input is available for use, false if the digital input cannot be found or is still disabled in configuration.

6.2.10.1.11 Shutdown()

```
void Shutdown() override
```

Shut down the infrastructure in a controlled manner.

6.2.11 imp::IAxisDevice::Movement Struct

```
#include <IAxisDevice.h>
```

```
struct Movement
```

Contains the information necessary to describe an axis movement, primarily for the purposes of emulation.

6.2.11.1 Public Variables

6.2.11.1.1 Acceleration

```
double Acceleration
```

The maximum acceleration of the axis when the movement started.

6.2.11.1.2 Deceleration

```
double Deceleration
```

The maximum deceleration of the axis when the movement started.

6.2.11.1.3 Jerk

`double` Jerk

The maximum jerk of the axis when the movement started.

6.2.11.1.4 PositionEnd

`double` PositionEnd

The (requested) position of the axis when the movement finishes.

6.2.11.1.5 PositionStart

`double` PositionStart

The position of the axis before the movement started.

6.2.11.1.6 Velocity

`double` Velocity

The maximum velocity of the axis when the movement started.

6.2.12 imp Namespace

6.2.12.1 Public Functions

6.2.12.1.1 operator<<()

```
std::ostream& operator<<(std::ostream &stream,
                          const CommunicationSettings &value)
```

6.2.12.1.2 BitMask()

```
uint8_t BitMask(DigitalIOBit bit)
```

6.2.12.1.3 operator<<()

```
std::ostream& operator<<(std::ostream &stream,
                          const MotionErrorSettings &value)
```

6.2.12.1.4 operator<<()

```
std::ostream& operator<<(std::ostream &stream,
                          const MotionTrackingSettings &value)
```

6.2.12.2 Enumerations

6.2.12.2.1 imp::AxisRecoveryOperation

```
enum imp::AxisRecoveryOperation
{
    NoneRequired, // The axis is already in a state where it can perform moves and no recovery action is
                  // required.

    ResetPosition, // The axis has encountered a position error and ResetPosition() must be called to
                  // reset its commanded position to match its actual position.

    ResetPositionAndEnable, // The axis has encountered a position error and the motor has been disabled
                  // (de-energized). ResetPosition() must be called to reset its commanded position to match its actual
                  // position, followed by Enable() to re-energize the motor.

    Initialize, // The axis has encountered a general error and must be re-initialized by calling
               // Initialize().
```

```
    LAST_ENUM,
}
```

Defines the recovery operation required return the axis to a state where it can performs moves.

6.2.12.2.2 imp::CanBaudRate

```
enum imp::CanBaudRate
{
    Baud1000000,
    Baud800000,
    Baud500000,
    Baud250000,
    Baud125000,
    Baud50000,
    Baud20000,
    Baud10000,
    LAST_ENUM,
}
```

Defines the set of baud rates that can be used for CAN communications.

6.2.12.2.3 imp::DigitalIOBitState

```
enum imp::DigitalIOBitState
{
    LOW,
    HIGH,
    LAST_ENUM,
}
```

Enumeration that represent the state of a Digital IO bit.

6.2.12.2.4 imp::DigitalIOBit

```
enum imp::DigitalIOBit
{
    ZERO,
    ONE,
    TWO,
    THREE,
    FOUR,
    FIVE,
    SIX,
    SEVEN,
    LAST_ENUM,
}
```

Enumeration to represent each bit of a Digital IO register.

Details

Used as zero-based indices into an N-bit-wide register.

6.2.12.2.5 imp::DigitalOutputTrigger

```
enum imp::DigitalOutputTrigger
{
    Disabled, // Digital output is set low and triggers are disabled (i.e. the digital output state is
              // not affected by the axis state).

    InMotion, // The digital output will be set high whenever the axis is in motion.

    MotionComplete, // The digital output will be set high whenever the axis completes a move.

    AtMaxVelocity, // The digital output will be set high when the axis reaches maximum velocity during
                  // its move.

    Immediate, // The digital output will be set high immediately (i.e. the digital output is directly
              // controlled by software).
```

```
    LAST_ENUM,
}
```

Defines the set of triggers which can be used to set the AxisOut1 general purpose output (GPO).

6.2.12.2.6 `imp::MotionCompleteMode`

```
enum imp::MotionCompleteMode
{
    CommandedPositionReached, // A move is complete as soon as the commanded position has been reached.
    The axis settle time and window is ignored.

    MotionSettles, // A move is completed once the axis has reached the commanded position and has
    maintained position within the settle window for the length of the settle time.

    LAST_ENUM,
}
```

Defines when the motion controller determines a move has completed.

6.2.12.2.7 `imp::MotionControllerConfigurationType`

```
enum imp::MotionControllerConfigurationType
{
    None, // Invalid motion controller configuration type.

    MotionTracking, // The motion tracking configuration settings (see MotionTrackingSettings).

    MotionError, // The difference between the actual and commanded position (see MotionErrorSettings).

    PositionLoop, // The position loop settings (see PositionLoopSettings).

    ActiveCommunication, // The active communication configuration settings (see CommunicationSettings).
    This requires the communications link to be re-established (i.e. disconnect / connect).

    InactiveCommunication, // The inactive communication configuration settings (see
    CommunicationSettings). Does not affect the current communications link.

    CurrentLoop, // The current loop settings (see CurrentLoopSettings).

    BiquadFilter, // The biquad filter settings (see BiquadFilterSettings).

    LAST_ENUM,
}
```

Defines the different types of motion controller configuration settings which can be written to NVRAM using the `IMotionControllerConfiguration.WriteToNVRAM` method.

6.2.12.2.8 `imp::MotionErrorAction`

```
enum imp::MotionErrorAction
{
    None, // Motion errors are ignored and no action is taken.

    AbruptStop, // The axis is brought to an abrupt stop.

    SmoothStop, // The axis is brought to a smooth stop using the current deceleration setting.

    DisableVelocityLoop, // Disable velocity loop and higher modules. Note this is currently not
    supported by SmartStage.

    DisablePositionLoop, // Disable position loop and higher modules.

    DisableCurrentLoop, // Disable current loop and higher modules.

    DisableMotor, // The motor is disabled (de-energized).

    AbruptStopWithPosErrorClear, // Abrupt stop with position error clear.

    PassiveBraking, // Passive braking. Note this is currently not supported by SmartStage.
}
```

```
    LAST_ENUM,
}
```

Defines the action performed by the motion controller when a motion error (i.e. position error) is detected.

6.2.12.2.9 imp::SerialBaudRate

```
enum imp::SerialBaudRate
{
    Baud1200,
    Baud2400,
    Baud9600,
    Baud19200,
    Baud57600,
    Baud115200,
    Baud230400,
    Baud460800,
    LAST_ENUM,
}
```

Defines the set of baud rates that can be used for Serial communications.

6.2.12.2.10 imp::SerialProtocol

```
enum imp::SerialProtocol
{
    PointToPoint, // RS232 point-to-point

    MultiDropUsingIdleLineDetection, // RS485 Multi-drop using idle-line detection

    LAST_ENUM,
}
```

Defines the protocol to use for serial communications.

6.2.12.2.11 imp::TraceTrigger

```
enum imp::TraceTrigger
{
    Immediate, // Trace data capture begins immediately and continues until StopTraceCapture() is
               // called.

    OnMotionStart, // Trace data capture begins at the start of the next move and ends when the move
                  // completes.

    OnMotionStartDelayed, // Trace data capture begins at a delayed time after the start of the next
                          // move and ends when the move completes. This is only supported by the UDP sine wave move.

    OnMotionStartNoEnd, // Trace data capture begins at the start of the next move and continues until
                       // StopTraceCapture() is called.

    LAST_ENUM,
}
```

Defines the trigger to begin a trace data capture.

6.2.12.2.12 imp::TraceVariable

```
enum imp::TraceVariable
{
    None, // Used to terminate the list of trace variables to capture.

    PositionError, // The difference between the actual and commanded position.

    CommandedPosition, // The commanded position output from the profile generator.

    CommandedVelocity, // The commanded velocity output from the profile generator.

    CommandedAcceleration, // The commanded acceleration output from the profile generator.

    ActualPosition, // The actual position of the motor.
}
```

`ActualVelocity`, // The actual velocity (calculated using a simple low-pass filter).

`ActiveMotorCommand`, // The instantaneous motor command.

`MotionControlICTime`, // The motion control IC time (units of servo cycles).

`PositionCaptureRegister`, // The contents of the high-speed Capture register.

`PositionLoopIntegratorSum`, // The integral value used in the position loop PID filter.

`PositionLoopIntegratorContribution`, // The contribution of the integral portion of the PID filter.

`PositionLoopDerivative`, // The derivative value of the position loop PID filter.

`Biquad1Input`, // The value input to biquad 1 filter.

`Biquad2Input`, // The value input to biquad 2 filter.

`EventStatusRegister`, // The Event Status register.

`ActivityStatusRegister`, // The Activity Status register.

`SignalStatusRegister`, // The Signal Status register.

`DriveStatusRegister`, // The Drive Status register.

`DriveFaultStatusRegister`, // The Drive Fault Status register.

`PhaseAngle`, // The motor phase angle.

`PhaseOffset`, // The phase offset.

`PhaseACommand`, // The output command for Phase A.

`PhaseBCommand`, // The output command for Phase B.

`PhaseCCommand`, // The output command for Phase C.

`AnalogInput0`, // The most recently read value from the Analog0 signal.

`AnalogInput1`, // The most recently read value from the Analog1 signal.

`AnalogInput2`, // The most recently read value from the Analog2 signal.

`AnalogInput3`, // The most recently read value from the Analog3 signal.

`AnalogInput4`, // The most recently read value from the Analog4 signal.

`AnalogInput5`, // The most recently read value from the Analog5 signal.

`AnalogInput6`, // The most recently read value from the Analog6 signal.

`AnalogInput7`, // The most recently read value from the Analog7 signal.

`PhaseAngleScaled`, // The normalized phase angle, scaled from 0 to 360 deg (0 to 0x7fff) rather than in encoder counts.

`PhaseAReference`, // The current loop reference for phase A.

`PhaseAError`, // The current loop error for phase A.

`PhaseAActualCurrent`, // The current loop actual current for phase A.

`PhaseAIntegratorSum`, // The current loop integral sum for phase A.

`PhaseAIntegratorContribution`, // The current loop integrator contribution to PI filter for phase A.

`CurrentLoopAOutput`, // The current loop output for phase A.

`PhaseBReference`, // The current loop reference for phase B.

PhaseBError, // The current Loop Error for phase B.

PhaseBAActualCurrent, // The current Loop actual current for phase B.

PhaseBIntegratorSum, // The current Loop integral sum for phase B.

PhaseBIntegratorContribution, // The current Loop integrator contribution to PI filter for phase B.

CurrentLoopBOutput, // The current Loop output for phase B.

FOCDReference, // The FOC reference for D (direct) Loop.

FOCDError, // The FOC D (direct) Loop error.

FOCDFeedback, // The FOC D (direct) feedback current.

FOCDIntegratorSum, // The FOC integral for D (direct) Loop.

FOCDIntegratorContribution, // The FOC integrator contribution for D (direct) Loop.

FOCDOutput, // The FOC output for D (direct) Loop.

FOCQReference, // The FOC reference for Q (quadrature) Loop.

FOCQError, // The FOC Q (quadrature) Loop error.

FOCQFeedback, // The FOC Q (quadrature) feedback current.

FOCQIntegratorSum, // The FOC integral for Q (quadrature) Loop.

FOCQIntegratorContribution, // The FOC integrator contribution for Q (quadrature) Loop.

FOCQOutput, // The FOC output Q (quadrature) Loop.

FOCAlphaOutput, // The FOC output for phase Alpha.

FOCBetaOutput, // The FOC output for phase Beta.

BusVoltage, // The bus voltage.

Temperature, // The temperature of the drive's output stage.

FoldbackEnergy, // The i2t total foldback energy.

LegACurrent, // Current1A reading

LegBCurrent, // Current1B reading

LegCCurrent, // Current1C reading

LegDCurrent, // Current1D reading

PWMOutputA, // Phase A PWM output

PWMOutputB, // Phase B PWM output (used with Brushless DC motors only)

PWMOutputC, // Phase C PWM output (used with Brushless DC and step motors only)

ContourOutput, // UDP active profile output based on current master source index and corresponding position in profile table.

ContourOffset, // UDP active profile offset (add to ContourOutput to calculate axis target position).

SignalStatus_EncoderA, // Signal Status bit 0 on axis 1

SignalStatus_EncoderB, // Signal Status bit 1 on axis 1

SignalStatus_EncoderIndex, // Signal Status bit 2 on axis 1

SignalStatus_CaptureInput, // Signal Status bit 3 on axis 1

```

SignalStatus_Gate, // Signal Status bit 3 on axis 1 (alternative name for TOP use)
SignalStatus_EncoderHome, // Signal Status bit 3 on axis 1 (treated as a flag)
SignalStatus_PositiveLimit, // Signal Status bit 4 on axis 1
SignalStatus_NegativeLimit, // Signal Status bit 5 on axis 1
SignalStatus_AxisIn, // Signal Status bit 6 on axis 1
SignalStatus_AxisOut, // Signal Status bit 10 on axis 1
SignalStatus_FaultOut, // Signal Status bit 14 on axis 1
SignalStatus_GPI1, // Signal Status Encoder A (bit 0) on axis 2
SignalStatus_GPI2, // Signal Status Encoder B (bit 1) on axis 2
SignalStatus_Trigger, // Signal Status Trigger (bit 2 & 3) on axis 2.
ActivityStatus_PhasingInitialized, // Activity Status - PhasingInitialized (bit 0)
ActivityStatus_AtMaximumVelocity, // Activity Status - AtMaximumVelocity (bit 0)
ActivityStatus_Tracking, // Activity Status - Tracking (bit 0)
ActivityStatus_CurrentProfileMode, // Activity Status - Current Profile Mode (bit 3-5)
ActivityStatus_Reserved, // Activity Status - Reserved
ActivityStatus_AxisSettled, // Activity Status - Axis Settled (bit 7)
ActivityStatus_MotorOn, // Activity Status - Position Loop Enabled (bit 8)
ActivityStatus_PositionCapture, // Activity Status - Position Capture (bit 9)
ActivityStatus_InMotion, // Activity Status - In-motion (bit 10)
ActivityStatus_InPositiveLimit, // Activity Status - In Positive Limit (bit 11)
ActivityStatus_InNegativeLimit, // Activity Status - In Negative Limit (bit 12)
ActivityStatus_ProfileSegment, // Activity Status - Profile Segment (bit 13-15)
LAST_ENUM,
}

```

Defines the set of trace variables which can be captured during a move. See `EnableTraceCapture()`.

6.2.12.3 Typedefs

6.2.12.3.1 DigitalIORegisterCallback

```
using DigitalIORegisterCallback = typedef std::function<void( uint8_t registerValue )>
```

6.2.12.3.2 DigitalIOBitCallback

```
using DigitalIOBitCallback = typedef std::function<void( DigitalIOBit bit, DigitalIOBitState bitState )>
```

6.2.12.3.3 IDispatcherSPtr

```
typedef std::shared_ptr< IDispatcher > IDispatcherSPtr
```

6.2.12.3.4 IConfigurationSPtr

```
typedef std::shared_ptr< IConfiguration > IConfigurationSPtr
```

6.2.12.3.5 ICCPDeviceSPtr

```
using ICCPDeviceSPtr = typedef std::shared_ptr<ICCPDevice>
```


6.2.12.3.6 IDispatcherThreadSPtr

```
using IDispatcherThreadSPtr = typedef std::shared_ptr<IDispatcherThread>
```

7 Linux Support

Motion Synergy has been tested with Ubuntu 22.04 LTS for x64 and arm64 platforms. The following information describes how to configure your Linux system to build and run the Motion Synergy software on Ubuntu.

7.1 Install Developer Tools

The following additional tools need to be installed to build and run the MotionSynergyAPI on Ubuntu 22.04 x64.

- Upgrade your CMake version to 3.27.2 (From a Linux shell)

```
sudo snap install cmake --classic
```

- Setup Python and Pip (From a Linux shell)

```
sudo apt update
sudo apt install python-is-python3
sudo apt install pip
```

- Add the Microsoft Package Repository

```
# Update the list of packages
sudo apt update
# Install pre-requisite packages.
sudo apt install -y wget apt-transport-https software-properties-common
# Download the Microsoft repository GPG keys
wget -q "https://packages.microsoft.com/config/ubuntu/$(lsb_release -rs)/packages-microsoft-prod.deb"
# Register the Microsoft repository GPG keys
sudo dpkg -i packages-microsoft-prod.deb
# Delete the the Microsoft repository GPG keys file
rm packages-microsoft-prod.deb
# Update the list of packages after we added packages.microsoft.com
sudo apt update
```

- Install .NET 8.0

```
sudo apt install -y dotnet-sdk-8.0
```

If the above reports it cannot find any suitable packages, you will need to install it using Microsoft's install script. This is needed when installing on Ubuntu 20.04 on ARM.

```
cd ~
wget https://dot.net/v1/dotnet-install.sh -O dotnet-install.sh
chmod +x ./dotnet-install.sh
./dotnet-install.sh
```

If you've installed it using the above install script (rather than the via package manager), you will also need to set two environment variables. This can be done by adding the following lines to your .bashrc file.

```
export DOTNET_ROOT=$HOME/.dotnet
export PATH=$PATH:$DOTNET_ROOT:$DOTNET_ROOT/tools
```

To verify dotnet is installed correctly and can be found from the command line, enter the following command:

```
dotnet --version
```

It should report the current version, e.g:

```
6.0.416
```

- Install Powershell

```
# Install PowerShell
sudo apt install -y powershell
# Start PowerShell
pwsh
```

If the above reports it cannot find any suitable packages, you will need to install it manually:

```
cd ~
wget https://github.com/PowerShell/PowerShell/releases/download/v7.3.9/powershell-7.3.9-linux-arm64.tar.gz
mkdir powershell
cd powershell/
tar -xf ../powershell-7.3.9-linux-arm64.tar.gz
cd ..
```

If you've installed it manually (rather than the via package manager), you will also need add it to the \$PATH environment variable. This can be done by adding the following lines to your .bashrc file.

```
export PATH=$PATH:~/powershell
```

- Install ARM64 cross compiler for Raspberry Pi Builds. This is only needed if you are going to target *arm64* architectures.

```
sudo apt install gcc-aarch64-linux-gnu g++-aarch64-linux-gnu binutils-aarch64-linux-gnu
```

7.2 Linux Drivers

Motion Synergy has been tested with the following Linux drivers:

- IXXAT USB-CAN adapter
- MOXA RS232/RS485 USB to Serial drivers
- Raspberry Pi CAN-HAT
- Raspberry Pi RS-485 HAT

7.2.1 Installing the IXXAT CAN driver for Raspberry Pi

The IXXAT CAN Kernel drivers need to be built from source and installed.

- Copy the Drivers/IXXAT_SocketCAN_2_0_378_Modified_2023-03-15.tar archive to the Raspberry Pi
- Extract the tar file contents. e.g. `tar -xf IXXAT_SocketCAN_2_0_378_Modified_2023-03-15.tar`.
- Install gcc if it's not already installed: `sudo apt install gcc`.
- Install make if it's not already installed: `sudo apt install make`.
- Compile the kernel module by running make in the ix_usb_can_2.0.367-MOD folder: `make all`
- Install the kernel module: `sudo make install`

Note the CAN network needs to be brought up before use, by running the 'ip link' command. See the Configuration section for more details.

7.2.2 Installing the MOXA Serial driver for Raspberry Pi

The MOXA Serial drivers need to be installed.

- Copy the Drivers/moxa-uport-1100-series-linux-kernel-6.x-driver-v6.0.tgz archive to the Raspberry Pi
- Extract the tar file contents. e.g. `tar -xf moxa-uport-1100-series-linux-kernel-6.x-driver-v6.0.tgz`.
- Change directory to the extracted folder `cd mxu11x0/`.
- Run the installer `sudo ./mxinstall`.
- Reboot the RaspberryPi
- The MOXA should now be mounted at `/dev/ttyUSB0` or similar.
- Remember to update the Instrument.cfg setting `SerialComms1Options.hwid` to `/dev/ttyUSB0`.

8 Appendix

8.1 NVRAM Support

The Motion Controller chip used to control each axis has a block of NVRAM which is used to store a list of commands which are executed at power on by the Motion Controller.

Motion Synergy supports reading and writing the following commands from/to NVRAM:

- SetSampleTime
- SetSPIMode
- SetSerialPortMode
- SetCANMode
- SetMotorType
- SetEventAction
- ReSetEventStatus
- SetAnalogCalibration
- SetDrivePWM
- SetPositionErrorLimit
- SetSettleTime
- SetSettleWindow
- SetTrackingWindow
- SetGearMaster
- SetGearRatio
- SetPositionLoop
- SetMotorBias
- SetMotorLimit
- SetMotorCommand
- SetMotionCompleteMode
- SetCaptureSource
- SetBreakpointValue
- SetBreakpoint
- SetAuxiliaryEncoderSource
- SetPWMFrequency
- SetCurrentControlMode
- SetFOC

- SetCurrentLoop
- SetAxisOutMask
- SetFaultOutMask
- SetDriveFaultParameter
- SetCurrentFoldback
- SetPhaseCounts
- SetCommutationMode
- SetPhaseCorrectionMode
- SetPhaseInitializeMode
- SetPhaseInitializeTime
- SetPhasePrescale
- SetBreakpointUpdateMask
- SetInterruptMask
- SetPhaseOffSet
- SetProfileMode
- SetUpdateMask
- SetSynchronizationMode
- SetCommutationParameter (parameter 0 only)
- SetOutputMode
- SetOperatingMode
- Update
- SetEncoderSource
- SetSignalSense

Motion Synergy recognises the following commands and will generate a warning when read from NVRAM but will not permit them to be written to NVRAM:

- AdjustActualPosition
- CalibrateAnalog
- ClearDriveFaultStatus
- ClearInterrupt
- ClearPositionError
- DriveNVRAM
- ExecutionControl
- InitializePhase

- MultiUpdate
- NoOperation
- Reset
- RestoreOperatingMode
- SetAcceleration
- SetActualPosition
- SetActualPositionUnits
- SetAutoStopMode
- SetBufferLength
- SetBufferReadIndex
- SetBufferStart
- SetBufferWriteIndex
- SetCurrent
- SetDeceleration
- SetDefault
- SetDriveCommandMode
- SetEncoderModulus
- SetEncoderToStepRatio
- SetFeedbackParameter
- SetJerk
- SetOvertemperatureLimit
- SetPhaseAngle
- SetPhaseParameter
- SetPosition
- SetProfileParameter
- SetStartVelocity
- SetStepRange
- SetStopMode
- SetTraceMode
- SetTracePeriod
- SetTraceStart
- SetTraceStop
- SetVelocity

- SetVelocityLoop
- WriteBuffer
- WriteIO

All other commands will not be recognised and will generate an error during a read from NVRAM or a write to NVRAM.

Additionally, MotionSynergy requires certain command(s) to be executed **after** another:

- SetCommutationParameter: must appear after SetPhaseCounts and SetPhasePrescale

MotionSynergy will not write to NVRAM a list containing an out-of-order command, and will generate an error.

8.2 3rd-Party Software

The MotionSynergyAPI software depends on a number of 3rd-party software libraries. These libraries are documented in *Dover Motion 3rd-party Software Libraries Description.docx*.

8.3 PC Specification

The MotionSynergyAPI software has been designed to run on a standard Windows 10 64-bit computer with the following minimum specification.

Resource	Minimum Specification
CPU	64-bit 2x Core 1.2Ghz
Main Memory	2 Gb
Available Disk Capacity	10Gb
Operating System	Windows 10

The MotionSynergyAPI software has been designed to run on a Linux 64-bit computer with the following minimum specification.

Resource	Minimum Specification
CPU	64-bit 2x Core 1.2Ghz, x64 or arm64
Main Memory	2 Gb
Available Disk Capacity	10Gb
Operating System	Ubuntu 22.04

8.4 Referenced Documents

Documents that are specifically referenced within this document are listed in the table below:

Document Title

Dover Motion 3rd-party Software Libraries Description.docx

8.5 Definitions

Acronym	Definition
---------	------------

PMD	Performance Motion Devices - company that supplies the Magellan motion control chips used in the <i>SmartStage™</i> product range.
-----	--

8.6 Revision History

Issue	Date	By	Description
A	08 MAR 2022	Mark Gladding	Initial revision.

8.7 Part Numbers

Part Number	Part
-------------	------

41-1612	Motion Synergy API User Manual (i.e. this document)
---------	---

8.8 Support

You can request software support using the following link:

[Request Software Support](#)